

TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
KHOA SƯ PHẠM TOÁN – TIN



ThS. LÊ MINH THU' (Chủ biên)
ThS. NGUYỄN THỊ MỸ DUNG

Bài giảng

NGÔN NGỮ LẬP TRÌNH JAVA

Đồng Tháp, 05/2021

LỜI NÓI ĐẦU

Bài giảng “**Ngôn ngữ lập trình Java**” là tài liệu hướng dẫn học tập dùng cho sinh viên ngành Khoa học máy tính, Sư phạm Tin học ở bậc đại học của Trường Đại học Đồng Tháp. Bài giảng này trình bày những vấn đề từ cơ bản đến nâng cao của ngôn ngữ lập trình Java. Các nội dung được giới thiệu một cách súc tích, có nhiều ví dụ minh họa. Cuối mỗi chương đều có câu hỏi hoặc bài tập để người học củng cố những vấn đề đã tìm hiểu.

Nội dung của bài giảng được chia thành 4 chương như sau:

- *Chương 1: Các thành phần cơ bản của Java.* Trình bày lịch sử hình thành, khái niệm cơ bản về Java. Kiến trúc chương trình Java và thực thi chương trình Java trên môi trường NetBeans.

- *Chương 2: Kiểu dữ liệu và cấu trúc lệnh cơ bản.* Trình bày các khái niệm, cách khai báo về kiểu dữ liệu. Phương thức sử dụng cấu trúc lệnh cơ bản như: lệnh lựa chọn if..else, switch..case; cấu trúc vòng lặp for, while, do..while; các lệnh đặc biệt break, continue;....

- *Chương 3: Lập trình hướng đối tượng.* Trình bày các khái niệm cơ bản về lớp, đối tượng, phương thức. Cách thức xây dựng lớp, sử dụng đối tượng, kế thừa, đa hình trong lập trình.

- *Chương 4: Lập trình giao diện.* Trình bày công cụ hỗ trợ thiết kế giao diện, quản lý Layout, và một số giao diện đồ họa người dùng (GUI) cơ bản.

Nhóm biên soạn hy vọng tài liệu này sẽ có ích cho việc dạy và học môn Ngôn ngữ lập trình Java ngày càng tốt hơn. Trong quá trình biên soạn có thể còn nhiều thiếu sót, nhóm biên soạn rất mong nhận được sự góp ý của người đọc cũng như các đồng nghiệp. Chúng tôi chân thành cảm ơn!

Đồng Tháp, Tháng 05 năm 2021

Nhóm biên soạn

DANH MỤC TỪ VIẾT TẮT

<i>Từ viết tắt</i>	<i>Viết đầy đủ</i>
IDE	Integrated Development Environment – Môi trường phát triển tích hợp
JDK	Java Development Kit – Công cụ phát triển cho ứng dụng Java
OOP	Object Oriented Programming – Lập trình hướng đối tượng
JFC	Java Foundation Classes – Tập hợp giao diện
GUI	Graphical User Interface – Giao diện đồ họa người dùng
API	Application Programming Interface – Giao diện lập trình ứng dụng
AWT	Abstract Windows Toolkit – Giao diện lập trình ứng dụng tiêu chuẩn

MỤC LỤC

CHƯƠNG 1. CÁC THÀNH PHẦN CƠ BẢN CỦA JAVA.....	1
1.1. LÀM QUEN VỚI JAVA.....	1
1.1.1. Nguồn gốc và lịch sử phát triển của Java	1
1.1.2. Đặc điểm và thế mạnh của Java	2
1.1.3. Các ứng dụng Java.....	3
1.1.4. Java platform và cơ chế thực thi.....	4
1.2. CẤU TRÚC CHƯƠNG TRÌNH JAVA.....	5
1.2.1. Khai báo package.....	6
1.2.2. Khai báo thư viện.....	6
1.3. CHƯƠNG TRÌNH JAVA ĐẦU TIÊN	7
1.3.1. Trình dịch JDK.....	7
1.3.2. Trình soạn thảo NetBeans IDE.....	8
1.3.3. Chương trình Java đầu tiên	8
1.4. CÂU HỎI ÔN TẬP.....	10
CHƯƠNG 2. KIỂU DỮ LIỆU VÀ CẤU TRÚC LỆNH CƠ BẢN CỦA JAVA	12
2.1. CÁC KIỂU DỮ LIỆU CƠ SỞ.....	12
2.1.1. Kiểu số nguyên	12
2.1.2. Kiểu dấu chấm động	13
2.1.3. Kiểu ký tự (char).....	13
2.1.4. Kiểu luận lý (boolean)	14
2.2. BIẾN VÀ HẰNG SỐ	14
2.2.1. Biến – Variables	14
2.2.2. Hằng số – Constants	14
2.3. TOÁN TỬ VÀ BIỂU THỨC	15
2.3.1. Toán tử số học	15
2.3.2. Gán và gán mở rộng.....	15
2.3.3. So sánh	16
2.3.4. Logic	16
2.3.5. Tăng, giảm một đơn vị.....	16
2.3.6. Toán tử trên bit	17
2.3.7. Một số phương thức lớp Math.....	18

2.4. CÁC PHƯƠNG THỨC VÀO RA CƠ BẢN.....	19
2.4.1. Xuất dữ liệu	19
2.4.2. Nhập dữ liệu từ bàn phím.....	20
2.5. LỆNH LỰA CHỌN	22
2.5.1. Câu lệnh if	22
2.5.2. Câu lệnh Switch	24
2.5.3. Biểu thức điều kiện	26
2.6. LỆNH LẶP	26
2.6.1. Vòng lặp while.....	26
2.6.2. Vòng lặp do – while	27
2.6.3. Vòng lặp for.....	27
2.7. MẢNG	29
2.7.1. Định nghĩa mảng.....	29
2.7.2. Khai báo mảng.....	30
2.7.3. Độ dài mảng.....	31
2.7.4. Duyệt các phần tử của mảng	31
2.7.5. Sao chép mảng.....	32
2.7.6. Lớp Arrays.....	33
2.7.7. Sắp xếp mảng.....	33
2.7.8. Mảng 2 chiều	35
2.8. CHUỖI.....	36
2.8.1. Khai báo chuỗi.....	36
2.8.2. Một số phương thức xử lý chuỗi.....	37
2.9. QUẢN LÝ NGOẠI LỆ	41
2.9.1. Khai báo ngoại lệ	41
2.9.2. Lấy thông tin ngoại lệ	43
2.10. CÂU HỎI ÔN TẬP VÀ BÀI TẬP THỰC HÀNH.....	44
2.10.1. Câu hỏi ôn tập.....	44
2.10.2. Bài tập thực hành	48
CHƯƠNG 3. LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG	51
3.1. TỔNG QUAN VỀ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG	51
3.1.1. Đối tượng – Object.....	51
3.1.2. Lớp đối tượng – Class	52
3.1.3. Đặc trưng của lập trình hướng đối tượng.....	53
3.2. XÂY DỰNG LỚP	55

3.2.1.	Khai báo lớp	55
3.2.2.	Định nghĩa thuộc tính dữ liệu.....	55
3.2.3.	Định nghĩa phương thức	56
3.2.4.	Từ khóa this.....	57
3.2.5.	Phương thức khởi tạo – Constructors	58
3.3.	SỬ DỤNG ĐỐI TƯỢNG.....	59
3.3.1.	Khai báo đối tượng	59
3.3.2.	Truy cập thuộc tính dữ liệu/phương thức	59
3.4.	ĐÓNG GÓI TRƯỜNG DỮ LIỆU	60
3.4.1.	Phương thức get.....	60
3.4.2.	Phương thức set	60
3.5.	KẾ THỪA VÀ ĐA HÌNH.....	61
3.5.1.	Tính kế thừa.....	61
3.5.2.	Tính đa hình.....	70
3.6.	CẤU TRÚC DỮ LIỆU TẬP HỢP (COLLECTIONS).....	71
3.6.1.	Giới thiệu.....	71
3.6.2.	ArrayList	74
3.7.	CÂU HỎI ÔN TẬP VÀ BÀI TẬP THỰC HÀNH.....	76
3.7.1.	Câu hỏi ôn tập.....	76
3.7.2.	Bài tập thực hành.....	79
CHƯƠNG 4.	LẬP TRÌNH GIAO DIỆN	82
4.1.	TỔNG QUAN VỀ JAVA SWING	82
4.1.1.	JFC là gì	82
4.1.2.	Java Swing.....	83
4.1.3.	JFrame	85
4.1.4.	JPanel	89
4.2.	CÁC THÀNH PHẦN GUI CƠ BẢN	95
4.2.1.	Swing Control.....	95
4.2.2.	JTextField.....	96
4.2.3.	JLabel	97
4.2.4.	JCheckBox	99
4.2.5.	JRadioButton	100
4.2.6.	JTextArea	101
4.2.7.	JPasswordField	102
4.2.8.	JButton	103

4.2.9.	Xử lý sự kiện	104
4.2.10.	Dialog	105
4.2.11.	JOptionPane	106
4.2.12.	JTable	108
4.3.	BÀI TẬP THỰC HÀNH	111

CHƯƠNG 1. CÁC THÀNH PHẦN CƠ BẢN CỦA JAVA

Mục tiêu của chương

Sau khi học xong chương này, người học sẽ:

- *Trình bày được những khái niệm cơ bản về Java, điểm mạnh của ngôn ngữ lập trình Java so với những ngôn ngữ lập trình khác;*
- *Mô tả cấu trúc của ngôn ngữ Java đơn giản; trình bày lại được các thành phần cấu thành một đoạn chương trình Java;*
- *Thực thi được đoạn chương trình Java trên môi trường phát triển NetBeans IDE.*

1.1. LÀM QUEN VỚI JAVA

1.1.1. Nguồn gốc và lịch sử phát triển của Java

1.1.1.1. Java là gì?

Java là ngôn ngữ lập trình hướng đối tượng (tựa C++) do Sun Microsystem đưa ra vào giữa thập niên 90.

Chương trình viết bằng ngôn ngữ Java có thể chạy trên bất kỳ hệ thống nào có cài máy ảo Java (Java Virtual Machine).

1.1.1.2. Lịch sử phát triển

Ngôn ngữ lập trình Java do James Gosling và các cộng sự của công ty Sun Microsystem phát triển.

Đầu thập niên 90, Sun Microsystem tập hợp các nhà nghiên cứu thành lập nên nhóm đặt tên là Green Team. Nhóm Green Team có trách nhiệm xây dựng công nghệ mới cho ngành điện tử tiêu dùng. Để giải quyết vấn đề này nhóm nghiên cứu phát triển đã xây dựng một ngôn ngữ lập trình mới đặt tên là Oak tương tự như C++ nhưng loại bỏ một số tính năng nguy hiểm của C++ và có khả năng chạy trên nhiều nền phần cứng khác nhau. Cùng lúc đó World Wide Web bắt đầu phát triển và Sun đã thấy được tiềm năng của ngôn ngữ Oak nên đã đầu tư cải tiến và phát triển. Sau đó không lâu ngôn ngữ mới với tên gọi là Java ra đời và được giới thiệu vào năm 1995.

Ngày nay, Java cho phép phát triển nhiều dạng ứng dụng khác nhau như cơ sở dữ liệu, mạng, Internet, game, viễn thông,...

1.1.2. Đặc điểm và thế mạnh của Java

1.1.2.1. Máy ảo Java (JVM – Java Virtual Machine)

Tất cả các chương trình muốn thực thi phải được biên dịch ra mã máy. Mã máy của từng kiến trúc CPU của mỗi máy tính là khác nhau vì vậy trước đây một chương trình sau khi được biên dịch có thể chạy được trên một kiến trúc CPU cụ thể nào đó. Đối với CPU Intel chúng có thể chạy trên các hệ điều hành như Microsoft Windows, Unix, Linux, OS/2...Chương trình thực thi được trên Windows được biên dịch dưới dạng file có đuôi .exe còn trên Linux thì được biên dịch dưới dạng file có đuôi .elf. Vì vậy trước đây một chương trình chạy được trên Windows muốn chạy được trên hệ điều hành khác như Linux thì phải chỉnh sửa và biên dịch lại. Ngôn ngữ lập trình Java ra đời, nhờ vào máy ảo Java mà khó khăn nêu trên đã được khắc phục. Một chương trình viết bằng ngôn ngữ lập trình Java sẽ được biên dịch ra mã của máy ảo Java (mã Java bytecodes). Sau đó máy ảo Java chịu trách nhiệm chuyển mã Java bytecodes thành mã máy tương ứng. Sun Microsystem chịu trách nhiệm phát triển các máy ảo Java chạy trên các hệ điều hành trên các kiến trúc CPU khác nhau.

1.1.2.2. Thông dịch

Java là ngôn ngữ lập trình vừa biên dịch vừa thông dịch. Chương trình nguồn viết bằng ngôn ngữ lập trình Java có đuôi *.java đầu tiên được biên dịch thành tập tin có đuôi *.class và sau đó sẽ được trình thông dịch thông dịch thành mã máy.

1.1.2.3. Độc lập nền

Một chương trình viết bằng ngôn ngữ Java có thể chạy trên nhiều máy tính có hệ điều hành khác nhau (Windows, Unix, Linux...) miễn sao ở đó có cài đặt máy ảo Java (Java Virtual Machine). Viết một lần chạy mọi nơi (Write one – Run anywhere).

1.1.2.4. Hướng đối tượng

Hướng đối tượng trong Java tương tự như C++ nhưng Java là một ngôn ngữ lập trình hướng đối tượng hoàn toàn. Tất cả mọi thứ đề cập đến trong Java đều liên quan đến các đối tượng được định nghĩa trước, thậm chí màn hình chính của một chương trình viết bằng Java (hàm main) cũng phải đặt bên trong của lớp. Hướng

đối tượng trong Java không có tính đa kế thừa (multi inheritance) như trong C++ mà thay vào đó Java đưa ra khái niệm interface để hỗ trợ tính đa kế thừa. Vấn đề này sẽ được bàn chi tiết trong các chương sau.

1.1.2.5. Đa nhiệm – đa luồng

Java hỗ trợ lập trình đa nhiệm, đa luồng cho phép nhiều tiến trình, tiểu trình có thể chạy song song cùng một thời điểm và tương tác với nhau.

1.1.2.6. Khả chuyển (Portable)

Chương trình ứng dụng viết bằng ngôn ngữ Java chỉ cần chạy được trên máy ảo Java là có thể chạy được trên bất kỳ máy tính, hệ điều hành nào có máy ảo Java “Viết một lần, chạy mọi nơi”.

1.1.2.7. Hỗ trợ mạnh cho việc phát triển ứng dụng

Công nghệ Java phát triển mạnh mẽ nhờ vào “đại gia Sun Microsystem” cung cấp nhiều công cụ, thư viện lập trình phong phú hỗ trợ cho việc phát triển nhiều loại hình ứng dụng khác nhau cụ thể như: J2SE (Java 2 Standard Edition) hỗ trợ phát triển những ứng dụng đơn, ứng dụng Client – Server; J2EE (Java 2 Enterprise Edition) hỗ trợ phát triển các ứng dụng thương mại; J2ME (Java 2 Micro Edition) hỗ trợ phát triển ứng dụng trên các thiết bị di động, không dây v.v...

1.1.3. Các ứng dụng Java

1.1.3.1. Các ứng dụng cho hệ điều hành Android

Java hỗ trợ tối đa cho hệ điều hành Android vì thế chỉ cần có một chiếc điện thoại Android thì bất cứ một ứng dụng nào trên đó cũng đều được hình thành và phát triển trên nền tảng Java. Android sử dụng máy ảo JVM và các package khác, nhưng phần code thì vẫn được viết bằng Java.

1.1.3.2. Các ứng dụng yêu cầu độ bảo mật cao

Ngôn ngữ lập trình có tính bảo mật cao nên thường được sử dụng vào trong các dịch vụ tài chính, ngân hàng. Trên thế giới hiện nay có rất nhiều ngân hàng đầu tư toàn cầu như Goldman Sachs, Citigroup, Barclays, Standard Chartered và các ngân hàng khác sử dụng Java để viết các hệ thống giao dịch điện tử, các hệ thống xác nhận và kiểm toán, các dự án xử lý dữ liệu và một số công việc quan trọng khác.

1.1.3.3. Các ứng dụng web Java

Không chỉ được sử dụng trên thiết bị di động mà Java còn được biết đến là

nền tảng của các ứng dụng website (web). Java cũng được sử dụng nhiều trong các ứng dụng web và thương mại điện tử. Nhiều tổ chức chính phủ, y tế, bảo hiểm, giáo dục, quốc phòng và một số bộ phận khác có ứng dụng web được xây dựng bằng Java.

1.1.3.4. Các công cụ phần mềm

Hiện nay có rất nhiều các công cụ phát triển và phần mềm hữu ích được viết bằng Java. Ví dụ: Eclipse, IntelliJ Idea và NetBeans IDE.

1.1.3.5. Không gian nhúng

Ban đầu Java được phát triển để phục vụ cho các thiết bị nhúng nhưng sau đó ngày càng được phát triển mở rộng ra các lĩnh vực khác. Với châm ngôn "viết một lần, chạy mọi nơi" cho thấy rằng Java đang mang lại nhiều lợi ích to lớn.

1.1.3.6. Công nghệ Big Data

Hadoop và các công nghệ big data khác cũng đang sử dụng Java theo cách này hay cách khác, ví dụ: HBase, Accumulo (mã nguồn mở) và Elasticsearch.

1.1.3.7. Trong lĩnh vực giao dịch chứng khoán

Java có một lợi thế nữa đó là sử dụng phù hợp với các hệ thống có hiệu suất cao, bởi vì mặc dù hiệu suất có thua chút ít so với ngôn ngữ native, nhưng người dùng lại có được sự an toàn, linh động và bảo trì với tốc độ nhanh hơn.

1.1.3.8. Các ứng dụng khoa học

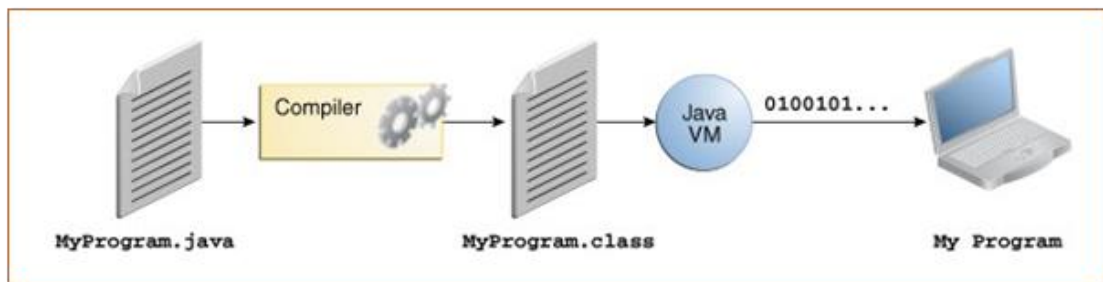
Hiện nay Java thường là một lựa chọn mặc định cho các ứng dụng khoa học, bao gồm cả xử lý ngôn ngữ tự nhiên.

1.1.4. Java platform và cơ chế thực thi

- Java Platform là một phần mềm dưới tên gọi Java Runtime Environment (JRE) chạy phía trên nền tảng phần cứng
- Java Platform có 2 thành phần:
 - + Java Virtual Machine (JVM - máy ảo Java): là phần mềm được cài đặt dưới dạng máy tính ảo.
 - + Java Application Programming Interface (API): là một tập hợp các thành phần phần mềm đã xây dựng sẵn để có thể xây dựng các ứng dụng tương tác đầy đủ trong Java
- Cơ chế thực thi chương trình Java:

+ Đầu tiên thông qua Java compiler, mã nguồn Java (*.java) sẽ được chuyển thành chương trình nhị phân chứa các mã bytecodes (*.class). Bytecodes là các chỉ thị máy của JVM.

+ Cuối cùng thông qua Java interpreter, nó kiểm tra và giải mã các bytecodes để đảm bảo an toàn khi thực thi. Sau đó, tiến hành thực thi các hoạt động bytecodes chỉ định trong JVM.



Hình 1.1. Tổng quát về tiến trình thực thi chương trình Java

1.2. CẤU TRÚC CHƯƠNG TRÌNH JAVA

Dạng cơ bản của một tập tin mã nguồn Java có cấu trúc như sau :

```
//khai báo tên gói, nếu có
package packagename;
//khai báo tên thư viện sẵn có, nếu cần dùng class classname
import java.awt.*;
{   /* đây là dòng ghi chú */
// khai báo biến
int var;
public void methodname()    // khai báo tên phương thức
{   /* phần thân của phương thức */
// lệnh thực hiện
statement (s);
}
}
```

Một tập mã nguồn Java có thể có ba phần chính:

- (1) Phần khai báo tên gói (khởi) bằng từ khoá package.
- (2) Phần khai báo thư viện tham khảo bằng từ khoá import.
- (3) Phần khai báo nội dung lớp bằng từ khoá class.

1.2.1. Khai báo package

Package được dùng để đóng gói các lớp trong chương trình lại với nhau thành một khối. Đây là một cách hữu hiệu để lưu trữ các lớp gần giống nhau hoặc có cùng một module thành một khối thống nhất.

- Cú pháp khai báo tên gói bằng từ khoá package:

package <tên gói>;

- Ưu điểm của package:

+ Cho phép nhóm các lớp vào với nhau thành các đơn vị nhỏ hơn. Việc thao tác trên các đơn vị khối sẽ gọn hơn thao tác trên một tập các lớp.

+ Tránh việc xung đột khi đặt tên lớp. Vì các lớp không cùng package thì có thể đặt tên trùng nhau. Khi số lượng lớp của chương trình quá lớn ta có thể tránh phải đặt tên khác nhau cho các lớp bằng cách đặt chúng vào các package khác nhau.

+ Cho phép bảo vệ các lớp. Khi chương trình lớn, việc chia nhỏ chương trình thành các package sẽ thuận lợi hơn cho việc quản lí và phát triển.

+ Tên gói còn được dùng để định danh lớp trong ứng dụng.

Lưu ý:

- Dòng lệnh khai báo tên khối phải được đặt đầu tiên trong tệp tin mã chương trình.
- Chỉ được khai báo tối đa một tên khối đối với mỗi tệp mã nguồn java.
- Các tệp tin của các lớp nằm cùng gói ứng dụng phải được lưu trong cùng một thư mục (tên thư mục là tên khối) theo cấu trúc khối của dự án.

1.2.2. Khai báo thư viện

Khai báo thư viện để chỉ ra những thư viện đã được định nghĩa sẵn mà chương trình sẽ tham khảo tới.

Cú pháp khai báo thư viện với từ khoá import như sau:

import <tên thư viện>;

- Java chuẩn cung cấp một số thư viện như sau:

+ **java.lang:** Cung cấp các hàm thao tác trên các kiểu dữ liệu cơ bản, xử lí lỗi và ngoại lệ, xử lí vào ra trên các thiết bị chuẩn như bàn phím và màn hình.

+ **java.applet:** Cung cấp các hàm cho xây dựng các applet.

+ **java.awt:** Cung cấp các hàm cho xây dựng các ứng dụng đồ hoạ với các thành phần giao diện multimedia.

+ **java.io:** Cung cấp các hàm xử lí vào/ra trên các thiết bị chuẩn và các thiết bị ngoại vi.

+ **java.util**: Cung cấp các hàm tiện ích trong xử lý liên quan đến các kiểu dữ liệu có cấu trúc như date, stack, vector.

Ví dụ, nếu trong chương trình cần đến các thao tác chuyển đổi kiểu dữ liệu tường minh (từ kiểu string sang kiểu int), thì ta sẽ phải tham khảo thư viện java.lang: **import java.lang.*;**

Lưu ý:

- Nếu muốn khai báo tham khảo nhiều thư viện, phải khai báo tham khảo mỗi thư viện với một từ khoá import.

- Nếu chỉ tham khảo một vài lớp trong một thư viện, nên chỉ rõ tham khảo lớp nào, thay vì phải khai báo tham khảo cả gói (bằng kí hiệu “*”) vì tham khảo cả gói sẽ tăng kích cỡ tệp tin class sau khi biên dịch.

- Nếu không tham khảo thư viện nào, không cần thiết phải khai báo các tham khảo với từ khoá import.

1.3. CHƯƠNG TRÌNH JAVA ĐẦU TIÊN

Để bắt đầu học lập trình Java, viết và chạy chương trình Java đầu tiên, trước tiên thì người học cần đảm bảo máy tính phải cài đặt: (1) Trình dịch Java (JDK); (2) Trình soạn thảo hoặc IDE chuyên dụng cho lập trình.

1.3.1. Trình dịch JDK

Java Development Kit (JDK - Bộ công cụ cho người phát triển ứng dụng bằng ngôn ngữ lập trình Java) là một tập hợp những công cụ phần mềm được phát triển bởi Sun Microsystems dành cho các nhà phát triển phần mềm, dùng để viết những applet Java hay những ứng dụng Java - bộ công cụ này được phát hành miễn phí gồm có trình biên dịch, trình thông dịch, trình giúp sửa lỗi.

JDK là một hệ tiêu chuẩn trong việc triển khai nền tảng Java để tạo các ứng dụng dựa trên nền tảng đó, bao gồm các trình thông dịch dịch và thư viện lớp.

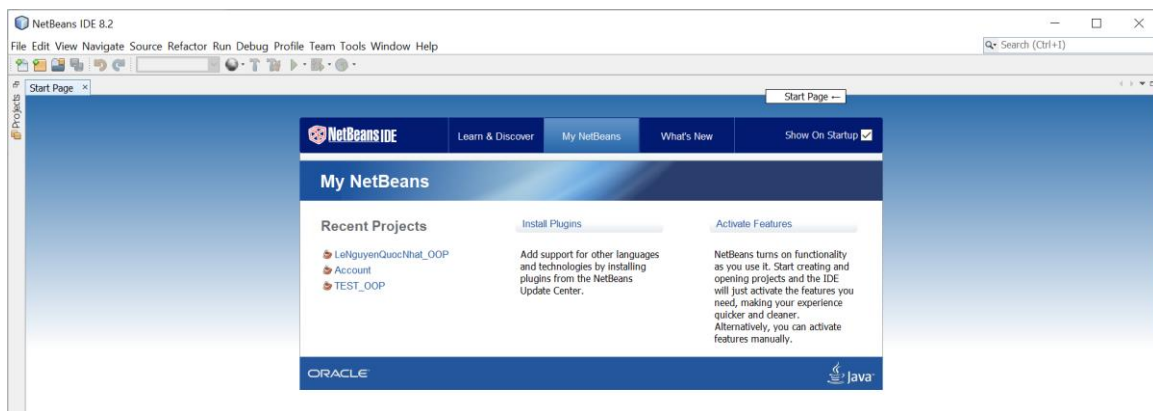
Java JDK có nhiều phiên bản JDK 7, JDK 8, JDK 8.1, JDK 9, JDK 10, JDK 11, JDK 12... Việc cài đặt các phiên bản cũng tương tự nhau. Tuy nhiên hệ điều hành mỗi người sử dụng khác nhau nên chọn phiên bản Java JDK phải phù hợp với hệ điều hành đang sử dụng.

Trong hầu hết các phiên bản thì Java SE 12.0.2 (LTS) hoạt động tốt và ổn định hơn vì nó được Oracle đánh dấu hỗ trợ dài hạn.

1.3.2. Trình soạn thảo NetBeans IDE

NetBeans IDE là một công cụ hỗ trợ lập trình viết mã code miễn phí hỗ trợ rất nhiều những ngôn ngữ lập trình như Java, Python, Ruby, JavaScript, Groovy, C / C + +, và cả PHP. Tuy nhiên công cụ này yêu cầu các máy tính phải có cấu hình RAM, CPU tương đối cao để vận hành.

Hiện nay NetBeans IDE đã cập nhật lên phiên bản 12.3. Cũng tương tự các công cụ hỗ trợ lập trình khác. NetBeans IDE gần giống với Notepad++. NetBeans IDE có gợi ý cú pháp khá tốt, và còn nhiều tính năng khá thú vị như tự động căn chỉnh mã nguồn,... Chính vì thế, nó được xem là môi trường phát triển ứng dụng cho các lập trình viên. Giao diện đầu tiên của NetBeans IDE như hình 1.2.

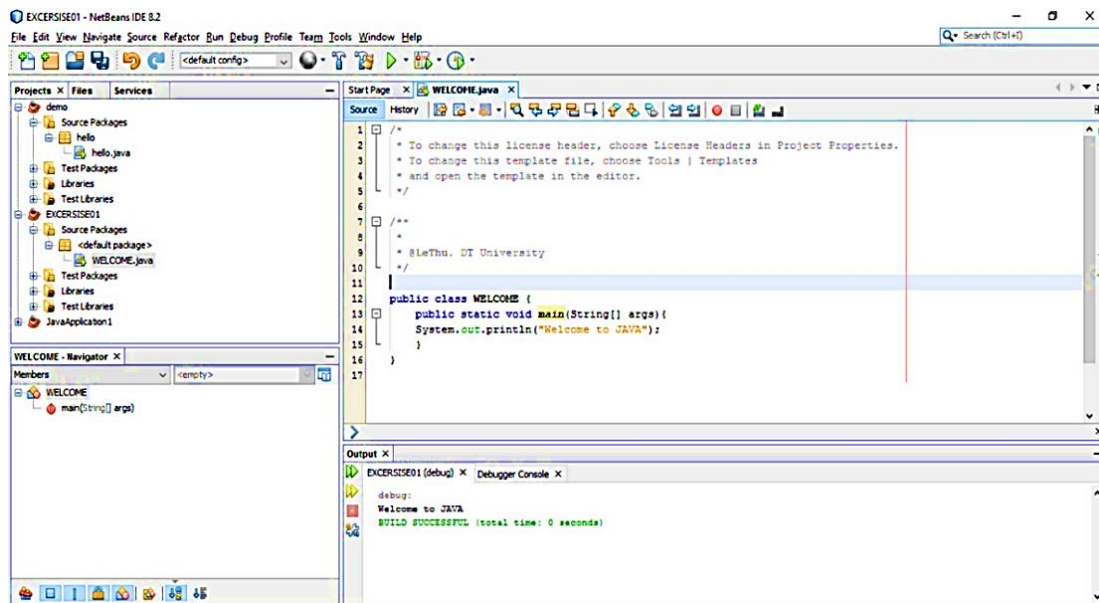


Hình 1.2. Giao diện trình soạn thảo NetBeans IDE 8.2

1.3.3. Chương trình Java đầu tiên

Chúng ta có thể sử dụng bất kỳ trình soạn thảo nào để soạn đoạn chương trình Java bên dưới và lưu với tên “WELCOME.java”.

```
class WELCOME {  
    public static void main(String[] args) {  
        // Display the string.  
        System.out.println("Welcome to Java!");  
    }  
}
```

Hình 1.3. Thực thi chương trình với NetBeans IDE 8.2

Phân tích đoạn chương trình

+ Mỗi chương trình Java có ít nhất 1 Class. Mỗi Class có 1 tên và trùng với tên file Java.

+ Để chạy 1 Class, Class phải chứa 1 phương thức có tên main. Chương trình thực thi từ phương thức main.

+ Mỗi phương thức gồm nhiều câu lệnh (statements) và mỗi câu lệnh kết thúc bằng dấu chấm phẩy (;)

+ Mỗi khối lệnh (block) được bao bởi cặp dấu { }.

+ Mã nguồn chương trình Java có phân biệt viết hoa thường.

+ Chú thích (Comment) có 2 dạng:

- Một dòng: dùng cặp dấu //
- Nhiều dòng: bao bởi cặp /* và */

+ *public static void main(String[] args)*: Đây là phương thức chính, từ đây chương trình bắt đầu việc thực thi. Tất cả các ứng dụng java đều một phương thức main.

- Từ khoá *public* là một chỉ định truy xuất cho biết thành viên của lớp có thể được truy xuất từ bất cứ đâu trong chương trình.

- Từ khoá *static* cho phép main được gọi tới mà không cần tạo ra một thể hiện (instance) của lớp và không phụ thuộc vào các thể hiện của lớp được tạo ra.

- Từ khoá *void* thông báo cho máy tính biết rằng phương thức sẽ không trả lại bất cứ giá trị nào khi thực thi chương trình.

- *String[] args* là tham số dùng trong phương thức *main*. Khi không có một thông tin nào được chuyển vào *main*, phương thức được thực hiện với các dữ liệu rỗng.

+ *System.out.println("Welcome to Java");* dòng lệnh này hiển thị chuỗi "Welcome to Java" trên màn hình. Lệnh *println()* cho phép hiển thị chuỗi được truyền vào lên màn hình.

1.4. CÂU HỎI ÔN TẬP

1. Giới thiệu tóm tắt về sự ra đời của ngôn ngữ lập trình Java?
2. Hãy cho biết Java là ngôn ngữ biên dịch hay thông dịch? Giải thích?
3. Nêu các đặc điểm chính của Java? Đặc điểm nào là thế mạnh của Java?

Ví dụ minh họa?

4. Trình bày một vài ứng dụng nổi bật của Java hiện nay?
5. Hãy cho biết cấu trúc của một chương trình Java? Cho ví dụ minh họa?
6. Điều kiện quan trọng nào được sử dụng để thực thi được một đoạn chương trình Java cơ bản.

7. Nêu một số công cụ hỗ trợ soạn thảo và thực thi chương trình Java mà bạn biết. So sánh ưu nhược điểm của công cụ bạn chọn so với công cụ NetBeans IDE.

8. Thực thi đoạn chương trình giới thiệu bản thân sinh viên gồm các thông tin: Họ và tên, lớp, điểm trung bình tích lũy. Mỗi thông tin hiển thị trên một dòng.

9. Sử dụng trình soạn thảo NetBeans IDE để hiển thị nội dung như bên dưới:

1. Các chương trình Java phải lưu với phần mở rộng **.java**.
 2. Tên tập tin phải khớp với tên của lớp. VD: tên của lớp **public class Hello**, thì chúng ta nên lưu nó trong một tập tin có tên **Hello.java**.
 3. Nên viết ghi chú để giải thích công việc của **class**, hoặc nội dung xử lý của phương thức.

10. Nhập đoạn chương trình bên dưới, thực thi đoạn chương trình đã nhập và cho biết kết quả thực hiện?

```
class Exercise10 {  
    public static void main(String[] args) {  
        // print boolean value  
        boolean flag = true;  
        System.out.println(flag);  
        // print integer value  
        int num = -11;  
        System.out.println(num);  
        // print float value  
        float number = 8.5f;  
        System.out.println(number);  
        // print letter value  
        char letter1 = '9';  
        System.out.println(letter1);  
        // print character value  
        char letter2 = 65;  
        System.out.println(letter2);  
        // print strings value  
        String str = "Java Programming";  
        System.out.println(str);  
    }  
}
```

CHƯƠNG 2. KIỂU DỮ LIỆU VÀ CẤU TRÚC LỆNH CƠ BẢN CỦA JAVA

Mục tiêu của chương

Sau khi học xong chương này, người học sẽ:

- Trình bày được các khái niệm về kiểu dữ liệu cơ bản, cách khai báo và sử dụng kiểu dữ liệu phù hợp theo từng bài toán cụ thể;
- Vận dụng thành thạo phương thức nhập, xuất dữ liệu;
- Lựa chọn hiệu quả việc sử dụng cấu trúc lệnh `if .. else`, `switch .. case`, vòng lặp `for`, `while`, `do .. while` vào bài toán cụ thể;
- Vận dụng thành thạo kiểu dữ liệu cấu trúc như mảng số nguyên, số thực, chuỗi ký tự.
- Mô tả ngoại lệ và hiển thị được vài phương thức xử lý ngoại lệ.

2.1. CÁC KIỂU DỮ LIỆU CƠ SỞ

Cũng giống như bất kỳ ngôn ngữ lập trình khác, ngôn ngữ lập trình Java cung cấp cho người sử dụng các kiểu dữ liệu cơ bản, có tám kiểu dữ liệu cơ sở bao gồm: `byte`, `short`, `int`, `long`, `float`, `double`, `boolean` và `char`.

2.1.1. Kiểu số nguyên

Java cung cấp bốn kiểu số nguyên khác nhau là: `byte`, `short`, `int`, `long`. Kích thước, giá trị mặc định, miền giá trị của các kiểu dữ liệu số nguyên được mô tả chi tiết trong bảng 2.1.

Bảng 2.1. Các kiểu dữ liệu số nguyên trong Java

Kiểu dữ liệu	Kích thước (số bit)	Giá trị mặc định	Miền giá trị
BYTE	8	0	-128 đến 127
SHORT	16	0	-32,768 đến 32,767
INT	32	0	- 2,147,483,648 đến 2,147,483,647
LONG	64	0L	-2^{63} đến $(2^{63}) - 1$

- Kiểu mặc định của các số nguyên là kiểu int; Các số nguyên kiểu byte và short rất ít khi được dùng
- Trong Java không có kiểu số nguyên không dấu như trong ngôn ngữ C/C++

Lưu ý:

- Nếu hai toán hạng kiểu long thì kết quả là kiểu long. Một trong hai toán hạng không phải kiểu long sẽ được chuyển thành kiểu long trước khi thực hiện phép toán.
- Nếu hai toán hạng đầu không phải kiểu long thì phép tính sẽ thực hiện với kiểu int.
- Các toán hạng kiểu byte hay short sẽ được chuyển sang kiểu int trước khi thực hiện phép toán.
- Trong Java không thể chuyển biến kiểu int và kiểu boolean như trong ngôn ngữ C/C++.

2.1.2. Kiểu dấu chấm động

Đối với kiểu dấu chấm động hay kiểu số thực, Java hỗ trợ hai kiểu dữ liệu là float và double

Kiểu float có kích thước 4 byte và giá trị mặc định là 0.0f

Kiểu double có kích thước 8 byte và giá trị mặc định là 0.0d

Lưu ý:

- Nếu mỗi toán hạng đều có dấu chấm động thì phép toán chuyển thành phép toán dấu chấm động
- Nếu có một toán hạng là double thì các toán hạng còn lại sẽ được chuyển thành kiểu double trước khi thực hiện phép toán
- Biến kiểu float và double có thể ép chuyển sang kiểu dữ liệu khác trừ kiểu boolean.

2.1.3. Kiểu ký tự (char)

Kiểu ký tự trong ngôn ngữ lập trình Java có kích thước là 2 bytes và chỉ dùng để biểu diễn các ký tự trong bộ mã Unicode. Như vậy, kiểu char trong Java có thể biểu diễn tất cả $2^{16} = 65536$ ký tự khác nhau.

Giá trị mặc định cho một biến kiểu char là null.

2.1.4. Kiểu luận lý (boolean)

Kiểu boolean chỉ nhận một trong hai giá trị: true hoặc false. Trong Java kiểu boolean không thể chuyển thành kiểu nguyên và ngược lại.

Giá trị mặc định của kiểu boolean là false.

2.2. BIẾN VÀ HẲNG SỐ

2.2.1. Biến – Variables

- Biến là một vùng nhớ được đặt tên, được dùng để lưu trữ thông tin trong chương trình và thông tin lưu trong nó có thể thay đổi.

Cú pháp khai báo:

<kiểu dữ liệu> <tên biến>;

Ví dụ 2.1:

```
int count;  
double radius;
```

- Nếu có nhiều biến cùng kiểu dữ liệu thì khai báo:

<kiểu dữ liệu> <tên biến 1>, <tên biến 2>, ...;

Ví dụ 2.2:

```
int i , j , k;
```

- Có thể khởi tạo các giá trị cho biến như trong các ví dụ sau:

Ví dụ 2.3: Vừa khai báo và khởi tạo

```
int count = 1;
```

Ví dụ 2.4: Khai báo trước, khởi tạo sau

```
int count; count = 1;
```

Ví dụ 2.5: Khai báo và khởi tạo cho nhiều biến cùng kiểu

```
int i = 1, j = 2;
```

Biến phải được khởi tạo trước khi sử dụng. Nếu khai báo trong phương thức phải khởi tạo giá trị biến trước khi sử dụng.

Nên khai báo và khởi tạo giá trị cho biến cùng lúc. Điều này làm chương trình dễ đọc và tránh được các lỗi..

2.2.2. Hằng số – Constants

Hằng số là đại diện cho dữ liệu cố định không bao giờ thay đổi.

Cú pháp khai báo:

final <kiểu dữ liệu> <TÊN HẲNG> = value;

Ví dụ 2.6: Khai báo và khởi tạo cho hằng số

final double PI = 3.14159;

- Phải khai báo và khởi tạo giá trị hằng số trong cùng câu lệnh.
- Tên hằng số nên viết hoa tất cả các từ.

2.3. TOÁN TỬ VÀ BIỂU THỨC

Toán tử trong Java là một ký hiệu được sử dụng để thực hiện một phép tính cho chức năng nào đó, Java cung cấp một số dạng toán tử như: Toán tử số học; toán tử quan hệ; toán tử logic; toán tử điều kiện; toán tử gán; toán tử bit.

2.3.1. Toán tử số học

Một số phép toán đối với số học được biểu diễn bởi các toán hạng và các toán tử số học, các toán hạng phải ở dạng số. Toán tử số học được mô tả chi tiết trong bảng 2.2.

Bảng 2.2. Bảng mô tả kiểu số học trong Java

Toán tử	Mô tả	Ví dụ	Kết quả
+	Phép cộng	34 + 2	36
-	Phép trừ	34.0 – 0.5	33.5
*	Phép nhân	200 * 20	4000
/	Phép chia	1.0 / 2.0	0.5
%	Phép lấy dư	7 % 2	1

2.3.2. Gán và gán mở rộng

Toán tử gán (=) dùng để gán một giá trị vào một biến và có thể gán nhiều giá trị cho nhiều biến cùng một lúc. Ngoài ra, các ngôn ngữ lập trình cấp cao đều cung cấp phép gán mở rộng để thuận lợi trong việc vừa thực hiện phép toán số học và vừa thực hiện phép gán. Phép gán và gán mở rộng được mô tả trong bảng 2.3.

Bảng 2.3. Bảng mô tả phép gán và gán mở rộng trong Java

Toán tử	Mô tả	Ví dụ	Kết quả
=	Phép gán đến giá trị một biến	int x = 10; double y = 1.0	x có giá trị 10; y có giá trị 1.0
+=	Cộng mở rộng	i += 3	i = i + 3
- =	Trừ mở rộng	i - = 3	i = i - 3

<code>*=</code>	Nhân mở rộng	<code>i * = 3</code>	<code>i = i * 3</code>
<code>/=</code>	Chia mở rộng	<code>i / = 3</code>	<code>i = i / 3</code>
<code>%=</code>	Lấy dư mở rộng	<code>i % = 3</code>	<code>i = i % 3</code>

2.3.3. So sánh

Các toán tử so sánh hay còn gọi là các toán tử quan hệ được sử dụng kiểm tra mối quan hệ giữa hai toán hạng. Kết quả của biểu thức quan hệ là giá trị Boolean (logic “true” hoặc “false”). Các toán tử so sánh được sử dụng trong các cấu trúc điều khiển và được mô tả trong bảng 2.4.

Bảng 2.4. Bảng mô tả phép so sánh toán tử trong Java

Toán tử	Mô tả	Ví dụ	Kết quả
<code><</code>	Nhỏ hơn	<code>3 < 9</code>	true
<code><=</code>	Nhỏ hơn or bằng	<code>5 <= 5</code>	true
<code>></code>	Lớn hơn	<code>3 > 9</code>	false
<code>>=</code>	Lớn hơn or bằng	<code>9 > = 3</code>	true
<code>==</code>	Bằng	<code>3 == 3</code>	true
<code>!=</code>	Khác	<code>9 != 9</code>	false

2.3.4. Logic

Các toán tử logic hay toán tử điều kiện làm việc với các toán hạng Boolean. Các toán tử logic được sử dụng trong các cấu trúc điều khiển và được mô tả chi tiết trong bảng 2.5.

Bảng 2.5. Bảng mô tả toán tử logic trong Java

Toán tử	Mô tả	Ví dụ	Kết quả
<code>&&</code>	So sánh AND	<code>(5 > 3) && (6 == 8)</code>	false
<code> </code>	So sánh OR	<code>(5 > 3) (6 == 8)</code>	true
<code>!</code>	Phủ định NOT	<code>!(8 >= 8)</code>	false

2.3.5. Tăng, giảm một đơn vị

Toán tử tăng giảm là toán tử một ngôi thường dùng trong các ngôn ngữ lập trình. Toán tử một ngôi được xem như là toán tử số học, phương thức thực hiện tương tự như toán tử số học. Toán tử tăng, giảm mô tả như bảng 2.6.

Bảng 2.6. Bảng mô tả toán tử tăng, giảm trong Java

Toán tử	Mô tả	Ví dụ (<code>i = 1</code>)
<code>++var</code>	Tăng giá trị biến var lên 1 và sử dụng giá trị var mới	<code>int j = ++i;</code> <code>// j = 2, i = 2</code>

var++	Tăng giá trị biến var lên 1 và sử dụng giá trị var ban đầu	int j = i++; // j = 1, i = 2
--var	Giảm giá trị biến var xuống 1 và sử dụng giá trị var mới	int j = --i; // j = 0, i = 0
var--	Giảm giá trị biến var xuống 1 và sử dụng giá trị var ban đầu	int j = i--; // j = 1, i = 0

Tuy nhiên, có sự khác nhau giữa “++var” và “var++” (“--var” và “var--”) là việc đưa toán tử phía sau có nghĩa là sau khi thực hiện các việc khác thì nó mới tăng (giảm) giá trị, còn đưa lên phía trước thì nó phải tăng (giảm) giá trị rồi mới thực hiện các việc khác.

Ví dụ 2.7: So sánh kết quả khác nhau đối với thứ tự toán tử trong phép toán tăng, giảm.

```
public class TangGiamDemo {
    public static void main(String[] args) {
        int a = 1;
        //In rồi mới tăng a=1
        System.out.println(a++);
        //Tăng rồi mới in a=3
        System.out.println(++a);
        //Cộng 1 và in ra rồi mới tăng a=4
        System.out.println(a+++1);
        //Tăng rồi mới cộng 1 và in ra a = 6
        System.out.println(++a+1);
    }
}
```

2.3.6. Toán tử trên bit

Java định nghĩa một số toán tử thao tác bit có thể được áp dụng cho các kiểu giá trị integer, long, int, short, char, và byte. Toán tử thao tác bit làm việc trên các bit. Giả sử nếu A = 60 (0011 1100) và B = 13 (0000 1101), thì trong định dạng nhị phân chúng sẽ như sau. Toán tử trên bit mô tả như bảng 2.7.

Bảng 2.7. Bảng mô tả toán tử thao tác trên bit

Toán tử	Mô tả	Ví dụ
&	Toán tử “Và” nhị phân sao chép một bit tới kết quả nếu nó tồn tại trong cả hai toán hạng	A = 0011 1100 B = 0000 1101

Toán tử	Mô tả	Ví dụ
		$A \& B = 0000\ 1100$ (12)
	Toán tử “Hoặc” nhị phân sao chép một bit tới kết quả nếu nó tồn tại trong một hoặc hai toán hạng	$A = 0011\ 1100$ $B = 0000\ 1101$ $A B = 0011\ 1101$ (61)
^	Toán tử “Hoặc” loại trừ nhị phân sao chép bit nếu nó được thiết lập trong một toán hạng nhưng không phải trong cả hai	$A = 0011\ 1100$ $B = 0000\ 1101$ $A^B = 0011\ 0001$ (49)
~	Toán tử đảo bit là toán tử một ngôi. Đảo bit 1 thành 0 và ngược lại	$A = 0011\ 1100$ $\sim A = 1100\ 0011$ (-61)
<<	Toán tử dịch trái. Giá trị toán hạng trái được dịch chuyển sang trái bởi số các bit được xác định bởi toán hạng bên phải.	$A = 0011\ 1100$ $A << 2 = 1111\ 0000$ (240)
>>	Toán tử dịch phải. Giá trị toán hạng trái được dịch chuyển sang phải bởi số các bit được xác định bởi toán hạng bên phải	$A = 0011\ 1100$ $A >> 2 = 0000\ 1111$ (15)
>>>	Toán tử dịch phải và điền 0 vào chỗ trống	$A = 0011\ 1100$ $A >>> 2 = 0000\ 1111$ (15)

2.3.7. Một số phương thức lớp Math

Thư viện toán học Math trong Java là thư viện cung cấp các phương thức có sẵn để giải quyết các vấn đề về số học. Các phương thức này phần lớn trả về giá trị kiểu double. Bảng dưới đây sẽ liệt kê tên và công dụng của các phương thức có trong thư viện này.

Bảng 2.8. Bảng mô tả các phương thức trong lớp Math

Phương thức	Diễn giải	Ví dụ
PI	Trả về giá trị PI.	<code>double g45 = Math.PI/4;</code>
abs(a)	Trả về giá trị tuyệt đối của số a.	<code>int b = Math.abs(-20);</code> <code>// 20</code>
max(a,b)	Trả về giá trị lớn nhất giữa 2 số a và b.	<code>int m = Math.max(10, 20);</code> <code>// 20</code>
min(a,b)	Trả về giá trị nhỏ nhất giữa 2 số a và b.	<code>int m = Math.min(10, 20);</code> <code>// 10</code>

Phương thức	Diễn giải	Ví dụ
sqrt(a)	Trả về căn bậc 2 của số a.	<code>double a = Math.sqrt(9); //3</code>
pow(x,y)	Tính lũy thừa x^y .	<code>double p = Math.pow(2, 3); // 8.0</code>
sin(radian)	Tính sin, với radian = $\text{Math.PI} * \text{góc} / 180$.	<code>double s = Math.sin(Math.PI/2); //1</code>
ceil()	Trả về giá trị double là số làm tròn tăng bằng giá trị số nguyên gần nhất.	<code>double c = Math.ceil(7.342); // 8.0</code>
floor()	Trả về double là số làm tròn giảm	<code>double f = Math.floor(7.343); // 7.0</code>
random()	Trả về một số ngẫu nhiên giữa 0.0 và 1.0 của kiểu double.	<code>double r = Math.random();</code>
toDegrees()	Đổi góc radian thành độ	<code>double goc = Math.toDegrees(Math.PI/2); //90</code>
toRadians()	Đổi góc đơn vị độ ra radian	<code>double goc = Math.toRadians(45); //0.7853981633974483</code>

2.4. CÁC PHƯƠNG THỨC VÀO RA CƠ BẢN

2.4.1. Xuất dữ liệu

Một số lệnh dùng để xuất dữ liệu ra màn hình: **System.out.print** | **[println | printf]**. Để thực hiện in nhiều thông tin dữ liệu khi hiển thị phải dùng toán tử “+”.

Cú pháp:

```
/*xuất dữ liệu anyvalue ra màn hình, không đưa trở chuột xuống dòng mới*/
System.out.print(anyvalue);
/*xuất dữ liệu anyvalue ra màn hình và đưa con trỏ chuột về dòng mới*/
```

```
System.out.println(anyvalue) ;
/* Xuất ra cách định dạng chuỗi (tương tự printf trong
lập trình C/C++) */
System.out.printf(anyvalue) ;
```

Một số mã ký tự đặc biệt:

Mã ký tự	Diễn giải
\b	Khoảng trắng
\t	Tab
\n	Xuống dòng mới
\\	Dấu \
\'	Dấu nháy đơn ‘
\”	Dấu nháy đôi “

Ví dụ 2.8: Xuất ra màn hình chuỗi

```
public class Main{
    public static void main(String[] args) {
        System.out.println("Hello: " + 2);
        System.out.print("Welcome to " + "\"Java\"!");
    }
}
```

→ Hello: 2

→ Welcome to “Java”!

2.4.2. Nhập dữ liệu từ bàn phím

Nhập dữ liệu từ bàn phím thông qua lớp **Scanner**, để sử dụng đối tượng **Scanner**, chúng ta cần nhập gói dữ liệu **java.util.Scanner** với câu lệnh sau:

```
import java.util.Scanner;
```

Sau đó, chúng ta cần tạo một đối tượng của lớp Scanner và có thể sử dụng đối tượng này để lấy đầu vào từ người dùng.

```
Scanner input = new Scanner(System.in);
```

```
int ten_bien = input.nextInt();
```

Cú pháp:

```
//lớp Scanner nằm trong thư viện java.util
import java.util.Scanner;
// tạo một Scanner
```

```

Scanner input = new Scanner(System.in);
//đọc số nguyên kiểu int
int intvar = input.nextInt();
//đọc số nguyên kiểu long
long longvar = input.nextLong();
//đọc số thực kiểu double
double doublevar = input.nextDouble();
//đọc số thực kiểu float
float floatvar = input.nextFloat();
//đọc chuỗi kết thúc bằng kí tự khoảng trắng
String str1 = input.next();
//đọc một dòng các ký tự kết thúc bằng phím enter
String str = input.nextLine();

```

Ví dụ 2.9: Nhập dữ liệu và in dữ liệu nhập

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner nhap_gia_tri = new Scanner(System.in);
        System.out.print("Nhap so mot so thuc: ");
        float a = nhap_gia_tri.nextFloat();
        System.out.println("So vua nhap la " + a);
        System.out.print("Nhap mot tu: ");
        String b = nhap_gia_tri.next();
        System.out.println(b);
        nhap_gia_tri.close();
    }
}

```

Ví dụ 2.10: Nhập xuất dữ liệu và in theo định dạng (printf).

```

import java.util.*; //thu vien dung lop Scanner
class Main{
    public static void main(String agrv[]){
        String hoten;
        int tuoi;
        double diemtb;
        //tao doi tuong inp thuoc lop Scanner
    }
}

```

```

Scanner inp = new Scanner(System.in);
//Lệnh in ra màn hình
System.out.print("Nhập họ tên : ");
hoten = inp.nextLine(); //nhập chuỗi
System.out.print("Nhập tuổi: ");
tuoi = inp.nextInt(); //nhập số nguyên
System.out.println("Nhập điểm trung bình: ");
diemtb = inp.nextDouble();
System.out.printf("Tên tôi: %s , Tuổi %d và
Điểm trung bình là: %.2f", hoten, tuoi, diemtb);
inp.close();
}
}

```

2.5. LỆNH LỰA CHỌN

2.5.1. Câu lệnh if

❖ Dạng 1: if một chiều

Mệnh đề if được sử dụng để kiểm tra giá trị dạng boolean của điều kiện. Khối lệnh sau if được thực thi nếu giá trị của điều kiện là True.

Cú pháp:

```

if (<biểu thức logic>) {
    Các câu lệnh;
}

```

Diễn giải:

- Nếu <biểu thức logic> đúng (true) thì khối lệnh sẽ được thực thi.
- Có thể bỏ cặp dấu {} khi chỉ có 1 câu lệnh bên trong if

Ví dụ 2.11: Cho đoạn chương trình xác định độ tuổi trưởng thành

```

public class Test {
    public static void main(String[] args) {
        int age = 20;
        if (age > 18) {
            System.out.print("Tuổi lớn hơn 18");
        }
    }
}

```

→ Tuổi lớn hơn 18

❖ Dạng 2: if hai chiều

Mệnh đề if-else cũng kiểm tra giá trị dạng boolean của điều kiện. Nếu giá trị điều kiện là True thì chỉ có khối lệnh sau if sẽ được thực hiện, nếu là False thì chỉ có khối lệnh sau else được thực hiện.

Cú pháp:

```
if (<biểu thức logic>) {  
    Khối lệnh 1 nếu <biểu thức logic> là true;  
} else {  
    Khối lệnh 2 nếu <biểu thức logic> là false;  
}
```

Ví dụ 2.12: cho đoạn chương trình xác định số chẵn lẻ

```
public class Test {  
    public static void main(String[] args) {  
        int number = 13;  
        if (number % 2 == 0) {  
            System.out.println("Số " + number +  
                               " là số chẵn.");  
        }  
        else{  
            System.out.println("Số " + number +  
                               " là số lẻ.");  
        }  
    }  
}
```

→ Số 13 là số lẻ.

❖ Dạng 3: if lồng (nested if)

Mệnh đề if-else-if cũng kiểm tra giá trị dạng boolean của điều kiện. Nếu giá trị điều kiện if là True thì chỉ có khối lệnh sau if sẽ được thực hiện. Nếu giá trị điều kiện if else nào là True thì chỉ có khối lệnh sau else if đó sẽ được thực hiện... Nếu tất cả điều kiện của if và else if là False thì chỉ có khối lệnh sau else sẽ được thực hiện.

Cú pháp:

```
if (<biểu thức logic 1>) {  
    Khối lệnh 1; }  
if (<biểu thức logic 2>) {  
    Khối lệnh 2; }  
...  
else {  
    Khối lệnh cuối;  
}
```

```
else if (<biểu thức logic 2>) {  
    Khối lệnh 2; }  
... else if (<biểu thức logic n>){  
    Khối lệnh n; }
```

Ví dụ 2.13: cho đoạn chương trình xếp loại điểm

```
public class Test {  
    public static void main(String[] args) {  
        int marks = 65;  
        if (marks < 50) {  
            System.out.println("Điểm I");  
        } else if (marks >= 50 && marks < 60) {  
            System.out.println("Xếp loại D");  
        } else if (marks >= 60 && marks < 70) {  
            System.out.println("Xếp loại C");  
        } else if (marks >= 70 && marks < 80) {  
            System.out.println("Xếp loại B");  
        } else if (marks >= 80 && marks < 90) {  
            System.out.println("Xếp loại A");  
        } else if (marks >= 90 && marks < 100) {  
            System.out.println("Xếp loại A+");  
        } else {  
            System.out.println("Giá trị không hợp lệ!");  
        }  
    }  
}
```

→ Xếp loại C

2.5.2. Câu lệnh Switch

Câu lệnh switch-case trong Java được sử dụng để thực thi 1 hoặc nhiều khối lệnh từ nhiều điều kiện.

Cú pháp:

```
switch(<biểu thức kiểm tra>) {  
    case <giá trị 1>: Khối lệnh 1;  
        break;  
    case <giá trị 2>: Khối lệnh 2;
```



```

        break;
    . . .
    case <giá trị n>: Khởi lệnh n;
        break;
    default: Khởi lệnh n+1;
}

```

Ví dụ 2.14: Cho đoạn chương trình xác định số 10, 20, 30

```

public class SwitchExample {
    public static void main(String[] args) {
        int number = 20;
        switch (number) {
            case 10:
                System.out.println("10");
                break;
            case 20:
                System.out.println("20");
                break;
            case 30:
                System.out.println("30");
                break;
            default:
                System.out.println("Not in 10, 20 or 30");
        }
    }
}

```

→ 20

Lưu ý:

- <biểu thức kiểm tra> phải là một trường kiểu char, byte, short, hoặc int và luôn được bao bởi cặp dấu ngoặc tròn.
- <giá trị 1>, ..., <giá trị n> là các biểu thức hằng (không chứa biến) và cùng kiểu dữ liệu với <biểu thức kiểm tra>.

2.5.3. Biểu thức điều kiện

Dùng để lựa chọn giá trị biểu thức dựa vào biểu thức logic.

Cú pháp:

<biểu thức logic> ? <biểu thức 1> : <biểu thức 2>

Ví dụ 2.15: Gán giá trị lớn hơn cho biến max

```
class Solonnhat{
    public static void main(String[] args) {
        int a = 1;
        int b = 2;
        int max = a>b ? a : b;
        System.out.println(max);
    }
}
```

2.6. LỆNH LẶP

2.6.1. Vòng lặp while

Vòng lặp while trong Java được sử dụng để lặp một phần của chương trình một vài lần. Nếu số lần lặp không được xác định trước thì vòng lặp while được khuyến khích sử dụng trong trường hợp này.

Cú pháp:

```
while (<điều kiện lặp>) {
    // Thân vòng lặp;
}
```

Ví dụ 2.16: Tính tổng 1 .. 10 bằng vòng lặp while

```
public class VongLapWhile {
    public static void main(String[] args) {
        // Tổng các số từ 1 đến 10
        int i = 1;
        int tong = 0;
        while (i <= 10) {
            tong = tong + i;
            i++;
        }
        System.out.println(tong);
    }
}
```

2.6.2. Vòng lặp do – while

Vòng lặp do-while trong Java được sử dụng để lặp một phần của chương trình một vài lần. Tương tự như vòng lặp while, ngoại trừ do-while thực hiện lệnh ít nhất một lần ngay cả khi điều kiện là False.

Cú pháp:

```
do (<điều kiện lặp>) {  
    // Thân vòng lặp;  
} while (<điều kiện lặp>)
```

Ví dụ 2.17: Tính tổng 1 .. 10 bằng vòng lặp do .. while

```
public class VongLapDoWhile {  
    public static void main(String[] args) {  
        // Tổng các số từ 1 đến 10  
        int i = 1;  
        int tong = 0;  
        do {  
            tong = tong + i;  
            i++;  
        } while (i <= 10);  
        System.out.println(tong);  
    }  
}
```

2.6.3. Vòng lặp for

Vòng lặp for trong Java được sử dụng để lặp một phần của chương trình nhiều lần. Nếu số lần lặp là cố định thì vòng lặp for được khuyến khích sử dụng, còn nếu số lần lặp không cố định thì nên sử dụng vòng lặp while hoặc do while.

Cú pháp:

```
for(<biểu thức khởi tạo>;<điều kiện lặp>;<biểu thức tăng>){  
    // Thân vòng lặp;  
    Khối lệnh;}
```

Lưu ý:

- <biểu thức khởi tạo> phải khai báo bên trong cấu trúc lặp hoặc trước vòng lặp đều được.

- <biểu thức khởi tạo> có thể là một danh sách rỗng hoặc các câu lệnh khai báo biến cách nhau bởi dấu phẩy hoặc biểu thức gán.

Ví dụ 2.18: Tính tổng từ 1 .. 10 với vòng lặp for

```
public class VongLapFor {
    public static void main(String[] args) {
        // Tổng các số từ 1 đến 10
        int tong = 0;
        for(int i = 1; i <= 10; i++) {
            tong = tong + i;
        }
        System.out.println(tong);
    }
}
```

Ví dụ 2.19: biểu thức 1, biểu thức 3 có thể thực hiện nhiều công việc

```
public class For2var {
    public static void main(String[] args) {
        int i, j;
        /* Phần khởi tạo và phần tăng/giảm chứa đựng nhiều
        hơn một câu lệnh */
        for (i = 0, j = 10; i <= j; i++, j--) {
            System.out.printf("\n%d + %d = %d", i, j, i+j);
        }
    }
}
```

Kết quả:

```
0 + 10 = 10
1 + 9 = 10
2 + 8 = 10
3 + 7 = 10
4 + 6 = 10
5 + 5 = 10
```

Ví dụ 2.20: biểu thức 1, biểu thức 3 có thể thực hiện nhiều công việc

```
for (int i=1, i<100; System.out.println(i), i++;) {  
    //do something;  
}
```

Ví dụ 2.21: biểu thức 1, biểu thức 2, biểu thức 3 rỗng (vòng lặp vô tận)

```
class VonglapForRong{  
    public static void main(String[] args) {  
        int i;  
        final int COUNT = 5;  
        System.out.print("Vong lap For:  ");  
        i = 1;  
        for (;;) {  
            System.out.print(i + " ");  
            if (i == COUNT) {  
                break;  
            }  
            i++;  
        }  
    }  
}
```

❖ Từ khóa break, continue

- *break*: được sử dụng trong vòng lặp để thoát khỏi vòng lặp ngay lập tức.
- *continue*: được sử dụng để thoát khỏi lần lặp hiện hành và tiếp tục bắt đầu với lần lặp tiếp theo.

2.7. MẢNG

2.7.1. Định nghĩa mảng

Mảng (array) là một tập hợp các phần tử có cùng kiểu được lưu trữ kế tiếp nhau trong bộ nhớ. Mảng trong Java là một đối tượng chứa các phần tử có cùng kiểu dữ liệu và mỗi phần tử trong mảng được truy xuất thông qua chỉ số của nó trong mảng. Với mảng chúng ta chỉ có thể lưu trữ một tập các phần tử có số lượng phần tử cố định.

2.7.2. Khai báo mảng

Mảng được dùng để lưu trữ một tập các dữ liệu hay mảng là một tập hợp các biến có cùng kiểu dữ liệu.

Cú pháp:

```
<kiểu dữ liệu>[] <tên mảng>;  
Hoặc: <kiểu dữ liệu> <tên mảng>[];
```

Ví dụ 2.22: Khai báo mảng

```
String DanhSachSV[]; //hoặc  
String[] DanhSachSV;  
//Khai báo nhiều mảng có kiểu dữ liệu giống nhau  
int[] arr1, arr2;
```

Không giống như trong C, C++ kích thước mảng trong Java được xác định khi khai báo. Để cấp phát bộ nhớ hay nói cách khác là kích thước cho mảng trong Java chúng ta cần dùng từ khóa **new**. Chẳng hạn để cấp phát vùng nhớ cho mảng gồm 10 số thực trong Java chúng ta thực hiện khởi tạo mảng:

Cú pháp:

```
<tên mảng> = new <kiểu dữ liệu>[<số ptử mảng>];  
Hoặc:  
<kiểu dữ liệu> <tên mảng>[] =  
    new <kiểu dữ liệu>[<kích thước>];  
Hoặc:  
<kiểu dữ liệu>[] <tên mảng> =  
    new <kiểu dữ liệu>[<kích thước>];  
//Khởi tạo giá trị ban đầu cho các phần tử mảng  
<kiểu dữ liệu>[] <tên mảng>=  
    {giá trị 1, giá trị 2, ..., giá trị n};  
Hoặc: <tên mảng>[i] = <giá trị i>;
```

Ví dụ 2.23: Khai báo và khởi tạo

```
double mangSoThuc = new double[10];  
int mangSoNguyen[] = {1, 2, 3};  
char mangKyTu[] = {'J', 'a', 'v', 'a'};
```

```
String mangChuoi[] ={"Ngo Van A","Tran B","Le Thi C"};
double [] mylist = new double [4];
mylist[0] = 3.6;
mylist[1] = 2.9;
```

2.7.3. Độ dài mảng

Để biết có bao nhiêu phần tử một mảng, sử dụng thuộc tính length.

Ví dụ 2.24: In số lượng phần tử mảng bằng thuộc tính length

```
class Mang1chieu{
    public static void main(String args[]){
        int array[] = new int[7];
        for (int count=0;count<7;count++){
            array[count]=count+1;
        }
        //In ra các phần tử và chỉ số phần tử trong mảng
        for (int count=0;count<7;count++){
            System.out.println("array["+count+"] = 
                                "+array[count]);
        }
        System.out.println("Length of Array = 
                                "+array.length);

        // array[6] =7;
    }
}
```

2.7.4. Duyệt các phần tử của mảng

Có 2 cách để duyệt các phần tử của mảng: Sử dụng vòng lặp for hoặc Sử dụng foreach.

Cú pháp:

```
<for (biến đếm = 0; biến đếm <= mảng.length - 1;
        biến đếm++) {
    //Code xử lý phần tử mảng
}
Hoặc: foreach (chỉ đọc - read only)
    for (<kiểu dữ liệu> <biến> :<tên mảng>){
        //Code xử lý phần tử mảng
    }
```

Ví dụ 2.25: Duyệt phần tử mảng

```
class DuyệtForForEach{
    public static void main(String[] args) {
        String[] LOPSV = {"CNTT", "Toan", "Van",
                           "HoaSinh", "Am nhac" };

        //sử dụng for
        for (int i = 0; i < LOPSV.length; i++) {
            System.out.println(LOPSV[i]);
        }

        //sử dụng foreach
        for (String LOPHOC : LOPSV) {
            System.out.println(LOPHOC);
        }
    }
}
```

2.7.5. Sao chép mảng

Có 2 cách phổ biến để sao chép mảng:

- Sử dụng vòng lặp để sao chép từng phần tử riêng biệt;
- Sử dụng phương thức *arraycopy* của lớp *System*

Cú pháp:

```
arraycopy(mang_nguồn,src_pos,mang_đích,tar_pos,length) ;
```

Trong đó:

- + src_pos: vị trí bắt đầu trong mảng nguồn
- + tar_post: vị trí bắt đầu trong mảng đích
- + length: số lượng phần tử copy từ mảng nguồn sang mảng đích

Lưu ý: Mảng đích phải được tạo trước đó

Ví dụ 2.26: Thực hiện sao chép mảng ban đầu sang mảng đích

```
class SaochepMang{
    public static void main(String[] args) {
        char[] copyFrom = {'L', 'O', 'P', 'C', 'N',
                           'T', 'T', '&', 'S', 'P', 'T', 'I', 'N'};
        char[] copyTo = new char[4];
        System.arraycopy(copyFrom, 3, copyTo, 0, 4);
    }
}
```



```

        System.out.println(new String(copyTo));
    }
}

```

2.7.6. Lớp Arrays

- java.util.Arrays cung cấp nhiều phương thức tĩnh để sắp xếp, tìm kiếm, so sánh và thay thế các phần tử mảng. Các phương thức này được nạp chồng cho các kiểu dữ liệu chính.

- Một số phương thức cơ bản:

- + *sort*: sắp xếp toàn bộ hoặc một phần mảng
- + *binarySearch*: tìm kiếm một phần tử trong mảng. Mảng cần được sắp xếp tăng dần trước. Nếu không tìm thấy trả về: $-(\text{insertion index} + 1)$
- + *equals*: kiểm tra 2 mảng có bằng nhau hay không. Hai mảng bằng nhau nếu giống nhau về nội dung
- + *fill*: thay thế giá trị phần tử toàn bộ hoặc một phần mảng.

2.7.7. Sắp xếp mảng

Có nhiều cách sắp xếp, có thể sắp xếp theo thứ tự tăng dần hoặc ngược lại. Sắp xếp tăng theo phương thức Arrays.sort, hoặc sử dụng for để sắp xếp.

Ví dụ 2.27: Sử dụng phương thức Arrays.sort để sắp xếp.

```

import java.io.IOException;
import java.util.*;
public class Sapxepmang{
    public static void main(String[] args) {
        String[] LOPSV = {"CNTT", "Toan", "Van",
                           "HoaSinh", "Am nhac" };
        // sap xep mang LOPSV theo thu tu tang dan
        Arrays.sort(LOPSV);
        System.out.println("LOPSV sau khi sap xep:");
        for (String LOP : LOPSV) {
            System.out.println(LOP);
        }
    }
}

```

Ví dụ 2.28: Sử dụng vòng lặp for để sắp xếp.

```
import java.io.IOException;
import java.util.*;
public class SapxepmangTang{
    public static void main(String[] args) {
        Scanner inp = new Scanner(System.in);
        System.out.print("Nhap so luong phan tu: ");
        int n = inp.nextInt();
        // khởi tạo array
        int [] arr = new int [n];
        System.out.print("Nhap phan tu: \n");
        for (int i = 0; i < n; i++) {
            System.out.printf("a[%d] = ", i);
            arr[i] = inp.nextInt();
        }
        //In mang sau khi nhap
        System.out.print("Mang sau khi nhap\n");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        //Sap xep mang
        int temp = arr[0];
        for (int i = 0 ; i < arr.length - 1; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] > arr[j]) {
                    temp = arr[j];
                    arr[j] = arr[i];
                    arr[i] = temp;
                }
            }
        }
        //In danh sach phan tu
        System.out.print("\nMang sau khi sap xep\n");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

2.7.8. Mảng 2 chiều

Mảng đa chiều chính là mảng chứa mảng. Để khai báo một mảng đa chiều, chúng ta phải đặc biệt chỉ định chỉ số bên trong ngoặc vuông.

Thông thường chúng ta làm việc với mảng 1 chiều, tuy nhiên trong trường hợp chúng ta muốn làm việc với một dữ liệu dạng bảng, với nhiều cột và nhiều dòng thì mảng 2 chiều được sử dụng. Cách khai báo, khởi tạo và gán giá trị tương tự mảng 1 chiều.

Cú pháp:

```
<Kiểu dữ liệu>[][]<Tên mảng>;  
kiểu_dữ_liệu[][]<tên_mảng> = new kiểu_dữ_liệu  
                                [số_hàng][số_cột];
```

Ví dụ 2.29: Nhập xuất mảng 2 chiều.

```
import java.io.IOException;  
import java.util.*;  
import java.util.Arrays;  
public class Mang2chieu{  
    public static void main(String[] args) {  
        // khai báo số dòng và số cột cho mảng  
        int soDong, soCot;  
        Scanner inp = new Scanner(System.in);  
        System.out.print("Nhap so dong: ");  
        soDong = inp.nextInt();  
        System.out.print("Nhap so cot: ");  
        soCot = inp.nextInt();  
        // khai báo và cấp phát bộ nhớ cho mảng  
        int[][] A = new int[soDong][soCot];  
        // Để nhập giá trị các phần tử cho mảng  
        for (int i = 0; i < soDong; i++) {  
            for (int j = 0; j < soCot; j++) {  
                System.out.print("Nhap phan tu thu  
                                [" + i + ", " + j + "]: ");  
                A[i][j] = inp.nextInt();  
            }  
        }  
        // hiển thị các phần tử trong mảng vừa nhập  
        System.out.println("Mang vua nhap: ");
```

```

for (int i = 0; i < soDong; i++) {
    for (int j = 0; j < soCot; j++) {
        System.out.print(A[i][j] + "\t");
    }
    // sau khi in xong 1 dòng thì xuống hàng
    System.out.print("\n");
}
}

```

2.8. CHUỖI

2.8.1. Khai báo chuỗi

Mảng được dùng để lưu trữ một tập các dữ liệu hay mảng là một tập hợp các biến có cùng kiểu dữ liệu.

- Chuỗi là một dãy tuần tự các ký tự. Trong Java, chuỗi được lưu dưới dạng đối tượng thuộc lớp String.

Cú pháp:

String <tên biến> = new String(<chuỗi khởi tạo>);

Hoặc: String <tên biến> = <chuỗi khởi tạo>;

Lưu ý:

- Chuỗi phải được đặt trong cặp dấu nháy đôi “ ”.
- Có thể khởi tạo chuỗi từ mảng các ký tự.
- Để nối hai chuỗi với nhau, sử dụng toán tử “+”.
- Nếu một trong hai chuỗi là null thì nó sẽ chuyển giá trị null sang chuỗi “null”.

Ví dụ 2.30: Khởi tạo và in chuỗi.

```

public class Main{
    public static void main(String[] args){
        //Khởi tạo chuỗi
        String chuoi1 = new String("Welcome to ");
        //Khởi tạo chuỗi 2 và nối chuỗi 1
    }
}

```

```

        String chuoi2 = chuoi1 + "Java";
        System.out.println(chuoi2); // in chuỗi 2
        System.out.println(chuoi1 + " " + chuoi2);
    }
}

```

2.8.2. Một số phương thức xử lý chuỗi

Một số phương thức thuộc lớp String trong ngôn ngữ Java cung cấp được mô tả trong bảng 2.9

Bảng 2.9. Bảng mô tả các phương thức thuộc lớp String

Tên phương thức	Mô tả
char charAt(int i)	Trả về kí tự thứ i
int compareTo(Object o)	So sánh chuỗi này với đối tượng khác
String concat(String str)	Nối chuỗi str vào cuối chuỗi này
int indexOf(String str)	Trả về chỉ mục trong chuỗi này của lần xuất hiện đầu tiên của chuỗi con được chỉ định.
int length()	Trả về độ dài của chuỗi

2.8.2.1. So sánh chuỗi

So sánh được sử dụng một trong các hình thức như: xác thực (bởi phương thức equals()), sắp xếp (theo phương thức compareTo()), so sánh tham chiếu (với toán tử ==). Có ba cách để so sánh chuỗi trong Java:

- Sử dụng phương thức equals();
- Sử dụng toán tử ==;
- Sử dụng phương thức compareTo().

Bảng 2.10. Bảng mô tả phương thức so sánh chuỗi

Tên phương thức	Mô tả
equals(s1: String): boolean	Trả về true nếu chuỗi so sánh bằng với chuỗi s1
equalsIgnoreCase(s1:String): boolean	Trả về true nếu chuỗi so sánh bằng với chuỗi s1 không phân biệt hoa thường
compareTo(s1: String):	0: chuỗi hiện hành lớn hơn s1
compareToIgnoreCase(s1: String):	= 0: chuỗi hiện hành bằng s1

Tên phương thức	Mô tả
startsWith(prefix: String): boolean	Trả về true nếu chuỗi so sánh bắt đầu giống như chuỗi chỉ định trong phương thức
endsWith(suffix: String): boolean	Trả về true nếu chuỗi so sánh kết thúc giống như chuỗi chỉ định trong phương thức

Ví dụ 2.31: Khởi tạo và so sánh chuỗi.

```
public class Teststringcomparison1 {
    public static void main(String args[]) {
        String s1 = "Hello";
        String s2 = "Hello";
        String s3 = new String("Hello");
        String s4 = "Hallo";
        String s5 = "HELLO";
        System.out.println(s1.equals(s2)); //true
        System.out.println(s1.equals(s3)); //true
        System.out.println(s1.equals(s4)); //false
        System.out.println(s1.equals(s5)); //false
        System.out.println(s1.equalsIgnoreCase(s5));
                                //true
        System.out.println(s1 == s2); //true
        System.out.println(s1 == s4); //false
        System.out.println(s1.compareTo(s2)); //0
        System.out.println(s1.compareTo(s4)); //4
        System.out.println(s4.compareTo(s1)); //-4
    }
}
```

2.8.2.2. Chiều dài, ký tự và nối chuỗi

- Phương thức trả về chiều dài của chuỗi ký tự là đếm số lượng ký tự có trong chuỗi (length).
- Nối chuỗi là sự kết hợp của ít nhất 2 chuỗi. Có 2 phương thức nối chuỗi: phương thức concat() và sử dụng toán tử "+". Cả hai phương pháp này cho ra một kết quả tương tự nhau.
- Phương thức để trả về ký tự tại vị trí xác định có trong chuỗi với phương thức charAt(vị trí).

Bảng 2.11. Bảng mô tả phương thức lấy chiều dài và nối chuỗi

Tên phương thức	Mô tả
length(): int	Trả về số lượng ký tự của chuỗi hiện hành
charAt(index: int): char	Trả về ký tự tại vị trí chỉ định trong chuỗi
concat(s1: String): String	Trả về chuỗi mới là sự kết hợp của chuỗi hiện hành với s1

Ví dụ 2.32: Tính độ dài, tìm ký tự theo vị trí và nối chuỗi.

```
public class Teststring2 {
    public static void main(String args[]) {
        String s1 = "Welcome to Java";
        String s2 = " Programming";
        System.out.println(s1.length()); //15
        System.out.println(s1.charAt(0)); //W
        //Welcome to Java Programming
        System.out.println(s1.concat(s2));
    }
}
```

Ví dụ 2.33: Nhập chuỗi và in mỗi ký tự trên 1 dòng.

```
import java.util.*;
public class inkytu{
    public static void main(String[] args) {
        String chuoi;
        char kyTu;
        Scanner inp = new Scanner(System.in);
        System.out.print("Nhap chuoi: ");
        chuoi = inp.nextLine();
        System.out.println("Cac ky tu co trong chuoi: ");
        for (int i = 0; i < chuoi.length(); i++) {
            // trả về ký tự tại vị trí thứ i trong chuỗi
            kyTu = chuoi.charAt(i);
            //In mỗi ký tự 1 dòng
            System.out.println(kyTu);
        }
    }
}
```

2.8.2.3. Trích chuỗi con

Một phần của chuỗi được gọi là chuỗi con hay nói cách khác, chuỗi con là một tập con của chuỗi khác. Phương thức trích chuỗi con substring mô tả bảng 2.12.

Bảng 2.12. Bảng mô tả phương thức trích chuỗi con

Tên phương thức	Mô tả
substring(beginIndex:int): String	Trả về chuỗi con từ vị trí beginIndex đến cuối chuỗi
substring(beginIndex:int, endIndex: int): String	Trả về chuỗi con từ vị trí beginIndex đến vị trí endIndex - 1

Ví dụ 2.34: Cắt chuỗi

```
public class TestSubstring2 {  
    public static void main(String args[]) {  
        String s = "Hello I'm Java";  
        System.out.println(s.substring(5));  
        //I'm Java  
        System.out.println(s.substring(0, 5)); //Hello  
    }  
}
```

2.8.2.4. Một số hàm thường dùng khác

Ngoài các phương thức trên, chúng ta còn gặp một số hàm xử lý chuỗi khác như: Chuyển đổi ký tự (hoa thường), thay thế và tách chuỗi được mô tả bảng 2.13.

Bảng 2.13. Bảng mô tả phương thức chuyển đổi, thay thế, tách chuỗi trong Java

Tên phương thức	Mô tả
toLowerCase(): String	Trả về chuỗi mới mà tất cả các ký tự đều là ký tự thường
toUpperCase(): String	Trả về chuỗi mới mà tất cả các ký tự đều là ký tự hoa
trim(): String	Trả về chuỗi mới mà cắt khoảng trắng ở hai bên chuỗi hiện hành
replace(oldChar: char, newChar: char): String	Trả về chuỗi mới mà thay thế các ký tự phù hợp với oldChar thành newChar

Tên phương thức	Mô tả
split(delimiter: String): String[]	Trả về mảng các chuỗi được tách từ chuỗi hiện hành bởi chuỗi tách delimiter

Ví dụ 2.35: Chuyển đổi chuỗi

```
public class TestString3 {
    public static void main(String args[]) {
        int value = 30;
        String s1 = "HELLO stRING";
        String s1lower = s1.toLowerCase();
        String s1upper = s1.toUpperCase();
        System.out.println(s1lower); // hello string
        System.out.println(s1upper); // HELLO STRING
        String s2 = String.valueOf(value);
        System.out.println(s2 + 10); // 3010
    }
}
```

2.9. QUẢN LÝ NGOẠI LỆ

2.9.1. Khai báo ngoại lệ

Ngoại lệ (Exception) là những lỗi phát sinh khi chương trình thực thi và làm chương trình kết thúc bất thường.

Cú pháp:

```
try{
    //Các câu lệnh có thể sinh lỗi
}
catch(<Kiểu ngoại lệ> ex){
    //Các câu lệnh xử lý ngoại lệ
}
finally{
    // Các câu lệnh xử lý dù có lỗi xảy ra hay không
}
```

Ví dụ 2.36: Một số thông báo lỗi ngoại lệ

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    // Prompt the user to enter two integers
    System.out.print("Enter two integers: ");
    int number1 = input.nextInt();
    int number2 = input.nextInt();
    System.out.println(number1 + " / " + number2 + "
        is " + (number1/number2));
}
```

Enter two integers: 5 0

→ Exception in thread "main"

java.lang.ArithmeticException: /by zero at
sample.Welcome.main (Welcome.java:16)

Ví dụ 2.37: Để đoạn code ở ví dụ 2.36 không bị lỗi tương tự như thế, có thể sử dụng *try – catch* như sau:

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    // Prompt the user to enter two integers
    System.out.print("Enter two integers: ");
    int number1 = input.nextInt();
    int number2 = input.nextInt();
    try {
        System.out.println(number1 + " / "
            + number2 + " is " + (number1/number2));
    }
    catch (Exception ex) {
        System.out.println("Exception: an integer " +
            "cannot be divided by zero ");
    }
}
```

Ví dụ 2.38: Sử dụng *try – catch – finally*

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    // Prompt the user to enter two integers
```

```

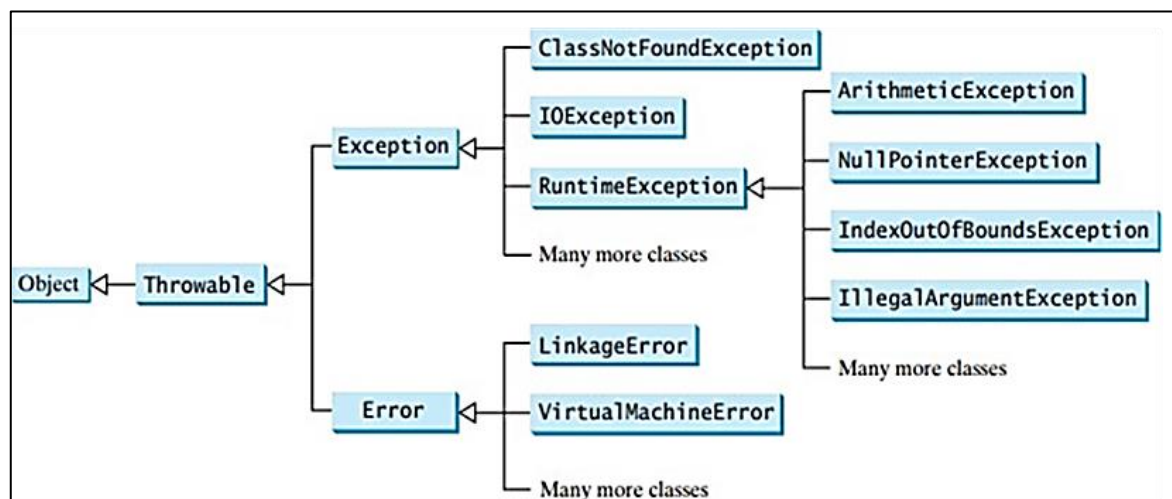
System.out.print("Enter two integers: ");
int number1 = input.nextInt();
int number2 = input.nextInt();
try {
    System.out.println(number1 + " / "
        + number2 + " is " + number1/number2));
}
catch (Exception ex) {
    System.out.println("Exception: an integer " +
        "cannot be divided by zero ");
}
finally{
    System.out.println("End of program");
}
}

```

- Kiểu ngoại lệ:

+ Có 3 kiểu ngoại lệ: system errors, exceptions và runtime exceptions

+ Sơ đồ phân cấp các kiểu ngoại lệ



Hình 2.1. Sơ đồ phân cấp các kiểu ngoại lệ

2.9.2. Lấy thông tin ngoại lệ

Sử dụng các phương thức trong lớp java.lang.Throwable sau như bảng 2.14.

Bảng 2.14. Bảng mô tả một số phương thức lấy thông tin ngoại lệ trong Java

Tên phương thức	Mô tả
getMessage(): String	Trả về thông điệp của đối tượng ngoại lệ
toString(): String	Trả về chuỗi có dạng: (1) tên lớp ngoại lệ; (2) “:”; (3) thông điệp từ getMessage()
printStackTrace(): void	In ra đối tượng Throwable và những thông tin dò tìm lỗi trên console
getStackTrace(): StackTraceElement[]	Trả về mảng các phần tử dò tìm lỗi

- **Một số lưu ý:**

+ Mô tả ngoại lệ cho phương thức: dùng từ khóa *throws* tại phần khai báo phương thức

```
public void myMethod()
    throws Exception1, Exception2,...ExceptionN
```

+ Ném ngoại lệ: dùng từ khóa *throw*

```
throw new IllegalArgumentException("Wrong Argument");
```

2.10. CÂU HỎI ÔN TẬP VÀ BÀI TẬP THỰC HÀNH

2.10.1. Câu hỏi ôn tập

Câu 1: Chương trình sau đây không đúng. Hãy sắp xếp lại các dòng để chương trình hiển thị dòng “morning” trước “afternoon”

```
public void main(String[] args) {
}
public class Welcome {
    System.out.println("afternoon");
    System.out.println("morning");
}
```

Câu 2: Tìm và sửa lỗi trong chương trình sau

```
public class Welcome {
    public void Main(String[] args) {
        System.out.println('Welcome to Java!');
    }})
```

Câu 3: Giả sử `int a = 1` và `double d = 1.0`. Giải thích và cho biết kết quả của các biểu thức sau?

```
1. a = 46 / 9;  
2. a = 46 % 9 + 4 * 4 - 2;  
3. a = 45 + 43 % 5 * (23 * 3 % 2);  
4. a %= 3 / a + 3;  
5. d = 4 + d * d + 4;  
6. d += 1.5 * 3 + (++a);  
7. d -= 1.5 * 3 + a++;
```

Câu 4: Các câu lệnh sau đây có đúng hay không? Nếu có, cho biết kết quả tương ứng của từng câu lệnh:

```
1. System.out.println ("25 / 4 is " + 25 / 4);  
2. System.out.println ("25 / 4.0 is " + 25 / 4.0);  
3. System.out.println ("3 * 2 / 4 is " + 3 * 2 / 4);  
4. System.out.println ("3.0 * 2/4 is " + 3.0 * 2/4);
```

Câu 5: Xác định và sửa các lỗi trong chương trình sau:

```
public class Test {  
    public static void main(String[] args) {  
        int i;  
        int k = 100.0;  
        int j = i+1;  
        System.out.println("j is" + j + " and k is" + k);  
    }  
}
```

Câu 6: Các câu lệnh chuyển đổi kiểu dữ liệu sau có đúng hay không? Nếu có, cho biết kết quả chuyển đổi và giải thích:

A. <code>char c = 'A';</code> <code>int i = (int)c;</code> <code>double d = 1000.34;</code> C. <code>int i = (int)d;</code>	B. <code>float f = 1000.34f;</code> <code>int i = (int)f;</code> <code>int i = 97;</code> D. <code>char c = (char)i;</code>
--	--

Câu 7: Cho biết kết quả của các chương trình sau. (Chú ý: Vẽ table và liệt kê các biến theo cột để hiển thị giá trị biến và kết quả chi tiết ở từng lần lặp).

A.

```
public class Test {  
    public static void main(String[] args) {  
        for (int i = 1; i < 5; i++) {  
            int j = 0;  
            while (j < i) {  
                System.out.print(j + " ");  
                j++;  
            }  
        }  
    }  
}
```

B.

```
public class Test {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 5) {  
            for (int j = i; j > 1; j--)  
                System.out.print(j + " ");  
            System.out.println("*****");  
            i++;  
        }  
    }  
}
```

C.

```
public class Test {  
    public static void main(String[] args) {  
        int i = 5;  
        while (i >= 1) {  
            int num = 1;  
            for (int j = 1; j <= i; j++) {  
                System.out.print(num + "xxx");  
                num *= 2;  
            }  
        }  
    }  
}
```

```

        }
        System.out.println();
        i--;
    }
}
}

```

D.

```

public class Test {
    public static void main(String[] args) {
        int i = 1;
        do {
            int num = 1;
            for (int j = 1; j <= i; j++) {
                System.out.print(num + "G");
                num += 2;
            }
            System.out.println();
            i++;
        } while (i <= 5);
    }
}

```

Câu 8: Cho biết kết quả in ra màn hình của đoạn mã lệnh và giải thích?

```

int x = 30;
int[] numbers = new int[x];
x = 60;
System.out.println("x is " + x);
System.out.println("The size of numbers is " +
    numbers.length);

```

Câu 9: Cho biết phát biểu nào đúng hoặc sai trong các phát biểu sau:

- 1) Mỗi phần tử trong mảng có cùng kiểu dữ liệu.
- 2) Kích thước mảng sẽ cố định sau khi nó được khai báo.
- 3) Kích thước mảng sẽ cố định sau khi nó được khởi tạo.
- 4) Các phần tử trong một mảng phải là kiểu dữ liệu chính.

Câu 10: Các câu lệnh nào không phải câu lệnh khai báo và khởi tạo mảng?

```
int i = new int(30);  
double d[] = new double[30];  
char[] r = new char(1..30);  
int i[] = (3, 4, 3, 2);  
float f[] = {2.3, 4.5, 6.6};  
char[] c = new char();
```

2.10.2. Bài tập thực hành

Câu 1: Viết chương trình yêu cầu người dùng nhập bán kính và chiều cao của hình trụ. Sau đó tính diện tích, thể tích và hiển thị các kết quả ra màn hình. Công thức tính diện tích và thể tích:

Diện tích = bán kính * bán kính * π

Thể tích = Diện tích * chiều cao

Câu 2: Viết chương trình giải phương trình bậc 1: $ax + b = 0$

Câu 3: Viết chương trình giải phương trình bậc 2: $ax^2 + bx + c = 0$

Câu 4: Viết chương trình nhập vào 3 số a, b, c. Kiểm tra 3 số được nhập vào có phải là 3 cạnh của tam giác hay không? Nếu đúng là 3 cạnh của tam giác, cho biết đây là tam giác loại gì?

Câu 5: Viết chương trình tính tổng các số chẵn từ 2 đến N (nguyên dương, nhập N vào từ bàn phím)

Câu 6: Viết chương trình cho phép người dùng nhập vào số giây, sau đó hiển thị ra số phút và số giây.

Câu 7: BMI là chỉ số đo lường sức khỏe dựa trên chiều cao và cân nặng. Nó được tính bởi công thức $(\text{BMI} = \text{cân nặng} / (\text{chiều cao})^2)$

BMI	LỜI KHUYÊN
$\text{BMI} < 18.5$	Bạn quá gầy
$18.5 \leq \text{BMI} < 25.0$	Bạn bình thường
$25.0 \leq \text{BMI} < 30$	Bạn hơi thừa cân một tí
$30.0 \leq \text{BMI}$	Bạn quá béo

Viết chương trình cho phép người dùng nhập vào chiều cao và cân nặng, sau đó hiển thị chỉ số BMI của người dùng.

Câu 8: (Tổng các số trong một số nguyên). Viết 1 chương trình đọc một số nguyên kiểu long và cộng tất cả các số trong số nguyên đó. Ví dụ: Nếu số nguyên là 932 thì tổng tất cả các số là 14.

Câu 9: Viết chương trình sinh ra 2 số nguyên nhỏ hơn 100 và yêu cầu người dùng nhập tổng của 2 số này. Chương trình sau đó kiểm tra giá trị nhập với tổng 2 số đã sinh và trả về true nếu giống nhau, false nếu ngược lại.

Câu 10: Viết chương trình nhập vào số tự nhiên có 2 chữ số. Sau đó so sánh với số ngẫu nhiên được tạo ra từ máy tính.

- Nếu các số trùng với số ngẫu nhiên → được thưởng 10.000\$
- Nếu các số giống với số ngẫu nhiên nhưng khác vị trí → được thưởng 3.000\$
- Nếu các số chỉ giống với 1 số ngẫu nhiên → được thưởng 1.000\$
- Nếu số hoàn toàn khác → 0\$

Câu 11: Viết chương trình đoán số. Nhập vào một số bất kỳ và so sánh với số ngẫu nhiên được tạo ra từ máy tính. Với mỗi lần nhập vào của người dùng, chương trình sẽ đưa ra gợi ý giá trị người dùng nhập vào cao hơn hoặc thấp hơn so với số ngẫu nhiên được tạo ra từ máy tính. Nhập đến khi nào người dùng nhập vào chính xác số ngẫu nhiên được tạo ra từ máy tính.

Câu 12: Viết chương trình nhập mảng số nguyên gồm n phần. Yêu cầu:

- Nhập, in mảng số nguyên;
- Xét mảng số nguyên có phải là mảng tăng; Sắp xếp mảng tăng;
- Đếm số lượng phần tử lẻ;
- Tính tổng dãy số;
- Tìm số lớn nhất (nhỏ nhất) trong mảng;
- Nhập vào một số, tìm số có trong mảng hay không, nếu có đếm số lần xuất hiện trong mảng;
- Chèn thêm một phần tử vào mảng;
- Xóa một phần tử mảng.

Câu 13: Viết chương trình nhập vào một chuỗi tùy ý. Yêu cầu:

- Đếm từ trong chuỗi;
- Đổi chữ cái đầu mỗi từ thành hoa;
- Cắt chữ cuối của chuỗi;
- In chuỗi đảo ngược;
- Nhập vào một ký tự, tìm ký có trong chuỗi, số lần xuất hiện của ký tự.

Câu 14: Viết chương trình nhập 2 ma trận cùng cấp. Yêu cầu:

- Nhập, in 2 ma trận;
- Tính tổng 2 ma trận, in ra ma trận tổng;
- Tính tổng dòng trên ma trận tổng;
- Tính tổng cột trên ma trận tổng;

- Tìm số lớn nhất (nhỏ nhất) trên ma trận tổng;
- Đếm số phần tử chẵn (lẻ) trên ma trận tổng;
- Tính tổng các phần tử chẵn (lẻ) trên ma trận tổng.
- Kiểm tra ma trận A có phải là ma trận thưa không? ($0 \geq \frac{1}{2} \text{sodong} * \text{socot}$)

Câu 15: Viết chương trình hiển thị bảng tính

Multiplication Table									
	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

CHƯƠNG 3. LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Mục tiêu của chương

Sau khi học xong chương này, người học sẽ:

- Trình bày được các khái niệm về lập trình hướng đối tượng như: Đối tượng, lớp, phương thức, thuộc tính, ...;*
- Vận dụng thành thạo khai báo lớp, thiết kế lớp và xây dựng các phương thức xử lý lớp;*
- Lựa chọn tối ưu việc sử dụng các tính chất kế thừa, đa hình trong lập trình hướng đối tượng của Java;*
- Thực hành thành thạo cấu trúc dữ liệu tập hợp;*
- Phân tích được bài toán và cài đặt thành tạo các lớp đối tượng.*

3.1. TỔNG QUAN VỀ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Lập trình hướng đối tượng (OOP – Object Oriented Programming) là một phương pháp để thiết kế một chương trình bởi sử dụng các lớp và các đối tượng hay nói cách khác là một mô hình lập trình sử dụng các lớp (classes) và các đối tượng (objects). Java là một ngôn ngữ lập trình hướng đối tượng vì vậy nó cũng hỗ trợ các đặc tính của lập trình hướng đối tượng bao gồm: Đa hình (Polymorphism), Thừa kế (Inheritance), Đóng gói (Encapsulation), Trừu tượng (Abstraction).

3.1.1. Đối tượng – Object

Một thực thể trong thế giới thực hay sự vật mà chúng ta có thể cảm nắm, hoặc có thể gọi tên được. Ví dụ, để thiết kế phần mềm “*quản lý điểm cho sinh viên*”, thì khi phân tích thì điều đầu tiên chúng ta chọn *sinh viên* hay học phần môn học làm đối tượng để quản lý, và *kết quả* đánh giá môn học của sinh viên cũng là đối tượng. Như vậy, những gì xuất hiện trong một ứng dụng, mà chúng ta dùng làm thực thể để quản lý thì xem nó là một đối tượng. Đối tượng phải có trạng thái và hành vi.

- Trạng thái (State): những thông tin cơ bản để nhận diện hay dùng để nói

lên các đặc thù, hoặc đặc trưng của đối tượng đó. Một trạng thái trong phần mềm máy tính được lưu trữ thành các trường (fields).

- Hành vi (Behavior): là những gì đối tượng làm hay hành động (hoạt động) của đối tượng đó phải thực hiện. Các hành vi thể hiện trong phần mềm máy tính bằng các phương thức.

Ví dụ 3.1. Đối tượng xe máy có các trạng thái như: (1) Số xe, (2) Hiệu xe, (3) Màu sơn xe, (4) Nhà sản xuất, (5) Năm sản xuất. Hành vi của đối tượng xe máy được xác định: (1) Lái xe, (2) Nổ máy, (3) Dừng xe,...

3.1.2. Lớp đối tượng – Class

- Là khái niệm để mô tả các tính chất và hành vi chung của nhiều đối tượng. Cũng có thể nói lớp là một nhóm đối tượng có các thuộc tính chung, lớp là khuôn mẫu để từ đó tạo ra các thực thể (thể hiện). Vậy nên, khi thiết kế một lớp, chúng ta cần nghĩ đến những đối tượng sẽ được tạo ra từ lớp đó.

Lớp được xác định bởi 3 thành phần: (1) tên lớp, (2) trạng thái hay còn gọi thuộc tính lớp – thể hiện của lớp (instance attribute), (3) hành vi hay phương thức của lớp (method). Các phương thức được thiết kế để thao tác trên dữ liệu của đối tượng. Một đối tượng thường có các phương thức đọc/ghi giá trị cho các thể hiện của thực thể.

Tuy nhiên, một số trường hợp lớp đối tượng không có thuộc tính hoặc không có phương thức hoặc không có cả hai.

Ví dụ 3.2: Khoa có các thuộc tính (mã khoa, tên khoa) và các phương thức hành vi (nhập dữ liệu, xuất dữ liệu, lấy thông tin mã khoa, lấy thông tin tên khoa, cập nhật mã khoa, cập nhật tên khoa,...).

```
public class Khoa {  
    //thuộc tính  
    String makh;  
    String tenkh;  
    //Phương thức  
    void hienThiThongTin() {  
        System.out.println("Mã khoa: " + makh);  
        System.out.println("Tên khoa: " + tenkh);  
    }  
}
```

```
}  
  
//Các phương thức khác.....  
  
}
```

3.1.3. Đặc trưng của lập trình hướng đối tượng

Ngôn ngữ lập trình Java cũng như các ngôn ngữ lập trình hướng đối tượng khác cũng có 4 tính chất: (1) tính trừu tượng (Abstraction), (2) tính đóng gói (Encapsulation), (3) tính kế thừa (Inheritance), (4) tính đa hình (Polymorphism).

3.1.3.1. Tính trừu tượng

Tính trừu tượng là từ các mô tả kịch bản của chương trình tìm ra các đặc trưng, các hành vi để trừu tượng hóa thành các thể hiện của thực thể thành các đối tượng, các class.

Ví dụ 3.3: Viết chương trình nhập thông tin sinh viên gồm họ tên, ngày sinh, học lực. Từ yêu cầu bài toán ta rút ra được các danh từ: “sinh viên”, “họ tên”, “ngày sinh”, “học lực”. Xác định: (1) “sinh viên” là 1 đối tượng, (2) các thể hiện “họ tên”, “ngày sinh”, “học lực” là các thuộc tính. Trừu tượng hóa đối tượng sinh viên thành lớp – Class: SinhVien.

3.1.3.2. Tính đóng gói

Đóng gói ở đây là đóng gói các thuộc tính, phương thức thành các class; đóng gói các class thành 1 package...Việc đóng gói giúp che giấu thông tin, đảm bảo sự toàn vẹn dữ liệu.

Ví dụ 3.4: Tính đóng gói mà ta hay dùng nhất đó là qua quyền truy cập (access modifier): public, private, protected, default. Giả sử ta có một class Person với các thuộc tính: name, age

```
public class Person {  
    private String name;  
    private int age;  
    //phương thức - method  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

```
}  
}
```

Các thuộc tính name, age đều để private tức là không thể truy cập, chỉnh sửa dữ liệu name và age từ class bên ngoài, khi muốn thiết lập, lấy thông tin name, age ta phải thông qua các hàm get, set. Các class bên ngoài sẽ không biết được name và age trong class Person lưu trữ, từ đó đảm bảo toàn vẹn dữ liệu.

3.1.3.3. Tính kế thừa

Tính chất này thì đơn giản hơn, đó là sử dụng lại các thuộc tính, phương thức sẵn có từ các class khác mà không phải xây dựng từ đầu.

Ví dụ 3.5: Để xây dựng 1 class Student cũng có các thuộc tính giống của Person và thêm thuộc tính address thì chúng ta chỉ việc kế thừa lại class Person và thêm thuộc tính address.

```
public class Student extends Person {  
    private String address;  
    public String getAddress() {  
        return address;  
    }  
    public void setAddress(String address) {  
        this.address = address;  
    }  
}
```

3.1.3.4. Tính đa hình

Tính đa hình ở đây được hiểu là cùng 1 phương thức nhưng tùy vào tham số truyền vào hoặc cài đặt ở lớp con mà nó thực hiện các phép toán khác nhau.

Ví dụ 3.6: Để ghi đè (Overriding) hay định nghĩa lại phương thức mà lớp thừa kế từ lớp cha đã định nghĩa trước đó.

```
public class Person {  
    public void show() {  
        System.out.println("person show");  
    }  
}  
public class Student extends Person {  
    @Override
```

```

public void show() {
    System.out.println("student show");
}
}

```

3.2. XÂY DỰNG LỚP

3.2.1. Khai báo lớp

Cú pháp khai báo lớp như sau:

```

<Quyền truy cập> class <Tên lớp>{
    // Định nghĩa Data Fields (Thuộc tính dữ liệu)
    // Định nghĩa Methods (Phương thức)
}

```

Trong đó:

- Quyền truy cập: xác định phạm vi sử dụng lớp. Có thể là: public hoặc không chỉ định.
- Data Fields (Thuộc tính dữ liệu): là biến lưu trữ các dữ liệu cho lớp, còn gọi là thành viên dữ liệu (data members).
- Methods (Phương thức): Định nghĩa các hoạt động xử lý cho lớp. Phương thức sử dụng các trường dữ liệu.

Ví dụ 3.7: Định nghĩa lớp Circle (Hình tròn)

```

public class Circle {
    double radius = 1.0; // Thuộc dữ liệu

    ...

    public double getArea() { // Phương thức
        return radius * radius * Math.PI;
    }
}

```

3.2.2. Định nghĩa thuộc tính dữ liệu

Tương tự như khai báo biến

```

<Quyền truy cập> <Kiểu dữ liệu> <Tên biến>;

```

Quyền truy cập: xác định phạm vi truy cập thuộc tính dữ liệu từ các đối tượng, có thể là: public, protected, private hoặc không chỉ định.

Quyền truy cập	Lớp (Class)	Gói (Package)	Lớp con (Subclass)	Ngoài
public	Có	Có	Có	Có
protected	Có	Có	Có	Không
Không chỉ định	Có	Có	Không	Không
private	Có	Không	Không	Không

- Có 2 dạng trường dữ liệu:

+ Trường tĩnh (Static fields): được chia sẻ qua tất cả các đối tượng và truy xuất thông qua tên lớp. Khai báo bằng cách thêm từ khóa **static** trước tên trường.

+ Trường không tĩnh (Non-static fields): mỗi đối tượng có bản sao của trường này, được truy xuất qua tên đối tượng và khai báo như cú pháp đã nêu trên.

Ví dụ 3.8: Định nghĩa lớp Circle (Hình tròn)

```
public class Circle {
    double radius = 1.0; //Trường dữ liệu không tĩnh
    static int count = 0; //Trường dữ liệu tĩnh
    ...
    public double getArea() {
        return radius * radius * Math.PI;
    }
}
```

3.2.3. Định nghĩa phương thức

Phương thức trong Java là khối lệnh thực hiện các chức năng, các hành vi xử lý của lớp lên vùng dữ liệu.

- Định nghĩa như các phương thức thông thường:

```
<Quyền truy cập> <Kiểu trả về> <Tên PT> (DS tham số) {
    //Thân phương thức
}
```

- Quyền truy cập: xác định phạm vi truy xuất của các đối tượng khác đối với các phương thức của lớp. Các quyền truy cập thường được sử dụng:

+ **public**: phương thức có thể truy cập được từ bên ngoài lớp khai báo.

+ **protected**: có thể truy cập được từ lớp khai báo và những lớp dẫn xuất từ nó. Đây là thành phần được bảo vệ, bị hạn chế truy nhập từ bên ngoài.

+ **private**: chỉ được truy cập bên trong bản thân lớp khai báo.

🚩 Từ khóa **static**, **final**, **abstract** trong Java

- **static**: phương thức lớp dùng chung cho tất cả các thể hiện của lớp, có nghĩa là phương thức đó có thể được thực hiện kể cả khi không có đối tượng của lớp chứa phương thức đó.
- **final**: phương thức bỏ từ truy cập này không được khai báo chồng ở các lớp dẫn xuất.
- **abstract**: phương thức không cần cài đặt (không có phần source code) sẽ được thực hiện trong các lớp dẫn xuất từ lớp này.

Ví dụ 3.9: Định nghĩa phương thức

```
public class Circle{
    private double radius = 1.0;
    private static int count = 0;
    /* Phương thức xử lý diện tích */
    public double getArea() {
        return radius * radius * Math.PI;
    }
    /* Phương thức xử lý chu vi */
    public double getDiameter() {
        return 2 * radius;
    }
}
```

- Có 2 dạng phương thức:

+ **Phương thức tĩnh** (Static methods): được chia sẻ qua các đối tượng và truy xuất qua tên lớp. Khai báo bằng cách thêm từ khóa **static** trước tên phương thức

+ **Phương thức không tĩnh** (Instance methods): được tạo ra tương ứng với mỗi đối tượng và truy xuất qua tên đối tượng. Khai báo thông thường như cú pháp đã nêu.

3.2.4. Từ khóa **this**

- Mỗi phương thức không tĩnh có một biến với tên là **this** dùng để tham chiếu đến đối tượng hiện hành mà phương thức đang được gọi.

Ví dụ 3.10: Sử dụng từ khóa **this** trong phương thức

```
public class Circle {  
    double radius = 1.0;  
    ...  
    public double getArea() {  
        return this.radius * this.radius * Math.PI;  
    }  
    public String toString() {  
        return "radius: " + this.radius + "area: " +  
            this.getArea() ;  
    }  
}
```

3.2.5. Phương thức khởi tạo – Constructors

- Là phương thức đặc biệt được tự động gọi mỗi khi tạo đối tượng và được sử dụng để khởi tạo giá trị cho các trường dữ liệu.

- Nếu không định nghĩa phương thức khởi tạo, trình biên dịch sẽ gọi phương thức khởi tạo mặc định, phương thức này không tham số và không làm gì.

- Phương thức khởi tạo có tên trùng tên lớp, không có kiểu trả về kể cả void.

- Phương thức khởi tạo được gọi bằng việc sử dụng toán tử **new** khi tạo đối tượng.

- Phương thức khởi tạo được chia thành 2 loại

 - + Khởi tạo mặc định (không tham số)

```
<Quyền truy cập> <Tên PT> () { // Tên PT trùng với tên lớp  
    //Thân phương thức  
}
```

 - + Khởi tạo có đối số

```
<Quyền truy cập> <Tên PT> (DS tham số) {  
    //Thân phương thức  
}
```

Ví dụ 3.11: Tạo 2 phương thức khởi tạo cho lớp Circle

```
class Circle {
    double radius = 1;
    /* Phương thức khởi tạo không tham số */
    Circle() {
    }
    /* Phương thức khởi tạo có đối số */
    Circle(double newRadius) {
        radius = newRadius;
    }
    /* Phương thức tính diện tích */
    double getArea() {
        return radius * radius * Math.PI;
    }
}
```

3.3. SỬ DỤNG ĐỐI TƯỢNG

3.3.1. Khai báo đối tượng

Cú pháp:

```
<Tên lớp> <Tên đối tượng> = new <Tên lớp>();
```

Ví dụ 3.12: Khai báo 2 đối tượng Circle c1 và c2

```
public static void main (String [] args){
    Circle c1 = new Circle();
    // đối tượng c2 có bán kính là 25
    Circle c2 = new Circle(25);
}
```

3.3.2. Truy cập thuộc tính dữ liệu/phương thức

Cú pháp:

```
<Tên đối tượng>.<Thuộc tính dữ liệu>/
[<Tên phương thức>];
```

Ví dụ 3.13: Truy cập dữ liệu và truy cập phương thức

```
public static void main (String [] args){
    Circle c1 = new Circle();
    // Cách truy cập thuộc tính dữ liệu
    c1.radius = 2.5;
    //Truy cập phương thức
    System.out.println(c1.getArea());
}
```

3.4. ĐÓNG GÓI TRƯỜNG DỮ LIỆU

- Đóng gói trường dữ liệu (data field encapsulation) là ngăn không cho truy xuất trực tiếp trường dữ liệu từ bên ngoài lớp.
- Các thuộc tính dữ liệu này được khai báo với quyền truy cập: *private*
- Phương pháp truy cập: có 2 phương pháp thường sử dụng (1) Get: trả về giá trị của thuộc tính dữ liệu; và (2) Set: thiết lập giá trị mới cho trường dữ liệu.

3.4.1. Phương thức get

Cú pháp:

```
public <kiểu trả về> get <Thuộc tính dữ liệu>() {
    //Mã lệnh
}
```

- Nếu kiểu trả về là boolean, phương thức get được định nghĩa như sau:

```
public <kiểu trả về> is <Thuộc tính dữ liệu>() {
    //Mã lệnh
}
```

3.4.2. Phương thức set

Cú pháp:

```
public void set <Tên trường dữ liệu>
    (<Kiểu dữ liệu> <tên biến>){
    //Mã lệnh
}
```

Ví dụ 3.14: Phương pháp truy cập *get, set*

```
public class Circle{
    private double radius;
    private static int count = 0;
    public double getRadius() {
        return radius;
    }
    /*Set a new radius */
    public void setRadius(double radius) {
        this.radius = radius;
    }
    /** Return area */
    public double getArea() {
        return radius * radius * Math.PI;
    }
    /** Return diameter */
    public double getDiameter() {
        return 2 * radius;
    }
}
```

3.5. KẾ THỪA VÀ ĐA HÌNH

3.5.1. Tính kế thừa

- Kế thừa (Inheritance) là cơ chế cho phép tạo ra 1 lớp mới từ lớp tổng quát đã có. Lớp mới này thừa kế các thuộc tính và phương thức từ lớp tổng quát.

- Lớp mới được tạo từ lớp tổng quát gọi là lớp con (subclass) và lớp tổng quát gọi là lớp cha (superclass).

- Lớp con có thể truy cập các trường dữ liệu và phương thức từ lớp cha và cũng có thể thêm thuộc tính dữ liệu và phương thức của nó.

- Định nghĩa lớp kế thừa:

```
<Quyền truy cập> <Tên lớp con> extends <Tên lớp cha>{
    //Thuộc tính dữ liệu bổ sung
    //Phương thức bổ sung
}
```

Ví dụ 3.15: Programmer là lớp con và Employee là lớp cha. Mối quan hệ giữa hai lớp là Programmer IS-A (is a member of) Employee. Nghĩa là Programmer là một kiểu của Employee.

```
class Employee{
    float salary=40000;
}
class Programmer extends Employee{
    int bonus=10000;
public static void main(String args[]){
    Programmer p=new Programmer();
    System.out.println("Luong Lap trinh vien la:"
        + p.salary);
    System.out.println("Bonus cua Lap trinh vien
        la:" +p.bonus);
}
}
```

Đối tượng Programmer có thể truy cập trường của riêng lớp nó cũng như của lớp Employee, đó là ví dụ cho tính tái sử dụng.

Lưu ý:

- Lớp con không phải là tập con của lớp cha. Thực chất, lớp con thường chứa nhiều thông tin và phương thức hơn lớp cha.

- Các trường dữ liệu là private thì không thể được truy cập trực tiếp trong lớp con. Nếu muốn truy cập trong lớp con thì nó phải là public.

- Không phải một quan hệ is – a đều có thể kế thừa. Chẳng hạn, hình vuông là một (is - a) hình chữ nhật nhưng không thể định nghĩa lớp hình vuông (Square) mở rộng từ lớp hình chữ nhật (Rectangle) vì không có gì để có thể mở rộng từ hình chữ nhật sang hình vuông.

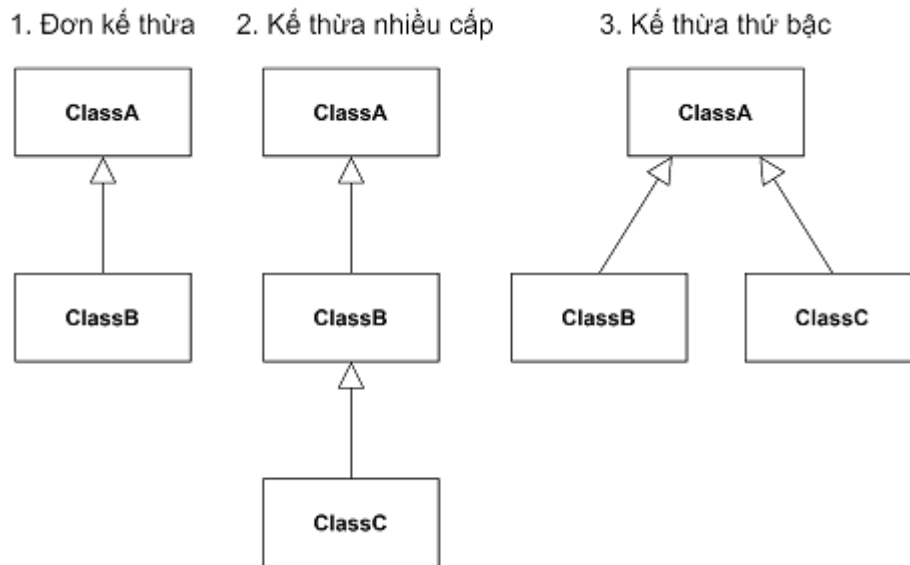
- Kế thừa là mô hình hóa của mối quan hệ is – a. Một lớp con và lớp cha của nó phải có mối quan hệ is – a.

- Kế thừa trong Java là kế thừa đơn nghĩa là một lớp chỉ có thể kế thừa trực tiếp từ một lớp cha.

❖ Các loại kế thừa trong Java

Có 3 kiểu kế thừa trong Java đó là đơn kế thừa, kế thừa nhiều cấp, kế thừa

thứ bậc. Khi một class được kế thừa từ nhiều class được gọi là đa kế thừa. Trong Java, đa kế thừa chỉ được cung cấp thông qua interface.



Hình 3.1. Minh họa các loại kế thừa trong Java

Ví dụ 3.16: Ví dụ về kế thừa đơn

```
class A{
    public void hello(){
        System.out.println("Hello!");
    }
}
class C extends A{
    public void helloJava(){
        System.out.println("Hello Java!");
    }
}
class Test{
    public static void main(String[] args){
        C obj = new C();
        obj.hello();
        obj.helloJava();
    }
}
```

→ Kết quả: Hello!

Hello Java!

Ví dụ 3.17: Ví dụ về kế thừa nhiều tầng

```
class A{
    public void hello(){
        System.out.println("Hello!");
    }
}
class B extends A{
    public void hellWorld(){
        System.out.println("Hello World!");
    }
}
class C extends B{
    public void helloJava(){
        System.out.println("Hello Java!");
    }
}
class Test{
    public static void main(String[] args){
        C obj = new C();
        obj.hello();
        obj.hellWorld();
        obj.helloJava();
    }
}
```

→ Kết quả: Hello!
Hello World!
Hello Java!

Ví dụ 3.18: Ví dụ về kế thừa thứ bậc

```
class A{
    public void hello(){
        System.out.println("Hello!");
    }
}
class B extends A{
```



```

        public void hellWorld() {
            System.out.println("Hello World!");
        }
    }
    class C extends A{
        public void helloJava() {
            System.out.println("Hello Java!");
        }
    }
    class Test{
        public static void main(String[] args){
            C obj = new C();
            obj.hello();
            obj.helloJava();
        }
    }
}

```

→ Kết quả: Hello!

Hello Java!

3.5.1.1. Từ khóa *super*

- Từ khóa *super* được sử dụng trong lớp con để tham chiếu đến lớp cha. Nó được sử dụng theo 2 cách: (1) Gọi phương thức khởi tạo của lớp cha; (2) Gọi phương thức tính toán của lớp cha.

❖ **Cú pháp gọi phương thức xây dựng lớp cha:**

`super()` hoặc `super (<các tham số>)`

Trong đó:

+ `super()`: gọi phương thức xây dựng không tham số từ lớp cha

+ `super (<các tham số>)`: gọi phương thức xây dựng từ lớp cha có <các tham số> phù hợp.

Ví dụ 3.19: Ví dụ về phương thức xây dựng lớp cha

```

class Vehicle {
    Vehicle() {
        System.out.println("Vehicle is created");
    }
}

```

```

    }
    class Bike2 extends Vehicle {
        Bike2() {
            super(); //gọi Constructor của lớp cha
            System.out.println("Bike is created");
        }
        public static void main(String args[]) {
            Bike2 b = new Bike2();
        }
    }
}

```

→ Kết quả: Vehicle is created

Bike is created

Các câu lệnh `super()` và `super(<các tham số>)` phải xuất hiện tại dòng đầu tiên trong phương thức xây dựng của lớp con. Đây là cách duy nhất để gọi tường minh hàm xây dựng của lớp cha.

Phương thức khởi tạo được sử dụng để xây dựng thể hiện của lớp. Phương thức khởi tạo từ lớp cha không thể kế thừa từ trong lớp con. Chúng chỉ có thể được gọi trong hàm xây dựng của lớp con bằng việc sử dụng từ khóa `super`.

Một phương thức xây dựng có thể gọi một phương thức xây dựng nạp chồng hoặc phương thức xây dựng từ lớp cha của nó. Nếu không chỉ định tường minh, trình biên dịch tự động thêm `super()` tại dòng đầu tiên của phương thức xây dựng.

Khi tạo một đối tượng, phương thức xây dựng đầu tiên của lớp con sẽ gọi phương thức xây dựng của lớp cha trước khi thực hiện các tác vụ. Nếu lớp cha kế thừa từ một lớp khác, phương thức xây dựng lớp cha gọi phương thức xây dựng của lớp cha của nó trước khi thực hiện các tác vụ. Tiến trình này tiếp tục cho đến phương thức xây dựng sau cùng.

Nếu một lớp được định nghĩa cho việc kế thừa, cũng nên định nghĩa phương thức xây dựng không tham số để tránh lỗi.

❖ **Cú pháp gọi phương thức tính toán lớp cha:**

```

super.method(<các tham số>)

```

Ví dụ 3.20: Ví dụ gọi phương thức tính toán lớp cha

```
class Person {
    void message() {
        System.out.println("welcome");
    }
}
public class Student extends Person {
    void message() {
        System.out.println("welcome to java");
    }
    void display() {
        //gọi phương thức message() của lớp hiện tại
        message();
        //gọi phương thức message() của lớp cha
        super.message();
    }
    public static void main(String args[]) {
        Student s = new Student();
        s.display();
    }
}
```

→ Kết quả: welcome to java

Welcome

3.5.1.2. Ghi đè phương thức – Overriding Methods

Ghi đè phương thức là cơ chế cho phép lớp con chỉnh sửa sự cài đặt một phương thức được định nghĩa trong lớp cha.

Ví dụ 3.21: Ghi đè phương thức *run()* của lớp cha *Vehicle* để hiển thị thông tin của lớp *Bike*

```
class Vehicle{
    void run() {
        System.out.println("Vehicle đang chạy");
    }
}
```

```

class Bike extends Vehicle{
void run() {
    System.out.println("Bike đang chạy an toàn");
}
public static void main(String args[]){
    Bike2 obj = new Bike2();
    obj.run();
}
}

```

Lưu ý:

- Phương thức không phải phương thức tĩnh có thể ghi đè nếu nó có khả năng truy cập. Phương thức private thì không thể ghi đè.

- Phương thức tĩnh cũng có thể được kế thừa. Tuy nhiên, nó không thể ghi đè. Nếu phương thức tĩnh trong lớp cha được định nghĩa lại trong lớp con thì phương thức tĩnh trong lớp cha là hidden. Để gọi các phương thức tĩnh dạng hidden có thể sử dụng cú pháp:

<Tên lớp cha>.<Phương thức tĩnh>

3.5.1.3. Ghi đè và nạp chồng

- Nạp chồng (overloads) có nghĩa là định nghĩa nhiều phương thức cùng tên nhưng khác danh sách tham số hoặc kiểu trả về.

- Ghi đè (overrides) cung cấp một sự cài đặt mới cho một phương thức trong lớp con. Phương thức này đã được định nghĩa trong lớp cha.

- Để ghi đè phương thức, phương thức phải định nghĩa trong lớp con cùng tên, tham số và kiểu trả về.

Ví dụ 3.22. Phương thức ghi đè – override

```

public class Test {
public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
}
}
class B {
    public void p(double i) {

```

```

        System.out.println(i * 2);
    }
}
class A extends B {
    // This method overrides the method in B
    public void p(double i) {
        /* Phương thức p(double i) trong class A ghi đè
        phương thức giống như trong lớp B */
        System.out.println(i);
    }
}

```

Ví dụ 3.23. Phương thức nạp chồng (Class A có 2 phương thức nạp chồng p(double i) và p(int i). Phương thức p(double i) kế thừa từ class B)

```

public class Test {
    public static void main(String[] args) {
        A a = new A();
        /* Class A có 2 phương thức nạp chồng p(double i)
        và p(int i). Phương thức p(double i) kế thừa
        từ class B*/
        a.p(10.0);
        a.p(10);
    }
}
class B {
    public void p(double i) {
        System.out.println(i * 2);
    }
}
class A extends B {
    // This method overloads the method in B
    public void p(int i) {
        System.out.println(i);
    }
}

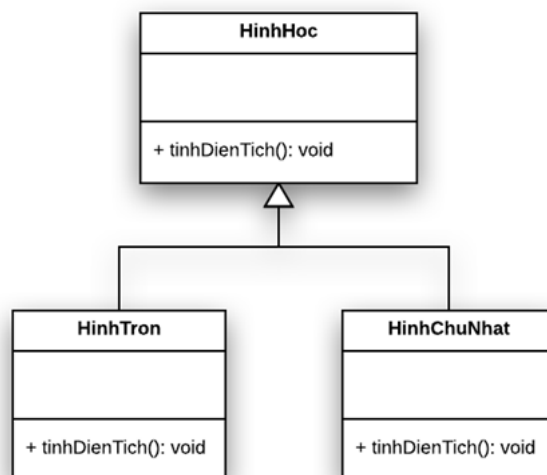
```

3.5.2. Tính đa hình

Đa hình là cơ chế một đối tượng của lớp con có thể được sử dụng ở bất cứ nơi nào đối tượng lớp cha của nó được sử dụng. Hoặc đơn giản: đa hình có nghĩa một biến có kiểu cha có thể tham chiếu đến đối tượng kiểu con.

Một thể hiện của lớp con có thể được chỉ định (thay thế) cho tham chiếu của lớp cha. Sau khi được thay thế, chúng ta có thể gọi các phương thức được định nghĩa trong lớp cha; chúng ta không thể gọi các phương thức được định nghĩa trong lớp con. Tuy nhiên, nếu lớp con ghi đè các phương thức được kế thừa từ lớp bậc trên, các phiên bản lớp con (ghi đè) sẽ được gọi.

Ví dụ 3.24: Lớp HìnhHoc là lớp cha, hai lớp con HìnhTron và HìnhChuNhat đều có override phương thức tinhDienTich() từ lớp cha như sau:



Hình 3.2. Minh họa đa hình của lớp Hình học

```
public class HìnhHoc {
    public void tinhDienTich() {
        System.out.println("Chưa biết hình nào");
    }
}

public class HìnhTron extends HìnhHoc {
    @Override
    public void tinhDienTich() {
        System.out.println("Đây là Diện tích
                               hình Tròn");
    }
}
```

```

    }
}
public class HìnhChuNhat extends HìnhHoc {
    @Override
    public void tinhDienTich() {
        System.out.println("Đây là Diện tích hình
                           Chữ nhật");
    }
}
public class MainClass {
    public static void main(String[] args) {
        HìnhHoc hìnhHoc = new HìnhHoc();
        hìnhHoc.tinhDienTich();
        /* Đoạn code này bình thường, sẽ in ra
           "Chưa biết hình nào"
           Có lúc hìnhHoc đóng vai trò là HìnhTron
           trong một ngữ cảnh nào đó*/
        hìnhHoc = new HìnhTron();
        hìnhHoc.tinhDienTich();
        /* Đoạn code này sẽ in ra
           "Đây là Diện tích Hình tròn"
           Có lúc hìnhHoc đóng vai trò là
           HìnhChuNhat trong một ngữ cảnh nào đó*/
        hìnhHoc = new HìnhChuNhat();
        hìnhHoc.tinhDienTich();
        /* Đoạn code này sẽ in ra
           "Đây là Diện tích Chữ nhật"*/
    }
}

```

3.6. CẤU TRÚC DỮ LIỆU TẬP HỢP (COLLECTIONS)

3.6.1. Giới thiệu

Một cấu trúc dữ liệu là một tập hợp dữ liệu được tổ chức theo một số khuôn dạng nào đó, hỗ trợ lưu trữ, truy xuất và khai thác dữ liệu.

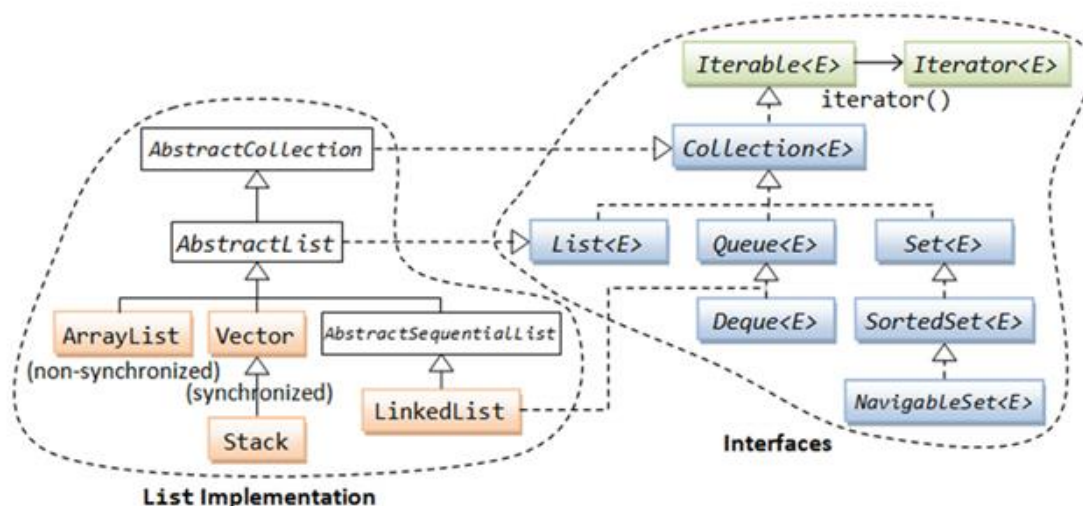
Trong OOP, cấu trúc dữ liệu, còn gọi là khung chứa (container)/đối tượng chứa (container object), là một đối tượng lưu trữ các đối tượng khác.

Java cung cấp nhiều cấu trúc dữ liệu được sử dụng để tổ chức và khai thác dữ liệu một cách hiệu quả. Java hỗ trợ 2 kiểu cấu trúc dữ liệu: (1) Collection (Tập hợp) dùng để lưu trữ một tập các phần tử; (2) Map dùng lưu trữ các cặp key/value.

Collection là một đối tượng mà nó nhóm các đối tượng khác thành phần tử và cung cấp các phương thức cơ bản để thêm, xóa, lấy, duyệt các phần tử... Phần tử của Collection không được phép là kiểu nguyên thủy.

Collections Framework thống nhất cách thức sử dụng các collection minh họa như hình 3.3, gồm 3 thành phần chính: (1) Giao diện; (2) Lớp triển khai; (3) Thuật toán.

Đối tượng Iterator được sử dụng để duyệt qua tất cả các phần tử của collection. Collection được xây dựng dựa trên kỹ thuật lập trình tổng quát, ưu điểm là tiện dụng và hiệu năng cao.



Hình 3.3. Minh họa Cấu trúc dữ liệu tập hợp

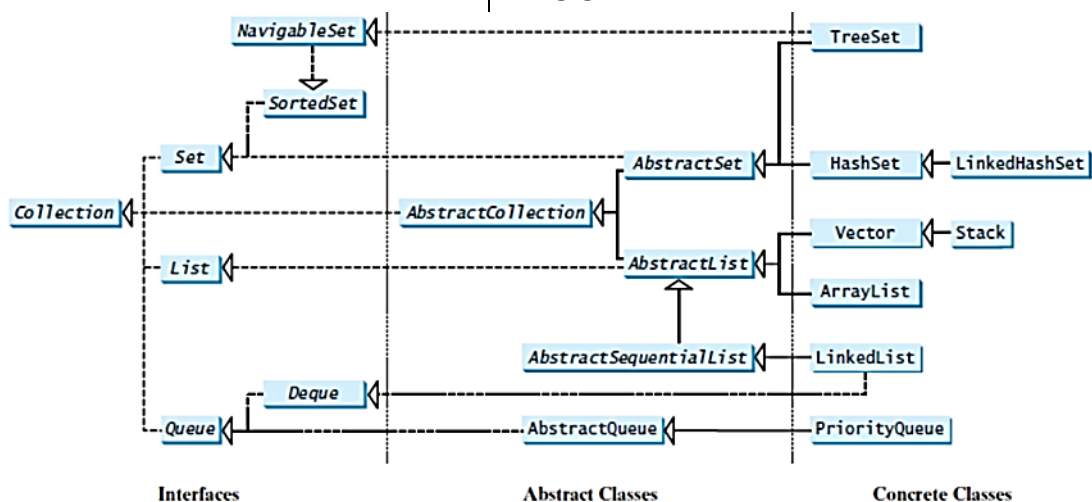
3.6.1.1. Collections interface

- Collection interface định nghĩa các phương thức chung cho các lớp Lists, Vectors, Stacks, Queues, Sets.
- Các phương thức chung thể hiện như bảng 3.1

Bảng 3.1. Các phương thức của Collection interface

Tên phương thức	Chức năng
boolean add(Object obj)	Thêm phần tử tới Collection đang gọi. Trả về true nếu phần tử được thêm tới Collection đó. Trả về false nếu phần tử đã là một thành viên

Tên phương thức	Chức năng
	trong Collection đó, hoặc nếu Collection đó không cho phép các bản sao
boolean addAll(Collection c)	Thêm tất cả phần tử của c tới Collection đang gọi. Trả về true nếu hoạt động này thành công (ví dụ: phần tử được thêm thành công). Nếu không là false
void clear()	Gỡ bỏ tất cả phần tử từ Collection đang gọi
boolean contains(Object obj)	Trả về true nếu phần tử là một phần tử của Collection đang gọi. Nếu không là false
boolean containsAll(Collection c)	Trả về true nếu Collection đang gọi chứa tất cả phần tử của c. Nếu không là false
boolean equals(Object obj)	Trả về true nếu Collection đang gọi và phần tử là cân bằng nhau. Nếu không là false
int hashCode()	Trả về hashcode cho Collection đang gọi này
boolean isEmpty()	Trả về true nếu Collection đang gọi là trống. Nếu không là false
int size()	Trả về số phần tử được giữ trong Collection đang gọi



Hình 3.4. Mối quan hệ giữa Collection và các lớp cài đặt

3.6.1.2. Iterator

Iterator là một lớp cung cấp cách thức đồng nhất để duyệt qua các phần tử trong nhiều kiểu tập hợp. Iterator cung cấp các phương thức chi tiết trong bảng 3.2.

Bảng 3.2. Các phương thức của Iterator

Tên phương thức	Chức năng
public boolean hasNext()	Trả về true nếu iterator còn phần tử kế tiếp phần tử đang duyệt.
public Object next()	Trả về phần tử hiện tại và di chuyển con trỏ tới phần tử tiếp theo.
void remove()	Loại bỏ phần tử cuối được trả về bởi Iterator. Nó hiếm khi được sử dụng.

3.6.1.3. Comparator interface

- Comparator có thể được sử dụng để so sánh các đối tượng của một lớp.
- Sử dụng Comparator: Định nghĩa một lớp cài đặt java.util.Comparator <T> và ghi đè phương thức compare.

```
public int compare (T element1, T element2)
```

- Phương thức này trả về:
 - + Một giá trị < 0: nếu element 1 < element2
 - + Một giá trị > 0: nếu element 1 > element2
 - + Một giá trị = 0: nếu element 1 = element2

3.6.2. ArrayList

Lớp ArrayList trong Java kế thừa AbstractList và triển khai List Interface. Lớp ArrayList hỗ trợ các mảng động mà có thể tăng kích cỡ nếu cần.

Các mảng trong Java chuẩn là có độ dài cố định. Sau khi các mảng được tạo, chúng không thể tăng hoặc giảm kích cỡ, người dùng phải xác định được số lượng phần tử có trong mảng.

ArrayList được tạo với một kích cỡ ban đầu. Khi kích cỡ này bị vượt, Collection tự động được tăng. Khi các đối tượng bị gỡ bỏ, ArrayList có thể bị giảm kích cỡ. ArrayList cung cấp các phương thức trong bảng 3.3.

Bảng 3.3. Các phương thức của ArrayList

Tên phương thức	Chức năng
ArrayList()	Khởi tạo một danh sách mảng trống.
boolean add(Object o)	Nối thêm phần tử được chỉ định vào cuối một danh sách.
void add(int index, Object o)	Chèn phần tử element tại vị trí index vào danh sách.
void clear()	Xóa tất cả các phần tử từ danh sách.

boolean contains(o)	Kết quả trả về là true nếu tìm thấy element trong danh sách, ngược lại trả về false.
Object get(int index)	Trả về phần tử tại index đã cho
int indexOf(Object o)	Trả về vị trí của phần tử tìm thấy đầu tiên
boolean isEmpty()	Trả về true nếu danh sách không chứa phần tử nào
int lastIndexOf(Object o)	Trả về vị trí của phần tử tìm thấy cuối cùng
boolean remove(Object o)	Xóa phần tử o ra khỏi danh sách
boolean remove(int index)	Xóa phần tử khỏi danh sách tại vị trí index
int size()	Trả về số lượng phần tử trong danh sách
Object set(int index, Object o)	Thiết lập phần tử tại vị trí index

Ví dụ 3.25: Minh họa một số phương thức trong ArrayList

```
import java.util.*;
public class ArrayListDemo {
    public static void main(String args[]) {
        // tao mot array list
        ArrayList al = new ArrayList();
        System.out.println("Size ban dau cua ArrayList
                           la: " + al.size());
        // them cac phan tu toi array list
        al.add("C");
        al.add("A");
        al.add("E");
        al.add("B");
        al.add("D");
        al.add("F");
        al.add(1, "A2");
        System.out.println("Size cua ArrayList sau khi
                           them la: " + al.size());
        // display the array list
        System.out.println("Noi dung cua ArrayList la: "
                           + al);
        // xoa cac phan tu tu array list
        al.remove("F");
        al.remove(2);
    }
}
```

```

        System.out.println("Size của ArrayList sau khi
                           xoa la: " + al.size());
        System.out.println("Nôi dung của ArrayList la: "
                           + al);
    }
}

```

→ Kết quả: Size ban đầu của ArrayList là: 0

Size của ArrayList sau khi thêm là: 7

Nội dung của ArrayList là: [C, A2, A, E, B, D, F]

Size của ArrayList sau khi xóa là: 5

Nội dung của ArrayList là: [C, A2, E, B, D]

3.7. CÂU HỎI ÔN TẬP VÀ BÀI TẬP THỰC HÀNH

3.7.1. Câu hỏi ôn tập

Câu 1: Cho biết điểm khác biệt giữa phương thức khởi tạo (constructor) và phương thức tính toán thông thường (methods)?

Câu 2: Nêu điểm sai (có giải thích) trong các đoạn chương trình sau?

A.

```

public class ShowErrors {
    public static void main(String[] args){
        ShowErrors t = new ShowErrors(5);
    }
}

```

B.

```

public class ShowErrors {
    public static void main(String[] args) {
        ShowErrors t = new ShowErrors();
        t.x();
    }
}

```

C.

```

public class ShowErrors {
    public void method1() {
        Circle c;
    }
}

```

```

        System.out.println("What is radius"+ c.getRadius());
        c = new Circle();
    }
}

```

D.

```

public class ShowErrors {
    public static void main(String[] args) {
        C c = new C(5.0);
        System.out.println(c.value);
    }
}

class C {
    int value = 2;
}

```

Câu 3: Chỉ ra chỗ sai (có giải thích) trong đoạn mã sau?

```

class Test {
    public static void main(String[] args) {
        A a = new A();
        a.print();
    }
}

class A {
    String s;
    A(String newS) {
        s = newS;
    }
    public void print() {
        System.out.print(s);
    }
}

```

Câu 4: Cho biết kết quả (có giải thích) của đoạn mã sau?

```

public class Foo() {
    private boolean x;
    public static void main (String [] args){
        Foo foo = new Foo();
        System.out.println(foo.x);
    }
}

```

Câu 5: Giả sử lớp Foo đã được định nghĩa và f là một đối tượng của Foo. Khi thực thi gọi lại đối tượng trong lớp Foo, kiểm tra xem câu lệnh nào đúng, câu lệnh nào sai (Giải thích)?

```
public class Foo{
    int i;
    static String s;
    void imethod() {
    }
    static void smethod() {
    }
}

System.out.println(f.i);
System.out.println(f.s);
f.imethod();
f.smethod();
System.out.println(Foo.i);
System.out.println(Foo.s);
Foo.imethod();
Foo.smethod();
```

Câu 6: Cho biết chỗ sai (có giải thích) của đoạn chương trình sau?

```
public class C {
    public static void main(String[] args) {
        method1();
    }
    public void method1() {
        method2();
    }
    public static void method2() {
        System.out.println("What is radius " +
                           c.getRadius());
    }
    Circle c = new Circle();
}
```

Câu 7: Trong đoạn mã sau, radius là private trong lớp Circle, và myCircle là một đối tượng của lớp Circle. Code được tô màu bên dưới có thể gây ra lỗi không? Giải thích?

```
public class Circle {  
    private double radius = 1;  
    /** Find the area of this circle */  
    public double getArea() {  
        return radius * radius * Math.PI;  
    }  
    public static void main(String[] args) {  
        Circle myCircle = new Circle();  
        System.out.println("Radius is " + myCircle.radius);  
    }  
}
```

3.7.2. Bài tập thực hành

Câu 1: Tạo lớp Person gồm có các thuộc tính name, age, address và các phương thức như trong bảng sau:

Phương thức	Mô tả
Person()	Phương thức khởi tạo mặc định
Person(String, int, String)	Phương thức khởi tạo với 3 đối số
getName()	Trả về giá trị thuộc tính name
setName(String)	Thiết lập giá trị mới cho thuộc tính name
getAge()	Trả về giá trị thuộc tính age
setAge(int)	Thiết lập giá trị mới cho thuộc tính age
getAdd()	Trả về giá trị thuộc tính address
setAdd(String)	Thiết lập giá trị mới cho thuộc tính address
display()	Hiển thị giá trị của thuộc tính

Câu 2: Tạo lớp Student thừa kế từ lớp Person, có thêm các thuộc tính Math, Physics, Chemistry, Average và các phương thức trong bảng sau:

Phương thức	Mô tả
Student()	Phương thức khởi tạo mặc định
Student(String name, int age, String add, float math, float physics, float chemistry)	Phương thức khởi tạo các thuộc tính của lớp
getAverage()	Trả về điểm trung bình $(=(\text{math} + \text{physics} + \text{chemistry})/3)$
getMath()	Trả về giá trị thuộc tính math
display()	Hiển thị name, add, age và average

Câu 3: Viết lớp chứa hàm main cho phép tạo danh sách sinh viên gồm N (nhập từ bàn phím) đối tượng Student và thực hiện các công việc sau:

- Nhập giá trị cho danh sách sinh viên;
- Sắp xếp danh sách sinh viên theo chiều tăng dần của điểm trung bình (average);
- Tìm đối tượng sinh viên của điểm toán (math) lớn nhất;
- Hiển thị tất cả sinh viên có tuổi (age) lớn hơn 23;
- Tìm tất cả các sinh viên có họ “Le” (gợi ý: sử dụng hàm startsWith() của lớp String);
- Hiển thị danh sách sinh viên có địa chỉ ở “Đồng Tháp”.

Câu 4: Viết chương trình quản lý Sinh viên gồm các loại sau:

- Ban cán sự: họ tên, ngày sinh, lớp, chức vụ;
- Sinh viên gồm các thuộc tính: họ tên, ngày sinh, lớp;

Chương trình cho phép thực hiện các công việc sau:

- + Nhập dữ liệu gồm 01 Lớp trưởng, 02 lớp phó và N ($N > 5$) sinh viên
- + Hiển thị thông tin sinh viên

+ Hiển thị sinh viên có sinh nhật trong tháng 05

+ Hiển thị tất cả sinh viên thuộc lớp CNTT

+ Tìm tên sinh viên có chữ “Văn”

Câu 5: Viết chương trình quản lý nhân sự của một công ty gồm các loại nhân sự sau:

a. Giám đốc bao gồm các thuộc tính: họ tên, ngày sinh, hệ số lương, hệ số chức vụ;

b. Quản đốc gồm các thuộc tính: họ tên, ngày sinh, hệ số lương, số lương nhân viên quản lý;

c. Nhân viên gồm các thuộc tính: họ tên, ngày sinh, hệ số lương, tên đơn vị (phòng/ban);

Chương trình cho phép thực hiện các công việc sau:

+ Nhập dữ liệu gồm 01 Giám đốc, 02 quản đốc và N ($N > 5$) nhân viên

+ Hiển thị thông tin các nhân sự

+ Hiển thị nhân sự có tổng lương cao nhất (Tổng lương = (hệ số lương + hệ số chức vụ (nếu có))*115000

+ Hiển thị nhân sự có sinh nhật trong tháng 12

+ Hiển thị tất cả nhân viên thuộc phòng kế toán

+ Tìm nhân viên có họ “Nguyễn”

CHƯƠNG 4. LẬP TRÌNH GIAO DIỆN

Mục tiêu của chương

Sau khi học xong chương này, người học sẽ:

- Thao tác thành thạo công cụ Swing để thiết kế giao diện;
- Vận dụng kiến thức lập trình Java cơ bản và thiết kế giao diện cho bài toán đã phân tích;
- Lựa chọn hiệu quả việc các thành phần trong công cụ Swing để thiết kế giao diện người dùng một cách thân thiện.

4.1. TỔNG QUAN VỀ JAVA SWING

Swing là một phần của các lớp Java Foundation (JFC), các phần khác của JFC là Java2D và bộ công cụ Abstract windowkit (AWT). AWT, Swing & Java 2D được sử dụng để xây dựng giao diện người dùng đồ họa (GUI) trong Java.

4.1.1. JFC là gì

JFC (Java Foundation Classes) là tập hợp các tính năng để xây dựng giao diện người dùng đồ họa – GUI, để tạo ra các chức năng đồ họa phong phú và tương tác với các ứng dụng Java. JFC là một dự án phối hợp giữa Netscape's Internet Foundation Classes(IFC) và IBM's Taligent division and Lighthouse Design.

JFC bao gồm 5 thư viện: (1) Swing GUI Components; (2) AWT (Abstract windowkit); (3) Plugable Look and Feel Support; (4) Accessibility Application Programming Interface; (5) Java 2D API.

4.1.1.1. Look and feel Manager

Thư viện Look and Feel Support là bộ phận quản lý giao diện nhằm làm thay đổi giao diện của chương trình Java. Chẳng hạn, có thể làm cho ứng dụng chạy trên PC với hệ điều hành Windows trông giống như chương trình chạy trên máy với hệ điều hành Macintosh (Mac OS), hoặc của Motif trong UNIX, v.v...

4.1.1.2. Accessibility Application Programming Interface

Accessibility Application Programming Interface là thành phần hỗ trợ người khuyết tật, bằng cách làm cho ứng dụng tương tác với các phương tiện khác như bộ đọc chữ Braille, bộ đọc chữ từ màn hình (Screen Reader).

4.1.1.3. Java 2D API

Java cung cấp một hệ thống thư viện giúp cho việc phát triển các ứng dụng đồ họa 2 chiều, qua đó có thể phát triển giao diện đồ họa người dùng, game, hiệu ứng chuyển động, các ứng dụng đa phương tiện và nhiều lĩnh vực khác nhau.

4.1.1.4. AWT (Abstract Windows Toolkit)

AWT là một bộ các lớp trong Java cho phép tạo GUI và chấp nhận các nhập liệu của người dùng thông qua bàn phím và chuột. AWT cung cấp các thành phần khác nhau để tạo GUI hiệu quả và lôi cuốn người sử dụng. Các thành phần này có thể là: Vật chứa (Container); Thành phần (Component); Trình quản lý cách trình bày (Layout manager); Đồ họa (Graphic) và các tính năng vẽ (draw); Phong chữ (Font); Sự kiện (Event).

4.1.1.5. SWING

SWING không phải là một từ viết tắt, Swing là một sản phẩm của gia đình lớn Java, một phần của dự án JFC. Swing bắt đầu được phát triển từ bản beta của JDK 1.1, khoảng mùa xuân năm 1997, đến tháng 3 năm 1998 thì bắt đầu được công bố rộng rãi. Khi phát hành thư viện Swing 1.0 chứa 250 class và 80 interface, đến nay phiên bản Swing 1.8 chứa 18 packages của 737 classes.

4.1.2. Java Swing

Java Swing là một phần của Java Foundation Classes (JFC) được sử dụng để tạo các ứng dụng Window-Based. Nó được xây dựng ở trên cùng của AWT (Abstract Windowing Toolkit) API và được viết hoàn toàn bằng Java.

Mỗi thành phần trong Swing được gọi là component. Component được chia làm 2 loại:

(1) Loại khung chứa: là những component định nghĩa khung chứa các component khác bên trong. Các component loại này ko thực hiện chức năng hiển thị nội dung, mà chỉ định nghĩa kích thước, nền, cách sắp xếp và hiển thị các

component bên trong. Các component khung chứa thường dùng như JFrame, JPanel, JDialog, ...

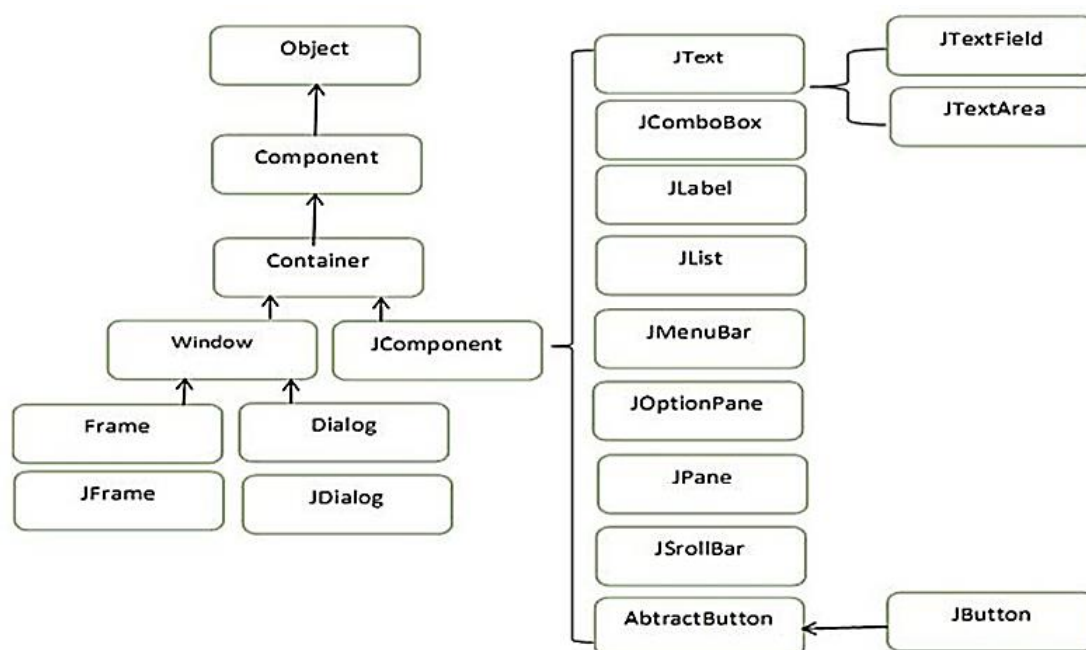
(2) Loại hiển thị: là những component đơn vị thực hiện chức năng hiển thị nội dung. Các component hiển thị thường dùng như JLabel, JButton, JList, JTextField,...

❖ **Sự khác nhau giữa AWT và SWING** (mô tả trong bảng 4.1)

Bảng 4.1. So sánh sự khác nhau giữa AWT và SWING

AWT	SWING
Xây dựng bằng native code	Xây dựng hoàn toàn bằng JAVA API
Khó phát triển thêm các linh kiện (widget) mới	Dễ phát triển các linh kiện
Cung cấp ít thành phần hơn SWING	Swing cung cấp các thành phần mạnh mẽ hơn như table, list, scrollpanes, colorchooser, tabbedpane ...

❖ **Kiến trúc SWING** (minh họa hình 4.1)



Hình 4.1. Sơ đồ cấu trúc thứ bậc các lớp trong Java Swing

❖ **Container trong Java Swing**

Container là thành phần chủ chốt trong các thành phần của SWING GUI. Một container cung cấp một không gian, là nơi đặt một thành phần. Một container

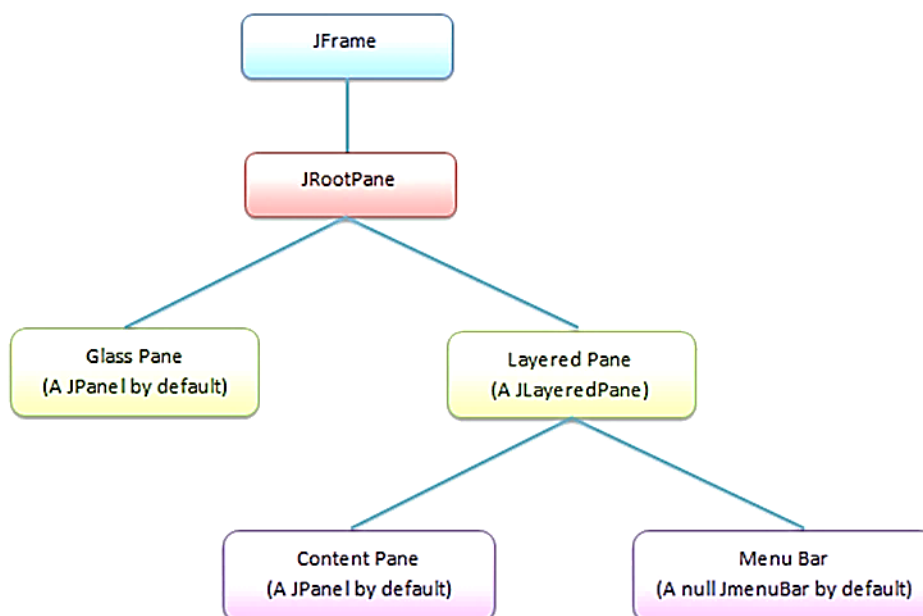
trong AWT chính là một component và nó có thêm khả năng để thêm các thành phần khác vào chính nó.

Có 2 kiểu container trong Swing: top-level container và multi-purpose container. Swing cung cấp 3 loại top-level container là: JFrame, JDialog, JApplet (được sử dụng cho ứng dụng web), JWindow.

4.1.3. JFrame

4.1.3.1. Khái niệm

JFrame là một top-level container thường được sử dụng để tạo giao diện ứng dụng người dùng được minh họa như hình 4.2. Tất cả các đối tượng liên quan tới JFrame được quản lý bởi thể hiện (instance) của JRootPane. JRootPane có 4 phần chính là GlassPane, LayeredPane, ContentPane và MenuBar



Hình 4.2. Sơ đồ hoạt động của JFrame

Các thành phần (component) sẽ không được thêm trực tiếp vào JFrame, thay vào đó, phần lớn các thành phần sẽ được thêm vào Content Pane bằng cách gọi phương thức:

`getContentPane().add(component);`

- Có hai cách tạo JFrame như sau:

+ **Cách 1:** Thừa kế javax.swing.JFrame

```
import javax.swing.JFrame;  
public class Demo_Frame extends JFrame{
```

```

public Demo_Frame() {
    ...
}
public static void main (String [] args){
    Demo_Frame demo = new Demo_Frame();
}
}

```

Điều kiện sử dụng cách tạo JFrame kiểu này là khi muốn tạo một JFrame với một lớp không được kế thừa từ bất cứ lớp nào khác. Sự thuận lợi của cách này là có thể gọi các phương thức của lớp Cha một cách trực tiếp.

+ **Cách 2:** Khai báo javax.swing.JFrame

```

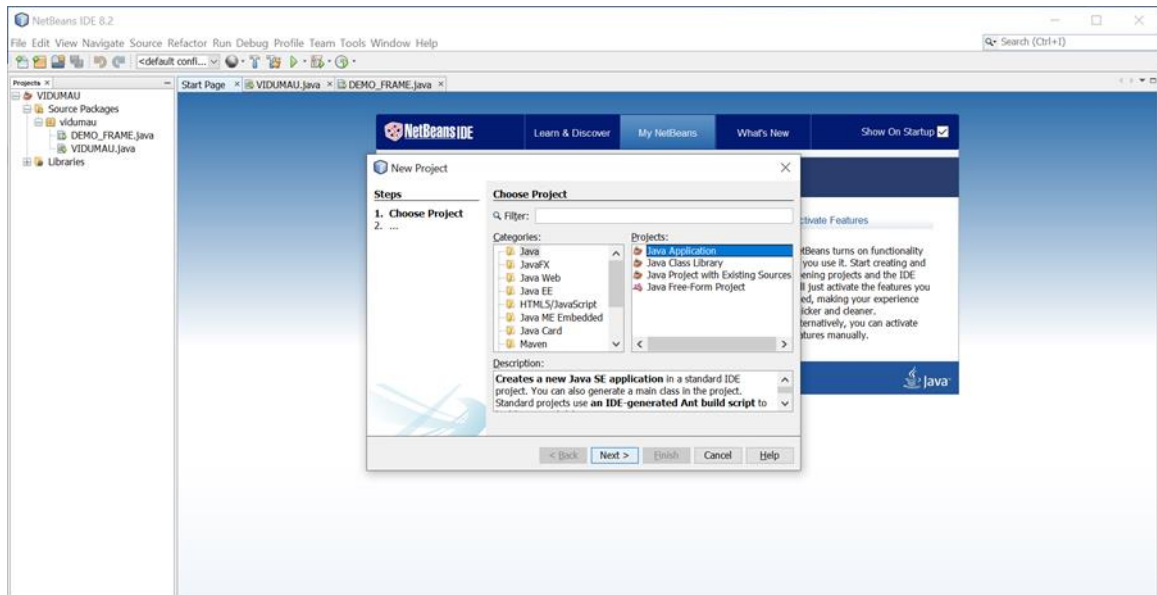
import javax.swing.JFrame;
public class Demo_Frame extends ClassABC{
    public Demo_Frame() {
        public static void main (String [] args){
            Demo_Frame demo = new Demo_Frame();
            JFrame demo = new JFrame();
        }
    }
}

```

JFrame sau khi được tạo thì sẽ không được nhìn thấy. Để nhìn được JFrame, phải sử dụng phương thức: *setVisible(boolean value)*

Tạo Project Java Swing trong NetBeans

Bước 1. Mở ứng dụng NetBeans IDE, New Project → Java → Java Application, nhấp Next, trên hộp thoại “New Java Application” điền thông tin trong hộp thoại. Giao diện Project Java Swing trong NetBeans minh họa như hình 4.3.



Hình 4.3. Giao diện khởi tạo project mới trong NetBeans

Sau khi tạo thành công, người dùng cứ chọn Next cho đến màn hình Name & Location và tiến hành đặt tên:

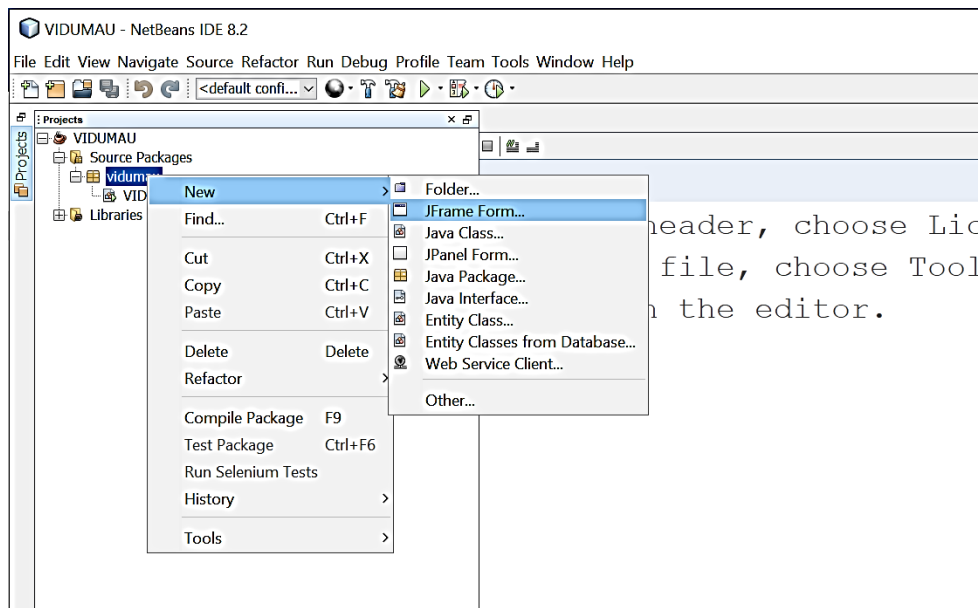
- + Project Name: Tên của dự án
- + Project location: Nơi lưu trữ dự án

→ Đây là hai thông tin cần nhập để đặt tên cũng như vị trí lưu trữ dự án của người lập trình.

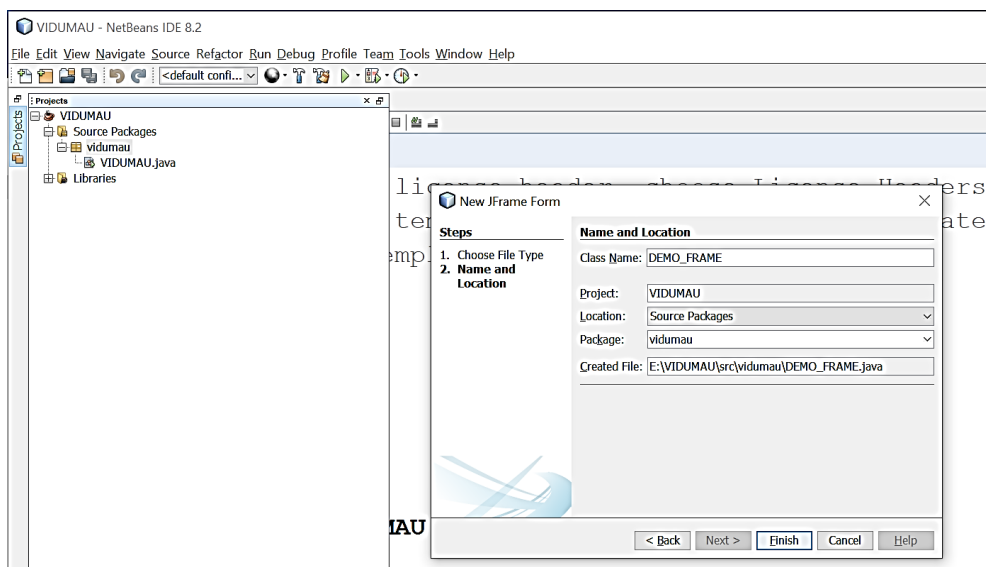
Lưu ý: có thể bỏ chọn Create Main Class vì chúng ta sẽ sử dụng JFrame để chạy chương trình.

Bước 2. Tạo JFrame

Sau khi khởi tạo project thành công, chọn New → JFrame và tiến hành đặt tên cho bài làm. Thao tác với JFrame minh họa như hình 4.4, hình 4.5.

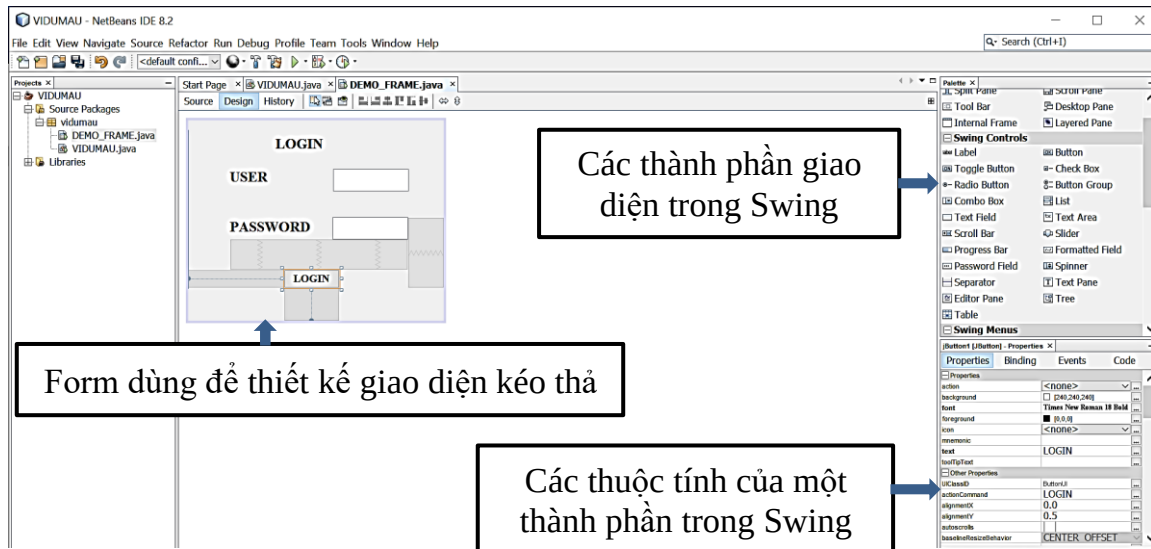


Hình 4.4. Giao diện tạo mới một JFrame



Hình 4.5. Giao diện đặt tên và nơi lưu trữ của JFrame

Bước 3. Giao diện khi tạo xong một JFrame. Thực hiện như minh họa hình 4.6.



Hình 4.6. Giao diện chính khi thiết kế JFrame trên NetBeans

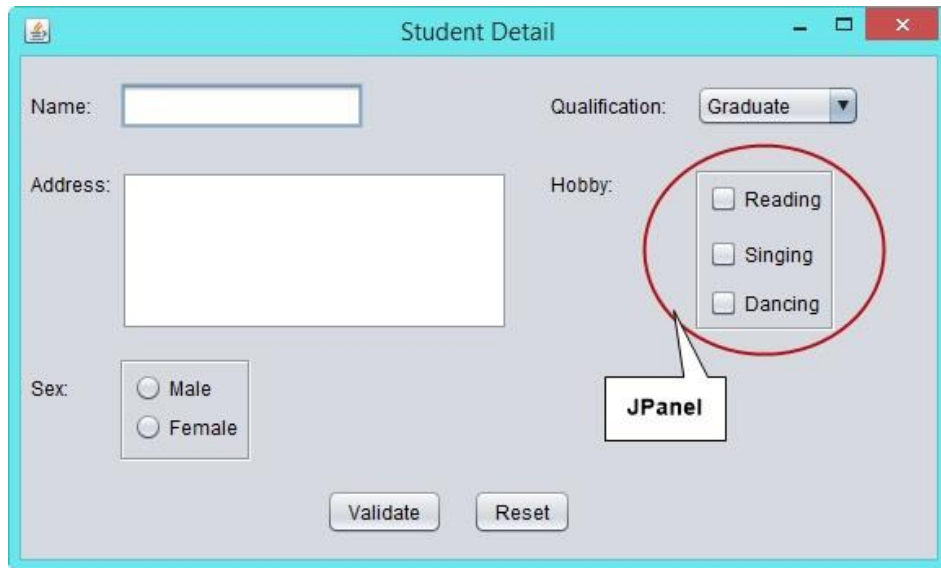
4.1.3.2. Một số thuộc tính thường dùng

Thuộc tính	Miêu tả
title	Tiêu đề màn hình
defaultCloseOperation	Thiết lập xử lý khi người dùng chọn nút dấu X ở góc trên bên phải
iconImage	Thiết lập icon ở góc trên bên trái của màn hình
resizable	Cho phép điều chỉnh kích thước màn hình hay không
size	Kích thước màn hình

4.1.4. JPanel

4.1.4.1. Khái niệm

- JPanel là một container dùng để chứa các thành phần đồ họa khác (tương tự như JFrame tuy nhiên nó không phải là 1 JFrame).
- Trong một JFrame chứa các JPanel, trong mỗi JPanel lại có thể chứa các đối tượng hoặc thậm chí là các JPanel khác. Minh họa JPanel như hình 4.7.



Hình 4.7. Giao diện sử dụng JPanel trong JFrame

- Phương thức khởi tạo JPanel:

+ JPanel(): Tạo 1 JPanel với Layout mặc định là FlowLayout (cách bố trí các đối tượng nối tiếp nhau).

JPanel panel1 = new JPanel();

+ JPanel(LayoutManager layout): Tạo 1 JPanel với Layout được chỉ định.

JPanel panel2 = new JPanel (new GridLayout());

4.1.4.2. Một số hàm thông dụng

- Đặt layout cho JPanel: *setLayout("layout")*.

- Thiết lập vị trí và kích thước cho JPanel: *setBound(x,y,width,height)*.

- Đặt màu nền cho JPanel: *setBackground(Color."color")*.

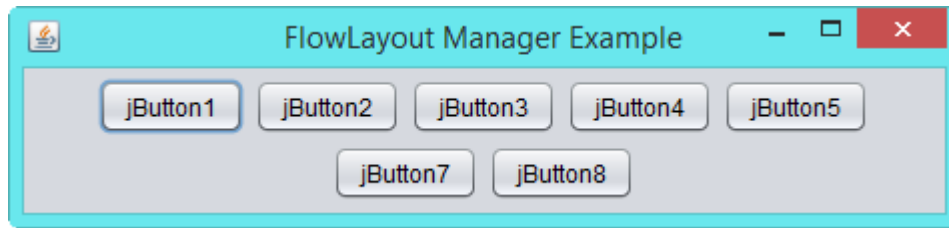
- Đặt đường viền cho JPanel: *setBorder(new "kiểu-border")*.

4.1.4.3. Một số layout thường gặp

Khung chứa container nhận các đối tượng từ bên ngoài đưa vào và nó biết phải làm thế nào để tổ chức sắp xếp “chỗ ở” cho các đối tượng đó. Mỗi đối tượng khung chứa đều có một bộ quản lý trách nhiệm thực hiện công việc đấy, đó là bộ quản lý trình bày (Layout Manager). Các bộ quản lý trình bày mà thư viện AWT cung cấp bao gồm:

a. FlowLayout

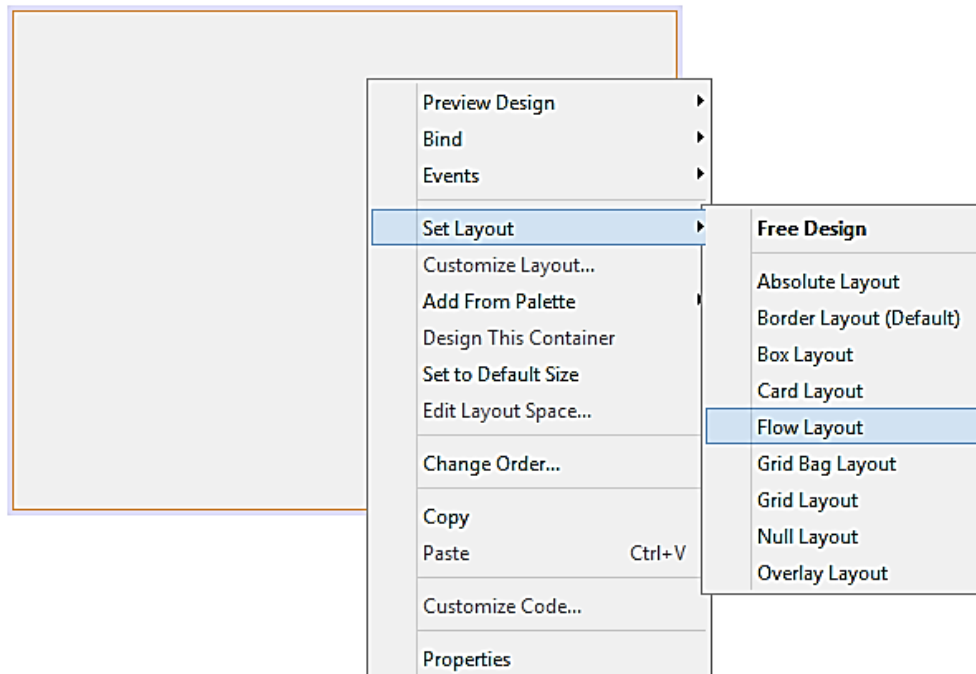
Sắp xếp các đối tượng từ trái qua phải và từ trên xuống dưới. Các đối tượng đều giữ nguyên kích thước của mình. Minh họa giao diện FlowLayout hình 4.8.



Hình 4.8. Hình minh họa giao diện FlowLayout

✚ Sử dụng Flow Layout trong NetBeans

Sử dụng chế độ thiết kế (Design), chuột phải vào container (ví dụ JFrame)
 → chọn Set Layout → chọn Flow Layout (Lưu ý sau khi chọn bố cục là Flow Layout, bộ quản lý bố cục sẽ bố trí các thành phần theo thứ tự từ trái qua phải như diễn giải ở trên). Thao tác như hình 4.9.



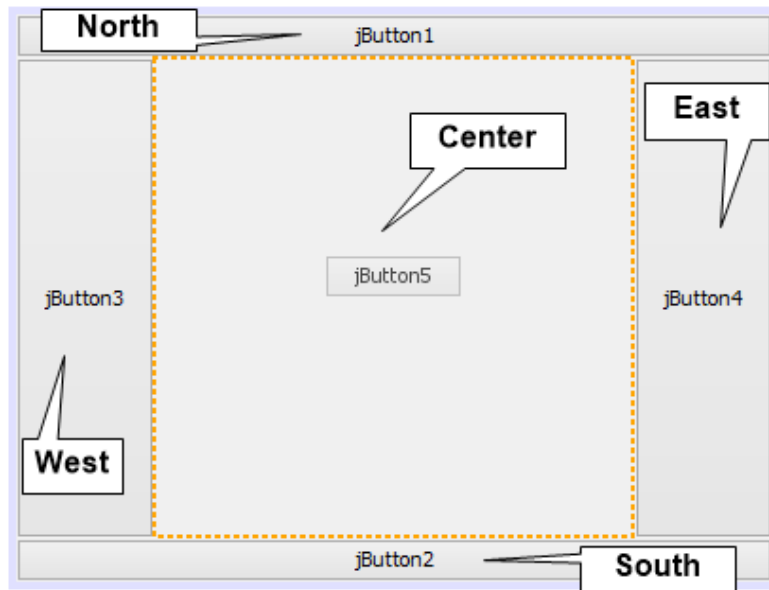
Hình 4.9. Thiết kế Flow Layout trong NetBeans

✚ Ưu và nhược điểm của Flow Layout Manager

Ưu điểm của Flow Layout là sử dụng đơn giản, tuy nhiên nếu container bị thay đổi kích cỡ một số thành phần có thể chuyển lên trên hoặc chuyển xuống dưới tùy thuộc vào chiều ngang.

b. BorderLayout

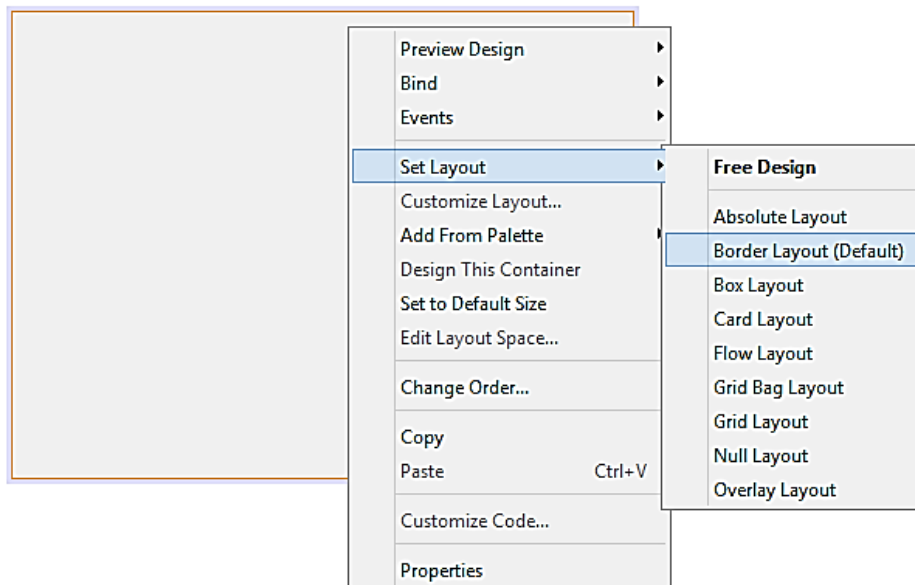
Các đối tượng được đặt theo các đường viền của khung chứa theo các cạnh "North", "South", "East", "West", "Center". Giao diện BorderLayout mô tả như hình 4.10.



Hình 4.10. Hình minh họa giao diện BorderLayout

Sử dụng Border Layout trong NetBeans

Tương tự như cách sử dụng Flow Layout, ngoại trừ chọn Border Layout. Thao tác Border Layout như hình 4.11.



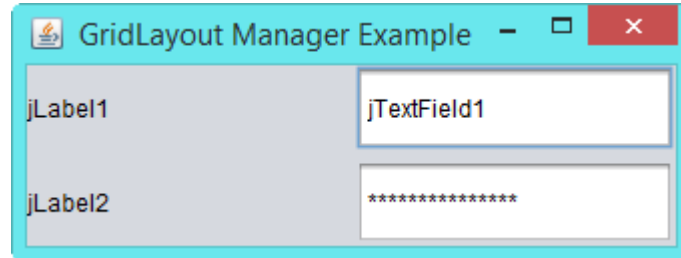
Hình 4.11. Thiết kế Border Layout trong NetBeans

Ưu và nhược điểm của Border Layout Manager

Border Layout cho phép chỉ định trực tiếp nơi mà một thành phần được đặt. Tuy nhiên khi sử dụng Border Layout, chúng ta chỉ có tối đa 5 vị trí trong một container.

c. Grid Layout

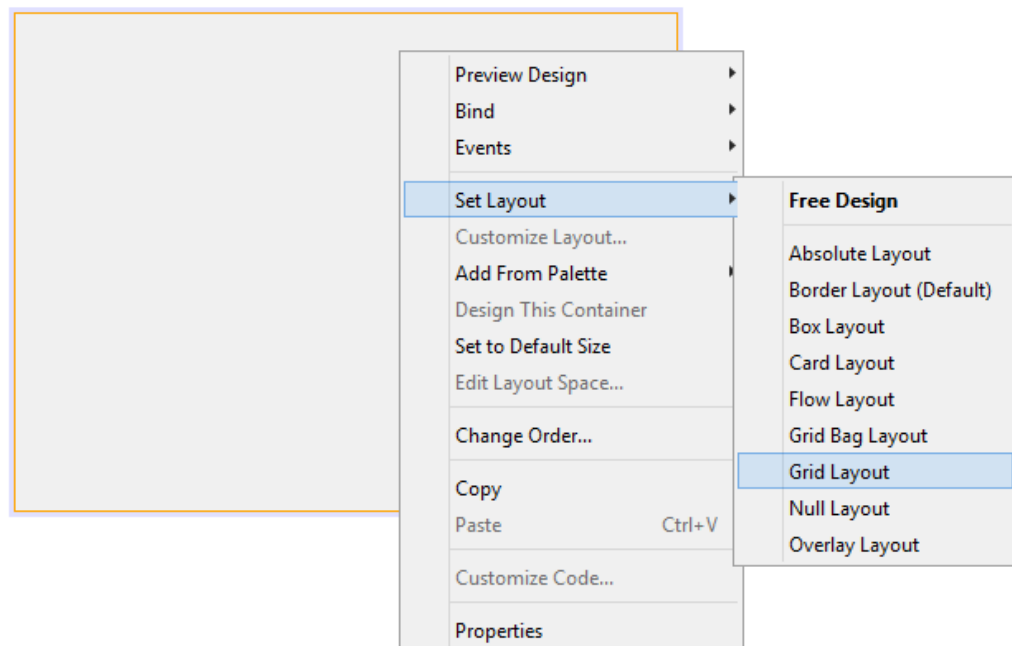
Grid Layout tạo một khung lưới vô hình với các ô bằng nhau. Các đối tượng sẽ đặt vừa kích thước với từng ô đó. Thứ tự sắp xếp từ trái qua phải và từ trên xuống. Giao diện Grid Layout có dạng như hình 4.12.



Hình 4.12. Hình minh họa giao diện GridLayout

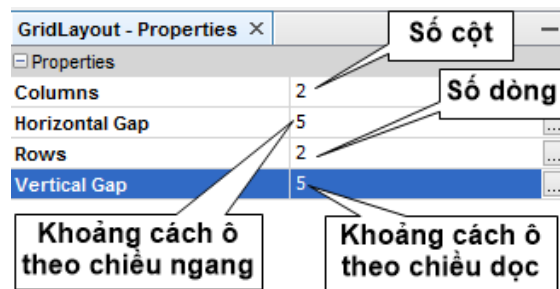
🔧 Sử dụng Grid Layout trong NetBeans

Chuột phải vào container → chọn Set Layout → chọn Grid Layout. Thao tác Grid Layout như hình 4.13.



Hình 4.13. Thiết kế Grid Layout trong NetBeans

- Chỉ định số dòng và số cột khi thiết kế được mô tả chi tiết như hình 4.14.



Hình 4.14. Thiết lập các giá trị trong Grid Layout

d. Box Layout

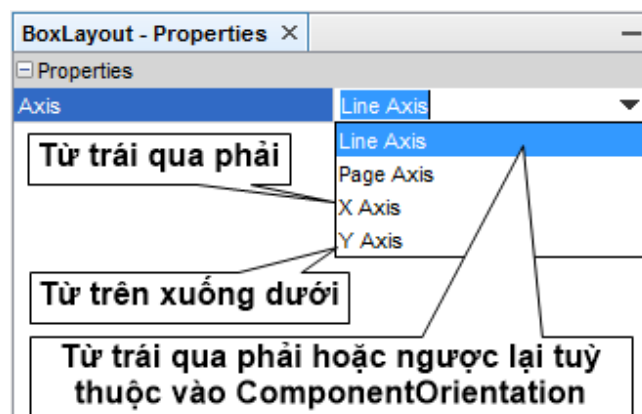
Box Layout tương tự như Flow Layout ngoại trừ bố trí các thành phần theo cột (từ trên xuống dưới hoặc từ dưới lên trên) hoặc theo dòng (từ trái qua phải hoặc từ phải qua trái). Giao diện Box Layout như hình 4.15.



Hình 4.15. Hình minh họa giao diện Box Layout

🔧 Sử dụng Box Layout trong NetBeans

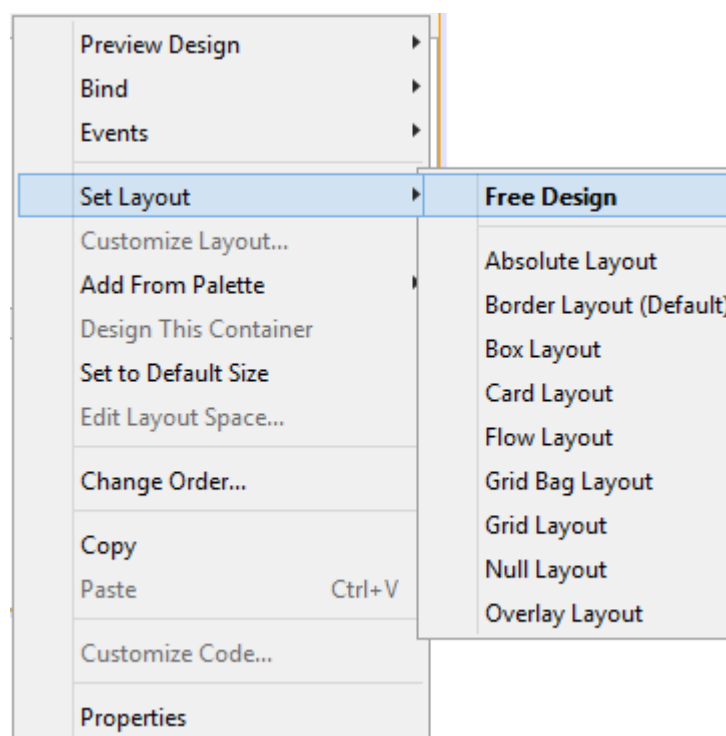
Chuột phải vào container → chọn Set Layout → chọn Box Layout.
Đồng thời lựa chọn cách bố trí được hướng dẫn chi tiết như hình 4.16.



Hình 4.16. Thiết lập cách bố trí trong Box Layout

e. Null Layout

Đây là cách trình bày tự do. Đối với cách trình bày này người lập trình phải tự động làm tất cả từ việc định kích thước của các đối tượng, cũng như xác định vị trí của nó trên màn hình. Ta không phụ thuộc vào những ràng buộc "North", "South", "East", "West", "Center". Thao tác thực hiện như hình 4.17.



Hình 4.17. Thiết kế Null Layout trong NetBeans

4.2. CÁC THÀNH PHẦN GUI CƠ BẢN

4.2.1. Swing Control

- Mỗi Swing control kế thừa các thuộc tính từ cấu trúc thứ bậc của lớp component.

- Lớp JComponent là lớp cơ sở trừu tượng của tất cả các Swing component (có tên bắt đầu chữ J - ngoại trừ top-container). Lớp JComponent kế thừa từ component và container.

- Lớp JComponent đóng gói hầu hết các đặc điểm và thao tác cơ bản của 1 thành phần GUI.

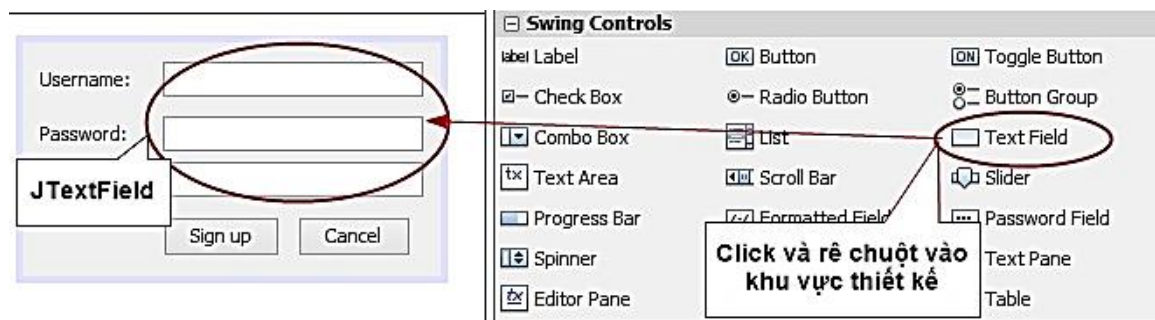
- Một số đặc điểm của lớp JComponent bao gồm: Cảm quan khả kiến (Pluggable); Phím tắt (tính dễ nhớ); Khả năng xử lý sự kiện chung;

- Một số thành phần của JComponent: JButton, JList, JLabel, JTextField, JTextArea; JComboBox, JRadioButton, JCheckBox; JProgressBar, JSlider, JTable, JToolBar, JTree; JInternalFrame, JScrollPane; JSplitPane, JTabbedPane.

4.2.2. *JTextField*

Lớp JTextField là một thành phần cho phép sửa đổi một dòng văn bản đơn. Thông thường được dùng để hiển thị dữ liệu, nhập dữ liệu.

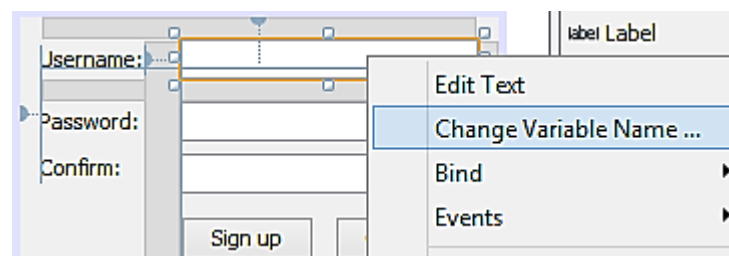
 Tạo JTextField trong NetBeans. Thao tác minh họa hình 4.18.



Hình 4.18. Thiết kế JTextField trong NetBeans

+ Thay đổi nội dung của JTextField: chuột phải vào JTextField → chọn Edit Text

+ Thay đổi tên đối tượng (tên được sử dụng để truy cập các phương thức và thuộc tính của JTextField): chuột phải vào JTextField → chọn Change Variable Name như hình 4.19.



Hình 4.19. Thay đổi tên biến của JTextField

4.2.2.1. Một số hàm tạo trong JTextField

Tên hàm	Ý nghĩa
JTextField()	Xây dựng một TextField mới.
JTextField(String text)	Xây dựng một TextField mới được khởi tạo với text đã cho.

Tên hàm	Ý nghĩa
TextField(int columns)	Xây dựng một TextField mới và trống với số cột đã cho.
TextField(String text, int columns)	Xây dựng một TextField mới được khởi tạo với text và các cột đã cho.
TextField(Document doc, String text, int columns)	Xây dựng một TextField mới mà sử dụng mô hình lưu trữ text đã cho và số cột đã cho.

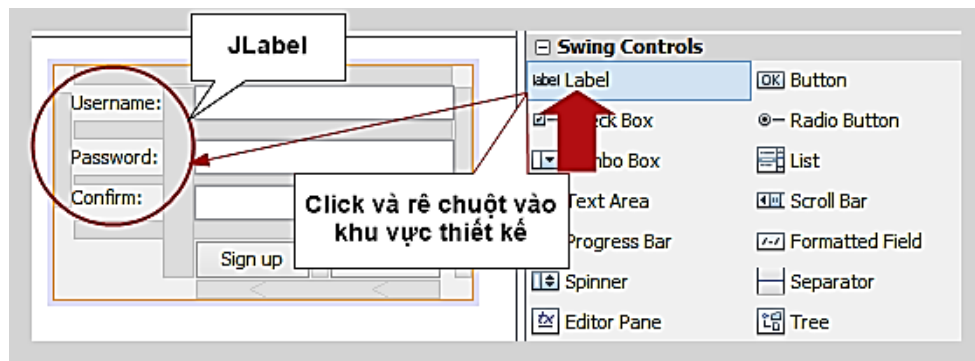
4.2.2.2. Một số phương thức thường dùng

Tên phương thức	Ý nghĩa
void setText(String t)	Thiết lập văn bản được hiển thị bởi TextField
String getText()	Lấy văn bản trong TextField
void setEditable(boolean b) boolean isEditable()	Thiết lập hoặc chỉ ra việc người dùng có thể chỉnh sửa nội dung văn bản trong TextField.
void setColumns(int) int getColumns()	Thiết lập hoặc lấy số cột của TextField
trim()	Bỏ đi khoảng trắng đầu và cuối chuỗi
setFont()	Thiết lập font chữ trong TextField
requestFocus()	Đặt con trỏ ở đầu ô TextField

4.2.3. JLabel

Lớp JLabel có thể hiển thị dạng văn bản, hình ảnh hoặc cả hai. Các nội dung của label được gán bởi việc thiết lập căn chỉnh ngang và dọc trong khu vực hiển thị của nó. Theo mặc định, các label được căn chỉnh theo chiều dọc trong khu vực hiển thị; text-only label là căn chỉnh theo cạnh, image-only label là căn chỉnh theo chiều ngang.

✚ Tạo JLabel trong NetBeans. Thao tác minh họa như hình 4.20.



Hình 4.20. Thiết kế JLabel trong NetBeans

+ Thay đổi nội dung hiển thị: chuột phải vào JLabel → chọn Edit Text

4.2.3.1. Một số hàm tạo trong JLabel

Tên hàm	Ý nghĩa
JLabel()	Tạo một đối tượng của JLabel, không có hình ảnh, và với một chuỗi trống cho tiêu đề.
JLabel(String text)	Tạo một đối tượng của JLabel với văn bản đã cho
JLabel(Icon image)	Tạo một đối tượng của JLabel với hình ảnh đã cho
JLabel(String text, int horizontalAlignment)	Tạo một đối tượng của JLabel với văn bản và căn chỉnh ngang đã cho.
JLabel(String text, Icon icon, int horizontalAlignment)	Tạo một đối tượng của JLabel với văn bản, hình ảnh, và căn chỉnh ngang đã cho.

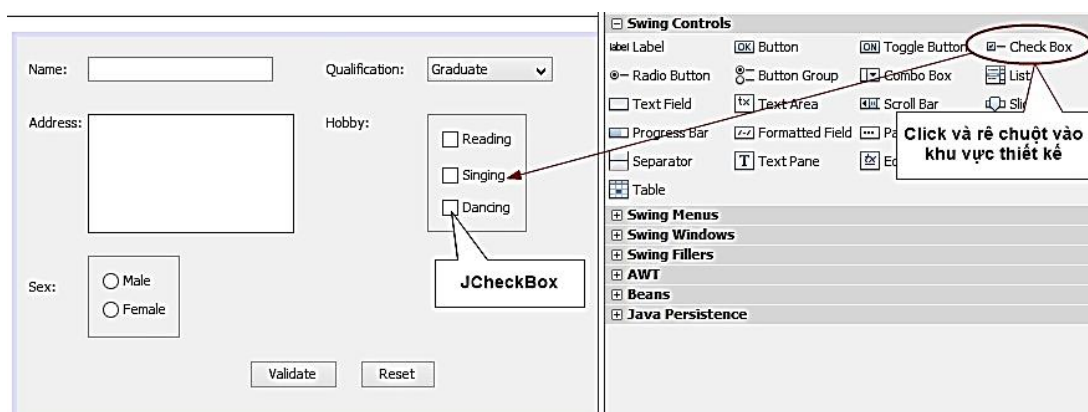
4.2.3.2. Một số phương thức thường dùng

Tên hàm	Ý nghĩa
void setText(String S)	Định nghĩa trường văn bản dòng đơn mà thành phần này sẽ hiển thị
void setIcon(Icon icon)	Định nghĩa hình ảnh mà thành phần này sẽ hiển thị
String getText()	Lấy dữ liệu từ nhãn (label)
int getLength()	Chiều dài của chuỗi văn bản.

4.2.4. JCheckBox

Một thành phần có 2 trạng thái là chọn (checked) và không chọn (unchecked). Trạng thái mặc định của JCheckBox là unchecked. Người dùng có thể chọn cùng lúc nhiều lựa chọn.

 Tạo JCheckBox trong NetBeans. Minh họa hình 4.21.



Hình 4.21. Thiết kế JCheckBox trong NetBeans

4.2.4.1. Một số hàm tạo trong JCheckbox

Tên hàm	Ý nghĩa
JCheckBox()	Tạo một checkbox không được lựa chọn ban đầu không có văn bản và hình ảnh
JCheckBox(String text)	Tạo một checkbox không được lựa chọn ban đầu với văn bản.
JCheckBox(String text, boolean selected)	Tạo một checkbox với văn bản và xác định rằng ban đầu nó là được lựa chọn hoặc không.
JCheckBox(String text, Icon icon)	Tạo một checkbox không được lựa chọn ban đầu với văn bản và icon đã cho.
JCheckBox(Action a)	Tạo một checkbox, các thuộc tính được lấy từ Action đã cho.

4.2.4.2. Một số phương thức thường dùng

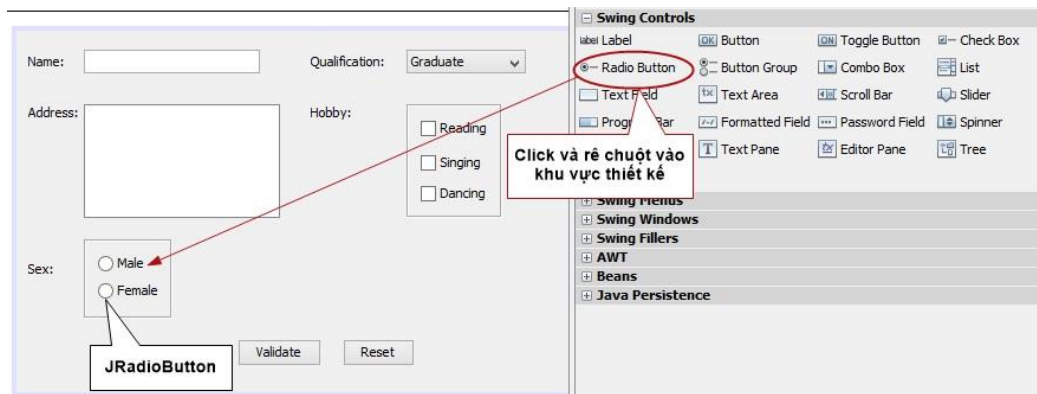
Tên phương thức	Ý nghĩa
boolean isSelected()	Trả về true nếu checkbox đang được chọn và false nếu ngược lại
setSelected(boolean b)	Thiết lập chọn hoặc bỏ chọn trong checkbox

4.2.5. JRadioButton

Lớp JRadioButton là một đối tượng cho phép chọn lựa các thuộc tính giống như JCheckBox. Tuy nhiên, JRadioButton cho phép người dùng chỉ chọn được một trong các thuộc tính. Ví dụ chọn giới tính thì người dùng chỉ được chọn trong danh sách lựa chọn là Nam hoặc Nữ.

Lưu ý: Khi tạo nhiều JRadioButton phải kết hợp với Button Group để ràng buộc thao tác chọn của người dùng (tại một thời điểm chỉ có một JRadioButton được chọn).

 Tạo JRadioButton trong NetBeans. Minh họa hình 4.22.



Hình 4.22. Thiết kế JRadioButton trong NetBeans

4.2.5.1. Một số hàm tạo trong JRadioButton

Tên hàm	Ý nghĩa
JRadioButton()	Tạo một radiobutton không được chọn không có văn bản.
JRadioButton(String s)	Tạo một radiobutton không được chọn với văn bản cho trước
JRadioButton(String s, boolean selected)	Tạo một radiobutton với văn bản đã cho và trạng thái là đã được lựa chọn

4.2.5.2. Một số phương thức thường dùng

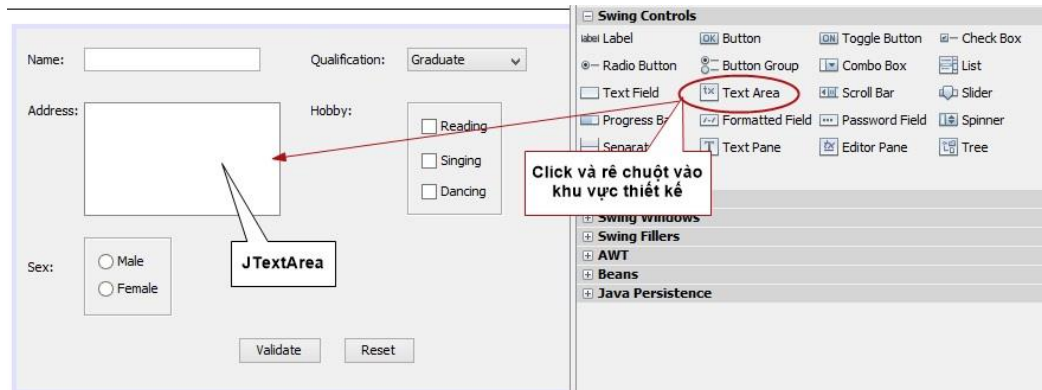
Tên phương thức	Ý nghĩa
boolean isSelected()	Trả về true nếu radiobutton đang được chọn và false nếu ngược lại

Tên phương thức	Ý nghĩa
setSelected(boolean b)	Thiết lập chọn hoặc bỏ chọn trong radiobutton

4.2.6. JTextArea

JTextArea là một thành phần cho phép hiển thị nhiều dòng văn bản đồng thời người dùng có thể chỉnh sửa văn bản.

 Tạo JTextArea trong NetBeans. Minh họa hình 4.23.



Hình 4.23. Thiết kế JTextArea trong NetBeans

4.2.6.1. Một số hàm tạo trong JTextArea

Tên hàm	Ý nghĩa
JTextArea()	Tạo một textarea mới.
JTextArea(String s)	Tạo một textarea mới với văn bản đã cho.
JTextArea(int row, int column)	Tạo một textarea trống mới với số hàng và cột đã cho.
JTextArea(String s, int row, int column)	Tạo một textarea mới với văn bản, số hàng và cột đã cho.

4.2.6.2. Một số phương thức thường dùng

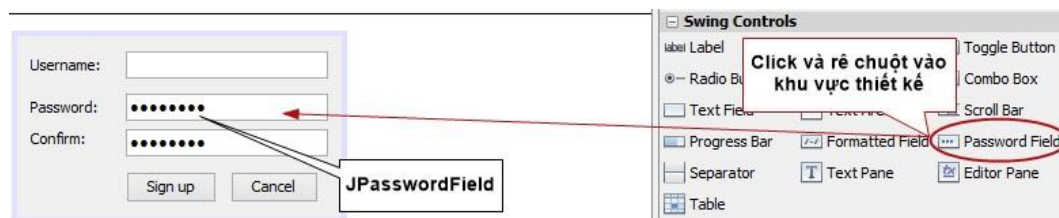
Tên hàm	Ý nghĩa
String text = getText()	Lấy nội dung của textarea
setText(String value);	Thiết lập nội dung của textarea
setEditable(boolean editable)	Thiết lập cho phép chỉnh sửa nội dung hay không. Nếu editable = true, được phép chỉnh sửa. Ngược lại, không được phép chỉnh sửa.

Tên hàm	Ý nghĩa
copy() và cut()	Chuyển vùng văn bản được chọn từ textarea vào clipboard
paste()	Chuyển nội dung từ clipboard vào textarea

4.2.7. JPasswordField

JPasswordField là đối tượng cho phép nhập vào một dòng text giống như JTextField nhưng được ẩn bởi các dấu sao (*) hoặc chấm tròn để tạo nên mật khẩu (password).

 Tạo JPasswordField trong NetBeans. Minh họa như hình 4.24.



Hình 4.24. Thiết kế JPasswordField trong NetBeans

4.2.7.1. Một số hàm tạo trong JPasswordField

Tên hàm	Ý nghĩa
JPasswordField()	Xây dựng một JPasswordField mới, với một tài liệu mặc định, chuỗi văn bản bắt đầu null và độ rộng cột 0.
JPasswordField (int columns)	Xây dựng một JPasswordField trống mới với số lượng cột được chỉ định.
JPasswordField (String text)	Xây dựng một JPasswordField mới được khởi tạo với văn bản được chỉ định.
JPasswordField (String text, int columns)	Xây dựng một JPasswordField mới được khởi tạo với văn bản và cột được chỉ định.

4.2.7.2. Một số phương thức thường dùng

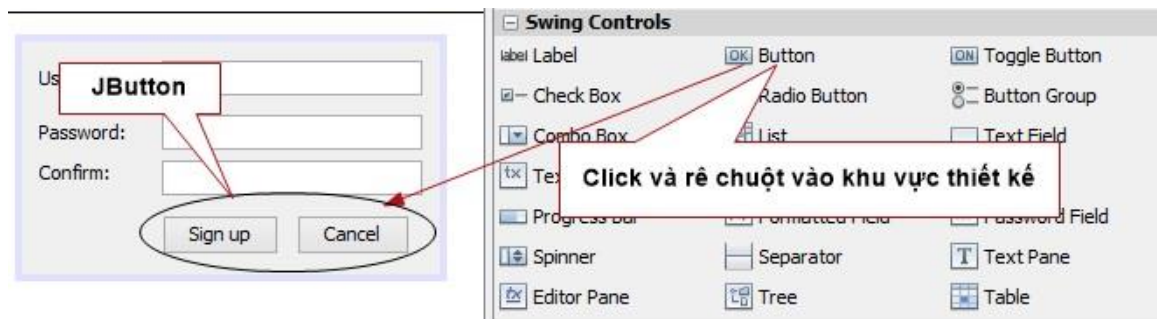
Tên phương thức	Ý nghĩa
String text = getText()	Lấy nội dung của PasswordField
setText(String value)	Thiết lập nội dung của PasswordField

Tên phương thức	Ý nghĩa
setEchoChar()	Thiết lập ký tự hiển thị
char getEchoChar()	Gán ký tự hiển thị
char[] getPassword()	Trả về mật khẩu như mảng các ký tự

4.2.8. JButton

Lớp JButton được sử dụng để tạo một nút button mà có trình triển khai là độc lập nền tảng. Thành phần này có một nhãn (label) và tạo một sự kiện (event) khi được nhấn. Nó cũng có thể có là hình ảnh.

 Tạo JButton trong NetBeans. Minh họa như hình 4.25.



Hình 4.25. Thiết kế JButton trong NetBeans

4.2.8.1. Một số hàm tạo trong JButton

Tên hàm	Ý nghĩa
JButton()	Tạo một button mà không thiết lập văn bản hoặc hình ảnh
JButton(Action a)	Tạo một button tại đây các thuộc tính được nhận từ Action đã cung cấp
JButton(Icon icon)	Tạo một button với một hình ảnh
JButton(String text)	Tạo một button với văn bản
JButton(String text, Icon icon)	Tạo một button với văn bản ban đầu và một hình ảnh

4.2.8.2. Một số phương thức thường dùng

Tên phương thức	Ý nghĩa
public void setText(String s)	Thiết lập văn bản đã cho trên button
public String getText()	Trả về văn bản của button
public void setForeground(Color fg)	Thiết lập màu chữ trên button

Tên phương thức	Ý nghĩa
public void setEnabled(boolean b)	Kích hoạt hoặc vô hiệu hóa button
public void setIcon(Icon b)	Thiết lập Icon đã cho trên button
public Icon getIcon()	Lấy Icon của button

4.2.9. Xử lý sự kiện

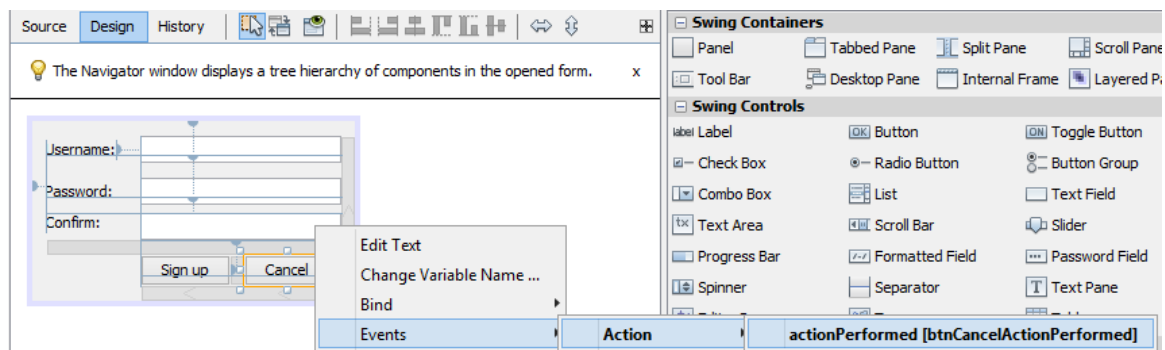
Sự kiện chính là sự phản ứng của chương trình khi có sự tương tác của người dùng. Ví dụ khi người dùng nhấn chuột vào nút nhấn (JButton), chọn hoặc không chọn JCheckBox/JRadioButton,... Và khi một sự kiện được phát sinh thì chương trình phải thực hiện xử lý sự kiện đó. Bảng bên dưới liệt kê những sự kiện thường gặp.

Components	Listeners	Methods
Button, Menu, List	ActionListener	void actionPerformed(ActionEvent ae)
Scrollbar	AdjustmentListener	void adjustmentValueChanged(AdjustmentEvent ae)
Check box, List	ItemListener	void itemStateChanged(ItemEvent ie)
Mouse	MouseListener MouseMotionListener	void mouseClicked(MouseEvent me) void mouseEntered(MouseEvent me) void mouseExited(MouseEvent me) void mousePressed(MouseEvent me) void mouseReleased(MouseEvent me) void mouseDragged(MouseEvent me) void mouseMoved(MouseEvent me)
Window	WindowListener	void windowActivated(WindowEvent we) void windowClosed(WindowEvent we) void windowClosing(WindowEvent we) void windowDeactivated(WindowEvent we) void windowDeiconified(WindowEvent we)

Components	Listeners	Methods
		void windowIconified(WindowEvent we) void windowOpened(WindowEvent we)

🛠 Đăng ký và xử lý sự kiện trong NetBeans như hình 4.26

Chọn chế độ Design → chuột phải lên thành phần muốn xử lý sự kiện → chọn Events → lựa chọn loại sự kiện muốn xử lý. Bên dưới là ví dụ về đăng ký sự kiện cho nút nhấn Cancel (Sự kiện click).



Hình 4.26. Xử lý nút Cancel trong NetBeans

Và kết quả nhận được như hình 4.27



Hình 4.27. Giao diện xử lý sự kiện cho các nút

4.2.10. Dialog

- Một cửa sổ hộp thoại (Dialog) là một cửa sổ con độc lập với ý nghĩa mang theo thông báo tạm thời ngoài cửa sổ chính của Swing Application. Phần lớn hộp thoại được dùng để hiển thị thông báo lỗi hoặc cảnh báo người dùng, nhưng hộp thoại cũng có thể hiển thị ảnh, cây thư mục, hoặc bất kỳ điều gì tương thích với ứng dụng Swing đang quản lý.

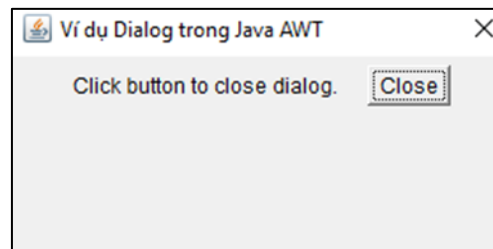
- Để thuận tiện thì một số lớp thành phần của Swing có thể trực tiếp khởi tạo và hiển thị hộp thoại. Để tạo các hộp thoại chuẩn một cách đơn giản thì người dùng có thể sử dụng lớp JOptionPane. Lớp ProgressMonitor có thể lấy một hộp thoại để hiển thị tiến trình của một hoạt động. Hai lớp khác là JColorChooser và JFileChooser cũng cho phép hiển thị các hộp thoại chuẩn. Để tạo hộp thoại in ấn

thì sử dụng API Printing. Để tạo một hộp thoại tùy chỉnh thì người dùng có thể sử dụng lớp trực tiếp là JDialog.

- Có 3 dạng hộp thoại: (1) Hộp thoại thông điệp (Message Dialog); (2) Hộp thoại xác nhận (Confirm Dialog); (3) Hộp thoại tập tin (JFileChooser)

❖ JDialog

- JDialog có 2 trạng thái: (1) Modal: Khi JDialog thực hiện xong mới được phép thao tác lên form cha; (2) Modeless: Sau khi hiển thị dialog như hình 4.28, người dùng có thể thao tác lên form cha. JDialog thường được sử dụng với trạng thái Modal.



Hình 4.28. Hình minh họa giao diện JDialog

4.2.11. JOptionPane

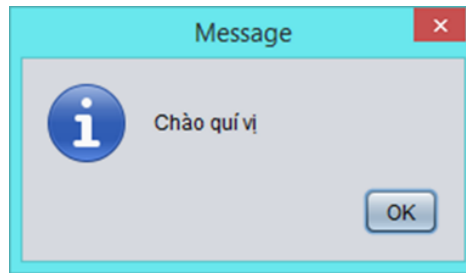
Lớp JOptionPane được sử dụng để cung cấp các hộp thoại tiêu chuẩn như hộp thoại thông báo, hộp thoại xác nhận và hộp thoại tập tin. Các hộp thoại này được sử dụng để hiển thị thông tin hoặc nhận đầu vào từ người dùng. Lớp JOptionPane kế thừa lớp JComponent.

Tên	Ý nghĩa
MessageDialog	Hiển thị hộp thoại dạng thông báo.
ConfirmDialog	Hiển thị hộp thoại yêu cầu người dùng xác nhận (Yes/No).
InputDialog	Hiển thị hộp thoại cho phép người dùng nhập thông tin.
OptionDialog	Cho phép lập trình viên chỉnh sửa hộp thoại theo ý của mình.

🌈 Ví dụ sử dụng JOptionPane

- Hiển thị hộp thoại dạng thông báo như hình 4.29.

```
JOptionPane.showMessageDialog(this, "Chào quý vị");
```



Hình 4.29. Giao diện hộp thoại dạng thông báo

- Hiện thị hộp thoại yêu cầu người dùng xác nhận như hình 4.30

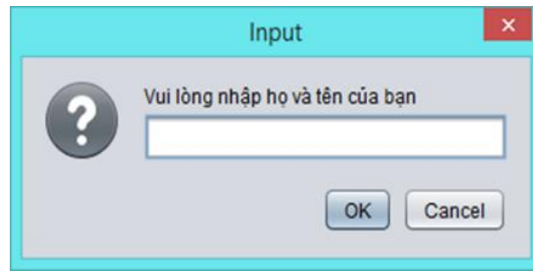
```
int choice=JOptionPane.showConfirmDialog(this,
                                     "This is a confirm dialog");
If(JOptionPane.YES_OPTION==choice) {
    //do something
}
else if(JOptionPane.NO_OPTION=choice){
    //do something
}
else{
}
```



Hình 4.30. Giao diện hộp thoại dạng xác nhận

- Hiện thị hộp thoại cho phép người dùng nhập thông tin như hình 4.31

```
String yourName =
    JOptionPane.showInputDialog("Your name is");
if(yourName != null) {
    JOptionPane.showMessageDialog(this,
        "Your name is " + yourName);
}
```



Hình 4.31. Giao diện hộp thoại cho phép người dùng nhập vào thông tin

4.2.11.1. Một số hàm tạo trong JOptionPane

Tên hàm	Ý nghĩa
JOptionPane()	Tạo một JOptionPane với một thông điệp kiểm tra.
JOptionPane(Object message)	Tạo một thể hiện của JOptionPane để hiển thị một thông báo.
JOptionPane(Object message, int messageType)	Tạo một thể hiện của JOptionPane để hiển thị một thông báo với loại thông báo được chỉ định và các tùy chọn mặc định.

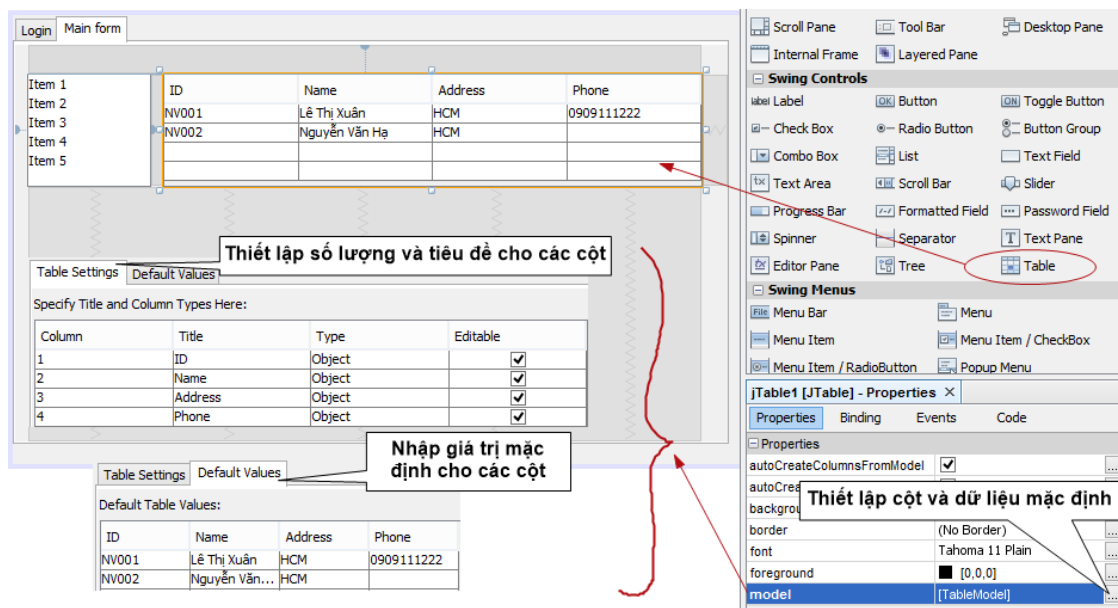
4.2.11.2. Một số phương thức thường dùng

Tên phương thức	Ý nghĩa
void selectInitialValue()	Yêu cầu giá trị khởi tạo ban đầu để được lựa chọn, thiết lập focus tới giá trị đó
void setInputValue(Object newValue)	Thiết lập giá trị input được lựa chọn hoặc được nhập bởi người dùng
void setMessageType(int newType)	Thiết lập kiểu thông điệp của hộp thoại

4.2.12. JTable

JTable là một trong những thành phần quan trọng của Java Swing package được sử dụng để hiển thị và chỉnh sửa dữ liệu theo dạng Table (Dữ liệu được trình bày theo dạng hàng và cột).

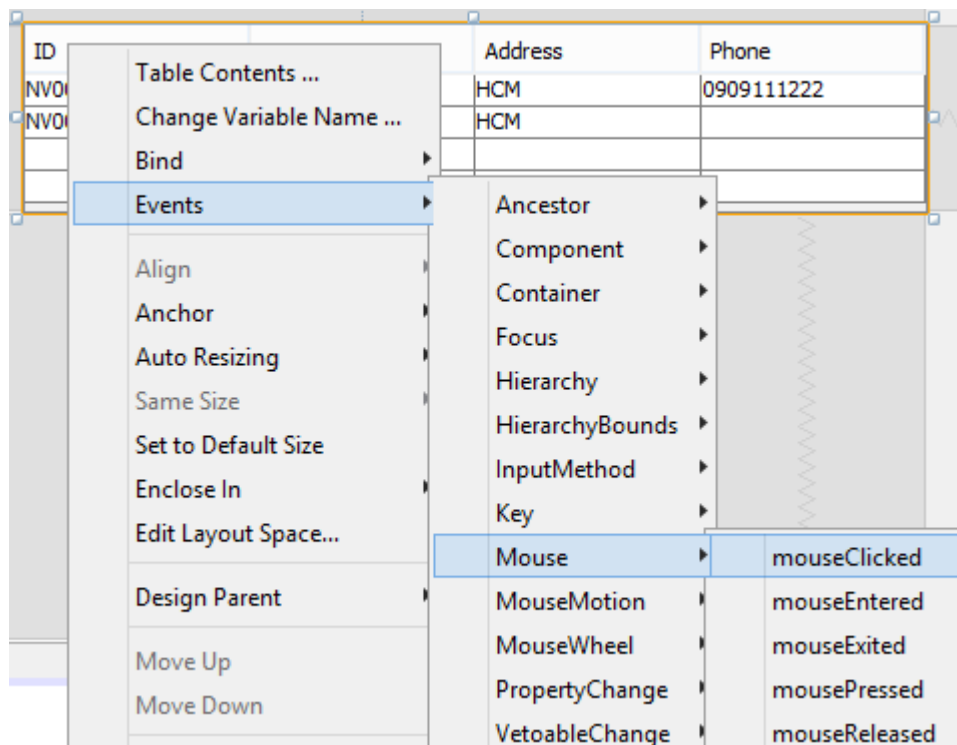
✚ Tạo jTable trong NetBeans thao tác như hình 4.32



Hình 4.32. Thiết kế jTable trong NetBeans

✚ Đăng ký sự kiện cho jTable

Thao tác click Chuột phải vào jTable → chọn Events → chọn Mouse → chọn mouseClicked, minh họa các bước như hình 4.33



Hình 4.33. Xử lý sự kiện cho jTable

✚ Lấy dữ liệu của ô được chọn trong Jtable

```
int col = jTable2.getSelectedColumn();  
int row = jTable2.getSelectedRow();  
String data = (String) jTable2.getValueAt(row, col);
```

Trong đó jTable2 là tên của Jtable với số dòng số cột tương ứng, data là biến chứa dữ liệu của ô được chọn.

4.2.12.1. Một số hàm tạo trong JTable

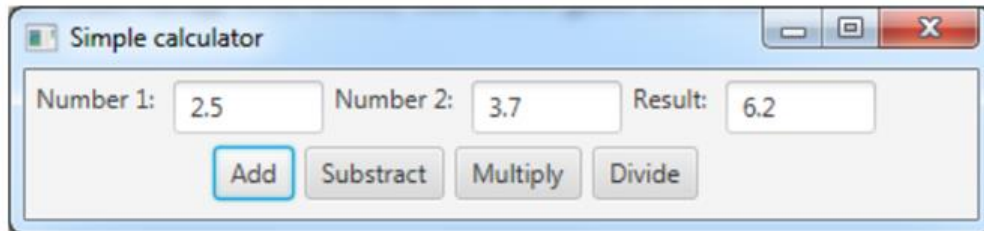
Tên hàm	Ý nghĩa
JTable()	Khởi tạo một JTable rỗng.
JTable(int rows, int cols)	Khởi tạo một JTable với số dòng và cột được chỉ định.
Jtable (Object[][] data, Object []Column)	Khởi tạo một JTable với các tên cột được chỉ định trong Object[] Column và dữ liệu trong Object[][] data.

4.2.12.2. Một số phương thức thường dùng

Tên hàm	Ý nghĩa
addColumn(TableColumn column)	Thêm một cột vào cuối bảng.
editCellAt(int row, int col)	Chỉnh sửa ô giao nhau của cột số col và hàng số row, nếu các chỉ số đã cho hợp lệ và ô tương ứng có thể chỉnh sửa được.
setValueAt(Object value, int row, int col)	Cập nhật giá trị tạo ô có số cột là col và hàng row thành giá trị value mới.
clearSelection()	Bỏ chọn tất cả các cột và hàng trước đó đã được chọn.

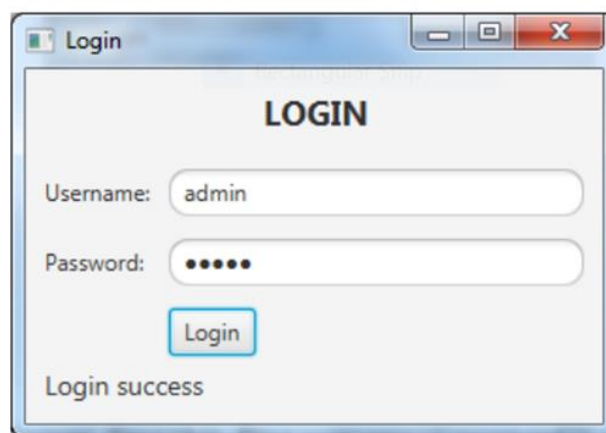
4.3. BÀI TẬP THỰC HÀNH

Câu 1: Viết chương trình thực hiện cộng (Add), trừ (Subtract), nhân (Multiply), chia (Divide) đơn giản như hình bên dưới:



A screenshot of a simple calculator application window titled "Simple calculator". It features two input fields for "Number 1" (containing 2.5) and "Number 2" (containing 3.7), and a "Result" field (containing 6.2). Below the input fields are four buttons: "Add", "Subtract", "Multiply", and "Divide". The "Add" button is highlighted with a blue border.

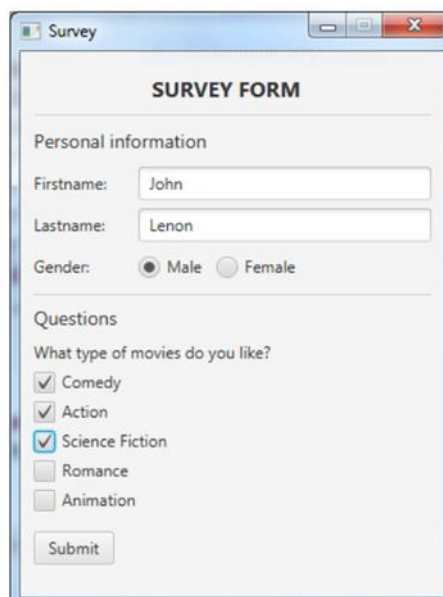
Câu 2: Thiết kế và xử lý giao diện đăng nhập như sau:



A screenshot of a login application window titled "Login". It has a title bar with standard Windows controls. The main content area has a title "LOGIN" in bold. Below the title are two input fields: "Username:" (containing "admin") and "Password:" (containing six dots). Below the password field is a "Login" button. At the bottom of the window, the text "Login success" is displayed.

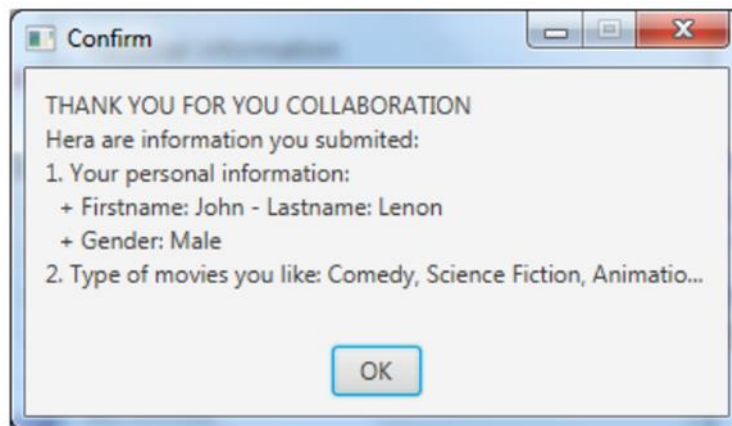
Yêu cầu xử lý: Khi người dùng nhập đúng username và password là admin thì hiển thị dòng thông báo “Login success”. Ngược lại, hiển thị “Login failed”.

Câu 3: Thiết kế và xử lý giao diện điều tra về sở thích phim của người dùng như sau:

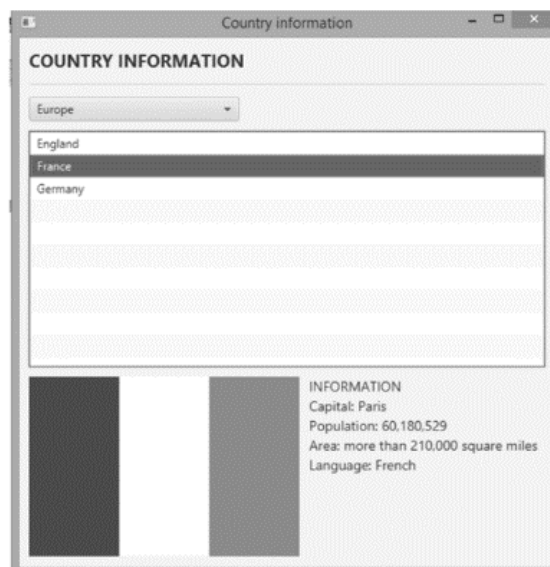


A screenshot of a survey application window titled "Survey". It has a title bar with standard Windows controls. The main content area has a title "SURVEY FORM" in bold. Below the title is a section titled "Personal information" with three input fields: "Firstname:" (containing "John"), "Lastname:" (containing "Lenon"), and "Gender:" (with radio buttons for "Male" and "Female", where "Male" is selected). Below this is a section titled "Questions" with the text "What type of movies do you like?". There are five checkboxes: "Comedy", "Action", "Science Fiction", "Romance", and "Animation". The "Science Fiction" checkbox is checked. At the bottom of the form is a "Submit" button.

Yêu cầu xử lý: Khi người dùng nhập, chọn đầy đủ thông tin và nhấn nút Submit thì chương trình hiển thị hộp thoại hiển thị về thông tin đã chọn như hình bên dưới.

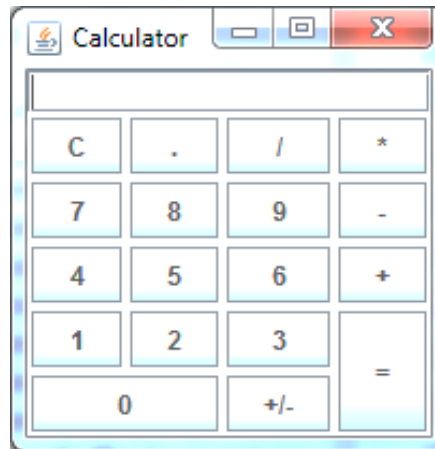


Câu 4: Thiết kế và xử lý giao diện xem thông tin quốc gia như sau:



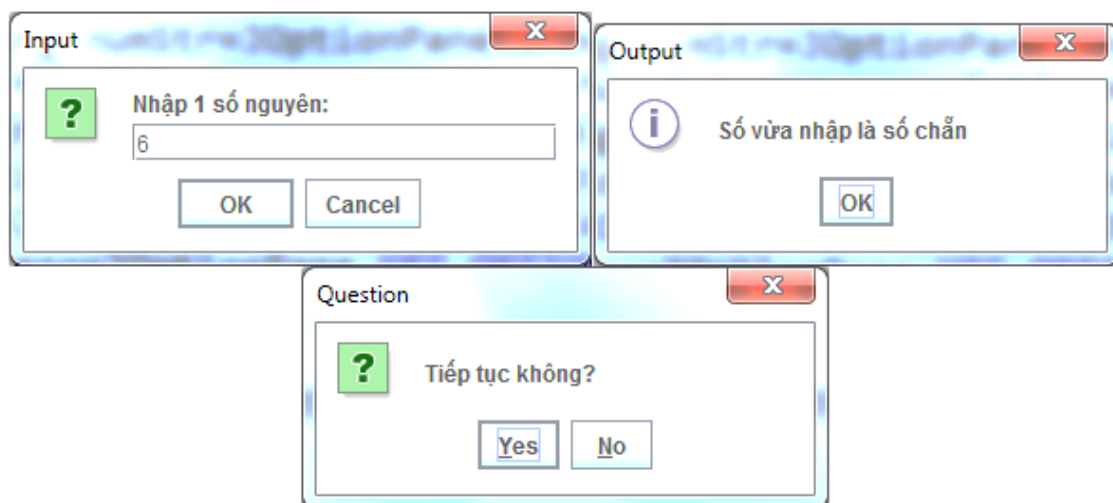
Yêu cầu xử lý: Người dùng chọn một châu lục trong ComboBox, sau đó hiển thị các nước tương ứng trong châu lục đó tại ListView. Cuối cùng, khi người dùng chọn tên một nước thì quốc kỳ và thông tin liên quan sẽ hiển thị ngay bên dưới.

Câu 5: Thiết kế và xử lý giao diện các chức năng cho máy tính bỏ túi như hình bên dưới:

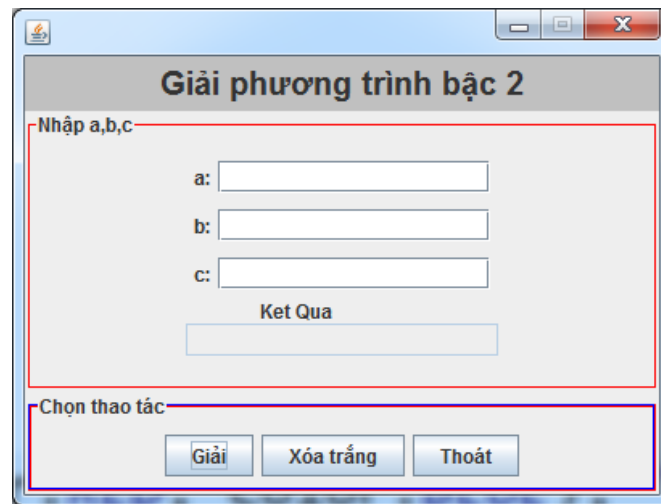


Yêu cầu xử lý: Sử dụng GridBagLayout và GridBagConstraints

Câu 6: Thiết kế và xử lý giao diện cho nhập vào một số nguyên dương, sử dụng Dialog boxes – JOptionPane để thông báo số chẵn hay số lẻ khi nhập.



Câu 7: Thiết kế và xử lý giao diện cho bài toán giải phương trình bậc 2



Câu 8: Thiết kế và xử lý giao diện cho bài toán tính chỉ số khối cơ thể (Body mass index – BMI) như hình bên dưới:



Yêu cầu xử lý:

- Nút **Tính**: chỉ số BMI được tính dựa vào công thức:

$$\text{BMI} = \text{cân nặng} / (\text{chiều cao} * \text{chiều cao})$$

Khi nhấn nút **Tính**, giá trị BMI sẽ được hiển thị trong ô Kết quả; đồng thời đưa ra Lời khuyên cho người dùng để có thể có chế độ ăn uống hợp lý. Lời khuyên được dựa vào form Bảng Giá Trị.

- Nút **Bảng Giá Trị**: liên kết đến form thể hiện cách tính bảng chỉ số BMI như hình:

BMI	LỜI KHUYÊN
BMI < 18.5	Bạn quá gầy
BMI >= 18.5 && BMI < 25.0	Bạn bình thường
BMI >= 25.0 && BMI < 30.0	Bạn hơi bị ngon
BMI > 30.0	Bạn mập quá rồi á

Trở về Thoát

- Nút **Nhập lại**: Xóa rỗng nội dung trong các JTextFields. Đặt con trỏ mặc định vào ô Chiều cao.
- Nút **Thoát**: Bật hộp thoại xác nhận để thoát chương trình.
- Bẫy lỗi các trường hợp có thể xảy ra.

Câu 9: Thiết kế và xử lý giao diện cho bài toán tính tiền điện như hình bên dưới:

Chỉ số cũ:

Chỉ số mới:

Tiêu thụ điện:

Tổng tiền:

Đăng kí Tính Bảng phụ Xóa Thoát

- Nút **Tính**: giá trị tiền điện được tính theo công thức

Tiêu thụ điện = Chỉ số mới – Chỉ số cũ

Tổng tiền = (Chỉ số mới – Chỉ số cũ)* Bảng giá tiền điện

- Nút **Bảng phụ**: liên kết đến form tra cứu bảng giá tiền điện như hình :



Bậc	Mức tiêu thụ(Kw/h)	Giá tiền (đồng)
1	0 - 50	1.678
2	50 - 100	1.734
3	100 - 200	2.014
4	200 - 300	2.536
5	300 - 400	2.834
6	400 trở lên	2.927

+ Nút Quay lại : Quay trở về form chính

+ Nút Thoát : Bật hộp thoại xác nhận để thoát chương trình

- Nút **Đăng kí**: liên kết form cho phép người dùng đăng ký sử dụng điện như sau:



+ Nút **Xác Nhận**: Bật ra hộp thoại xác nhận thông tin khách hàng vừa đăng ký.

+ Nút **Quay Lại**: Quay trở về form chính

+ Nút **Thoát** : Bật hộp thoại xác nhận để thoát chương trình

- Nút **Xóa** : Xóa rỗng nội dung trong các JTextFields. Đặt con trỏ mặc định vào ô Chỉ số cũ.
- Nút **Thoát** : Bật hộp thoại xác nhận để thoát chương trình
- Bẫy lỗi các trường hợp có thể xảy ra.

Câu 10: Thiết kế và xử lý giao diện cho bài toán quản lý sinh viên như sau :

Yêu cầu xử lý:

- Nút **Kết quả** : ghi điểm trung bình và kết quả của sinh viên đang nhập vào các JTextFields. Chú ý, ô nhập điểm phải là số.
- Nút **Thêm**: thêm một sinh viên vào bảng với các cột như hình. Chú ý không được thêm khi: Không nhập đủ dữ liệu; Điểm không phải là số; Trùng mã sinh viên.
- Nút **Reset**: xóa rỗng nội dung trong các JTextFields và JComboBox mã lớp được thiết lập lại là “CNTT”.
- Nút **Lưu**: lưu danh sách sinh viên vào file. Lưu ý, khi chương trình bắt đầu thực hiện, nếu tồn tại file dữ liệu đã lưu trước đó, thì nạp danh sách sinh viên lên trên bảng.
- Nút **Tìm** : Cho phép người sử dụng nhập mã sinh viên cần tìm vào một JOptionPane. Nếu tìm thấy, nạp thông tin vào các Components bên trái giao diện. Nếu không thấy, thông báo nội dung trong một JOptionPane.
- Nút **Xóa**: Cho phép người sử dụng nhập mã sinh viên cần xóa vào một JOptionPane. Nếu tìm thấy, hỏi nhắc người sử dụng trước khi xóa. Ngược lại, thông báo không tồn tại trong một JoptionPane
- Nút **Sửa**: Cho phép người sử dụng sửa thông tin sinh viên. Lưu ý, không được sửa mã số sinh viên.

TÀI LIỆU THAM KHẢO

- [1]. Đoàn Văn Ban, Đoàn Văn Trung, *Giáo Trình Lập trình Java*, NXB Giáo Dục, 2011.
- [2]. Huỳnh Công Pháp, *Bài Tập Lập Trình Java Cơ Bản*, NXB Thông Tin và Truyền Thông, 2014.
- [3]. Trần Đình Quế, Nguyễn Mạnh Hùng, *Lập Trình Hướng Đối Tượng*, Học Viện Công Nghệ Bưu Chính Viễn Thông, 2014.
- [4]. Đậu Quang Tuấn, Nguyễn Viết Linh, *Học Nhanh Kỹ Thuật Lập Trình Java*, NXB Trẻ, 2001.
- [5]. Atiwong Suchato, *Learning Computer Programming Using Java with 101 Examples*, Department of Computer Engineering Faculty of Engineering, Chulalongkorn University, 2011.
- [6]. Cay S. Horstmann, *Core Java Volume I*, Prentice Hall, 2016.
- [7]. David J. Eck, *Introduction to Programming Using Java*, Department of Mathematics and Computer Science Hobart and William Smith Colleges, 2014.
- [8]. Herbert Schildt, *Java: The Complete Reference*, 9th Edition, McGraw-Hill Education, 2014.
- [9]. Paul Deitel, Harvey Deitel, *Java How To Program (Early Objects)*, Pearson, 2015.
- [10] Y.Daniel Liang, *Introduction To Java Programming 10 Edition*, Prentice Hall, 2015.